台灣自編大學教科書

地圖建構與同步定位 (SLAM)

Simultaneous Localization and Mapping



大學教科書

第八篇

第28–31章

ROS ／ Python ／ C++ 實作

李世淵 (機器人叫獸)

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第八篇 ROS / Python / C++ 實作

第 28 章

SLAM 程式設計環境

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

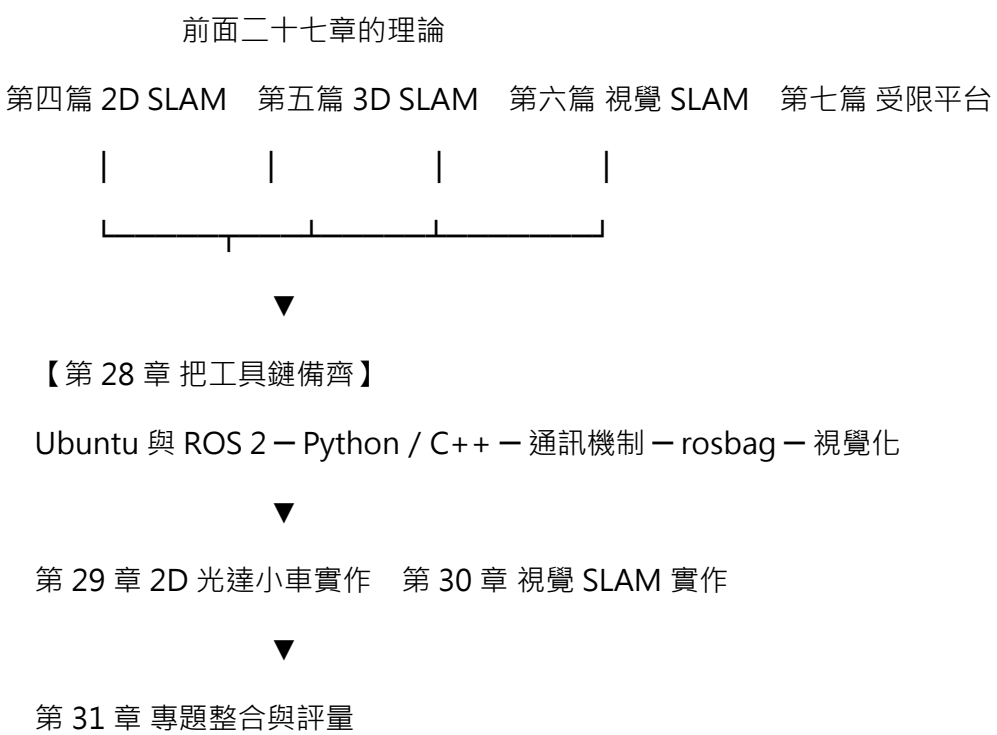
本章在全書的位置

前面二十七章講的都是「該怎麼做」。從本章起，第八篇要處理「怎麼真的跑起來」。

這一篇的性格與前面完全不同。它不談新的演算法，而談那些會讓專題卡上三天的事情：版本對不上、話題沒有資料、座標轉換樹斷掉、錄了兩小時的資料發現硬碟滿了。這些問題在論文裡不會出現，卻佔掉實作時間的一大半。

本章的三張圖都是真的跑出來的量測，而不是從規格表抄來的數字：圖 28-1 用同一段掃描配準的核心，分別以純 Python、NumPy 與 C++ 實作並計時；圖 28-2 用真實的行程間通訊量出「話題」與「服務」兩種模式的差別；圖 28-3 則產生逼真的感測資料，實測錄製一小時要多少空間。

圖 28-0　第 28 章的連結地圖



學習目標

前七篇建立了完整的理論與系統知識。從本篇開始，我們回到最實際的層面：如何把這一切真正跑起來。本章是第八篇的入口，負責把開發環境備妥、把工具鏈打通——這是所有後

續實作的地基。學完本章之後，同學應能夠達成下列目標：

1. 說明 Ubuntu 與 ROS 2 的版本對應關係，並完成環境安裝。
2. 說明 Python 與 C++ 在 ROS 2 中的分工，並理解 CMake 與建置系統的角色。
3. 說明話題、服務、動作三種通訊機制的差異與適用場合。
4. 使用 rosbag2 錄製與重播感測資料，並理解 MCAP 格式的意義。
5. 使用 RViz2、Gazebo 與 Foxglove 進行視覺化、模擬與除錯。
6. 說明閱讀 GitHub 開源專案的系統性方法。
7. 規劃從高中積木式實作到大學原始碼實作的學習路徑。

核心概念

中文名詞	英文名詞	概念說明
長期支援版	LTS	有較長維護期限的穩定版本，適合教學與產品。
工作空間	Workspace	存放 ROS 套件原始碼與建置產物的目錄結構。
套件	Package	ROS 中程式碼組織的基本單位。
建置工具	colcon	ROS 2 用來編譯工作空間中所有套件的工具。
節點	Node	執行特定功能的獨立程式（第 3 章）。
話題	Topic	節點間以發布訂閱模式交換資料的通道。
服務	Service	一問一答式的同步通訊機制。
動作	Action	可回報進度、可取消的長時間任務通訊機制。
啟動檔	Launch File	一次啟動多個節點並設定參數的腳本。
資料錄製	rosbag2	錄製與重播 ROS 話題資料的工具。
訊息擷取格式	MCAP	自我描述、可快速隨機存取的記錄檔格式。
視覺化工具	RViz2	ROS 2 內建的三維視覺化工具。

中文名詞	英文名詞	概念說明
模擬器	Gazebo	提供物理模擬與感測器模擬的環境。

28.1　Ubuntu 與 ROS / ROS 2

第 3.5 節介紹過 ROS 的角色與核心概念，本節聚焦環境建置的實務。

為什麼是 Ubuntu

ROS 2 支援多種作業系統，但 Ubuntu 是所謂的第一級平台（Tier 1）——也就是官方主要開發與測試的平台。實務上，絕大多數開源 SLAM 專案的說明文件、相依套件與社群支援都以 Ubuntu 為前提。因此本書的實作一律以 Ubuntu 為基準。

版本對應：最容易踩的第一個坑

ROS 2 與 Ubuntu 的版本是綁定的——每個 ROS 2 發行版只支援特定的 Ubuntu 版本。裝錯版本會導致一連串難以排查的相依性錯誤，這是初學者最常見的挫折來源。

ROS 2 每年 5 月發布新版本，偶數年為長期支援版（LTS）。截至本書撰寫時，主要的版本對應如下：

ROS 2 發行版	發布年	Ubuntu 版本	類型	備註
Humble Hawksbill	2022	22.04	LTS	支援至 2027 年 5 月，最成熟穩定
Jazzy Jalisco	2024	24.04	LTS	rosbag2 預設改用 MCAP 格式
Kilted Kaiju	2025	24.04	一般版	rosbag2 效能改善
Lyrical Luth	2026	26.04	LTS	最新的長期支援版

表 28-1　ROS 2 發行版與 Ubuntu 版本對應

版本選擇建議

- 教學與初學：建議選擇成熟的 LTS 版本。Humble 是目前教學資源最豐富的選擇——網路上絕大多數教學、開源專案與問答都以它為基準，遇到問題最容易找到答案。

- 新專案：若要長期維護，可選最新的 LTS 版本以獲得較長的支援期。
- 沿用既有專案：直接對齊該專案指定的版本，不要嘗試「順便升級」——這往往會浪費大量時間在相依性問題上。
- 重要提醒：全班應統一版本。不同版本之間的錄製資料未必能互通，教學時版本不一致會造成極大困擾。

安裝方式的三種選擇

1. 原生安裝：直接在實體電腦上安裝 Ubuntu 與 ROS 2。效能最好、硬體支援最完整，是連接實體機器人時的必要選擇。缺點是需要重灌或雙系統。
2. 虛擬機：在 Windows 或 macOS 上以虛擬機執行 Ubuntu。安裝容易、不影響原系統，適合純軟體的模擬練習。缺點是 3D 加速效能較差（Gazebo 可能會卡）、USB 裝置連接較麻煩。
3. 容器（Docker）：以容器封裝完整環境。優點是環境一致、可快速重建、不污染主系統——這對教學特別有價值，教師可以提供統一的容器映像，免去全班各自安裝的混亂。缺點是需要額外設定才能使用圖形介面與 USB 裝置。

教學上的實務建議

依多年教學經驗，建議的策略是：高中與大學部入門階段使用容器或虛擬機，讓所有同學在一致的環境中操作模擬；進入實體機器人實作階段，再改用原生安裝。這能把「環境問題」的干擾降到最低，讓課堂時間花在 SLAM 本身而非除錯安裝。

28.2 Python、C++、CMake

ROS 2 同時支援 Python 與 C++，兩者在 SLAM 開發中有清楚的分工。

兩種語言的分工

面向	Python(rclpy)	C++(rclcpp)
開發速度	快，適合快速原型	較慢，需編譯
執行效能	較慢	快，適合高頻運算
適合的任務	控制邏輯、資料分析、視覺化	SLAM 核心、影像處理、即時控制
學習門檻	低	較高（記憶體管理、模板）
本書用途	實作練習、資料處理	閱讀開源 SLAM 原始碼

表 28-2　Python 與 C++ 在 ROS 2 中的分工

一個實用的原則是：先用 Python 把邏輯跑通，若效能不足再用 C++ 重寫關鍵部分。同學會發現主流的 SLAM 系統（ORB-SLAM3、Cartographer、LOAM 系列）幾乎都是 C++ 寫的——因為它們需要處理高頻的影像與點雲資料，Python 的效能無法勝任。

CMake 與建置系統

C++ 專案需要編譯，而管理編譯過程的工具就是 CMake。它負責：找出相依的函式庫在哪裡、決定編譯順序、產生實際的編譯指令。

對 SLAM 學習者而言，CMake 最常出現的場合是「編譯開源專案時出錯」。第 22.7 節提過 ORB-SLAM2 在新系統上可能編不過，問題往往就出在 CMakeLists.txt 中指定的函式庫版本與系統上的版本不符。

- 必備技能：看懂 CMakeLists.txt 中的 find_package（尋找相依套件）與 target_link_libraries（連結函式庫）兩個指令，就能解決大部分編譯問題。
- 常見錯誤：找不到套件（需安裝或指定路徑）、版本不符（需修改版本要求或安裝對應版本）、C++ 標準不符（需調整編譯選項）。

工作空間與 colcon

ROS 2 的程式碼組織成「工作空間」，其中放置多個「套件」。建置工具 colcon 會自

動分析套件之間的相依關係，依正確順序編譯全部套件。

典型的工作流程是：建立工作空間目錄 → 把套件原始碼放進 src 子目錄 → 執行 colcon 建置 → 載入環境設定（source）→ 執行節點。這個流程同學會在整個第八篇反覆使用。

環境設定的常見陷阱

- 忘記 source：建置完成後必須載入環境設定，系統才找得到新編譯的套件。這是最常見的「明明編譯成功卻找不到節點」原因。
- 多個工作空間疊加：若同時有多個工作空間，載入順序會影響哪個版本的套件被使用。出現詭異行為時，先檢查環境變數。
- 修改程式後未重新建置：Python 節點在某些設定下可以直接生效，C++ 則一定要重新編譯。這個差異常讓初學者困惑。

量一量：Python 與 C++ 到底差多少

表 28-2 建議「Python 做原型與整合、C++ 做效能關鍵」。這個分工是對的，但「效能關鍵」的界線在哪裡？本小節用一段真實的程式碼把它量出來。

測試的對象是掃描配準的內層迴圈——把 N 個點依位姿變換後，查距離場並累加似然值（第 17 章的似然場評分，第 19、25 章的實驗都用過）。同一段邏輯寫三個版本：純 Python 迴圈、NumPy 向量化、以及 C++（g++ 加 -O2 最佳化），在同一台機器上計時。

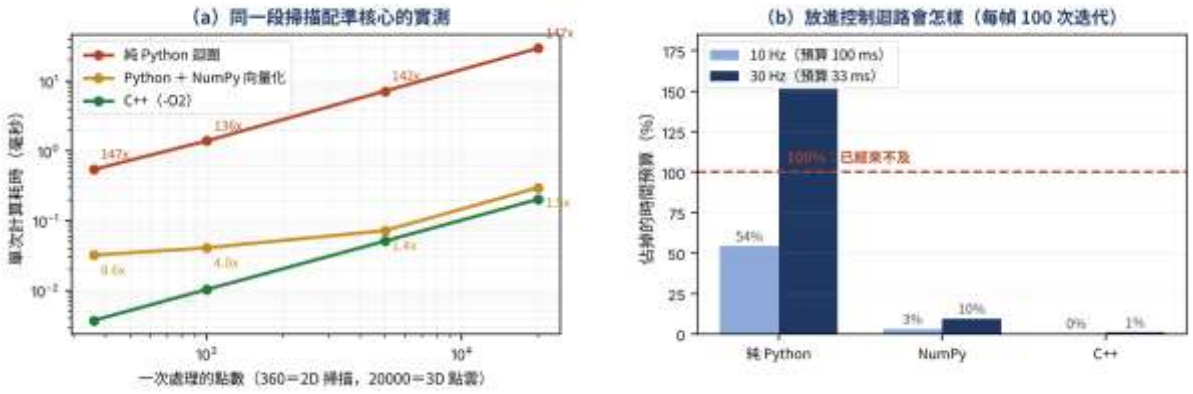


圖 28-1　同一段運算在三種語言下的實測效能

（a）是單次計算的耗時，兩軸都是對數刻度。以 360 點（典型的 2D 光達掃描）為例：C++ 是 0.0037 毫秒、NumPy 是 0.0320 毫秒、純 Python 是 0.5431 毫秒。純 Python 比 C++ 慢了 147 倍。

但真正有用的資訊是兩條紅、黃線的「相對位置會變」。純 Python 與 C++ 的倍數幾乎固定在 136 到 147 倍之間，不隨點數改變；NumPy 與 C++ 的差距卻從 360 點時的 8.6 倍，一路縮小到 20000 點（3D 點雲等級）時的 1.5 倍。

原因是 NumPy 的成本有兩塊：一塊是每次呼叫的固定開銷（型別檢查、記憶體配置），另一塊才是實際的運算。點數少的時候固定開銷佔大半，所以看起來慢；點數多的時候固定開銷被攤薄，效能就逼近 C++。這給了一條很實用的判準：NumPy 適合「一次處理一大批」，不適合「每次處理一點點但呼叫很多次」。

（b）把這些數字放回真實的控制迴路。假設掃描配準每一幀要做 100 次迭代（這是很常見的設定）。在 10 Hz（每幀 100 毫秒預算）下，純 Python 佔掉 54%、NumPy 3%、C++ 0.4%。54% 聽起來還能用，但那表示只剩一半的時間給導航、通訊與其他所有事。

到了 30 Hz（每幀 33 毫秒），純 Python 需要 163% 的時間預算——也就是根本來不及，系統會開始掉幀。NumPy 是 10%、C++ 是 1%，都還游刃有餘。這張圖給出的實務界線很清楚：純 Python 的迴圈只適合離線分析與原型驗證；一旦要進入即時迴路，最低限度是向量化。

什麼時候真的需要 C++

圖 28-1(b) 顯示 NumPy 在 30 Hz 下只佔 10% 的預算——對絕大多數教學與專題用途，這已經夠了。所以「一定要用 C++」這個想法，多半是過慮。

真正需要 C++ 的是三種情況：一是迭代次數遠超過 100 次（例如大規模的束調整）；二是運算無法向量化（例如有分支的樹狀搜尋、增量式的資料結構更新）；三是跑在第 25 章那種毫瓦級平台上，連 Python 直譯器都放不下。

務實的做法是：先全部用 Python 寫出能跑的版本，量出哪一段真的吃掉時間，只把那一段改寫成 C++。這與第 31 章的除錯順序是同一個原則——先量測，再優化。

28.3 ROS Topic、Service、Action

第 3.5 節簡介過這三種通訊機制，本節說明它們的差異與選用時機——這是設計 ROS 系統的核心判斷。

話題（Topic）：持續的資料流

話題採發布訂閱模式：發布者持續送出資料，訂閱者持續接收。雙方不需要知道彼此存在，只要約定好話題名稱與訊息格式。

- 特性：非同步、一對多、持續流動、不保證接收方收到。
- 適合：感測器資料（光達掃描、影像、IMU）、狀態廣播（位姿、地圖）、控制指令（速度）。
- SLAM 中的例子：光達驅動發布 /scan、SLAM 節點訂閱它並發布地圖（第 14.5 節的實作流程）。

服務（Service）：一問一答

服務是同步的請求回應模式：客戶端送出請求，等待伺服器回覆。

- 特性：同步、一對一、有明確的回覆、呼叫端會等待。

- 適合：短時間就能完成的查詢或設定，例如「取得目前地圖」「重設里程計」「切換模式」。
- 注意：因為呼叫端會等待，不適合用在耗時的操作——會阻塞整個節點。

動作（Action）：可監控的長任務

動作專為「需要花時間、且過程中想知道進度」的任務設計。它結合了三個元素：目標（要做什麼）、回饋（進行到哪了）、結果（最後如何）。

- 特性：非同步、可回報進度、可中途取消。
- 適合：導航到某個目標點、執行一段軌跡、完成一次掃描任務。
- SLAM 中的例子：第 29 章的 Nav2 導航就是以動作介面提供——你送出「去這個座標」的目標，系統持續回報剩餘距離，你也可以隨時取消。

機制	通訊模式	是否等待	可否取消	典型用途
話題	發布訂閱，一對多	不等待	不適用	感測器資料、狀態、指令
服務	請求回應，一對一	等待回覆	不可	快速查詢與設定
動作	目標回饋結果	不等待，可監控	可以	導航、長時間任務

表 28-3　三種通訊機制的比較

選用的判斷流程

遇到「這個功能該用哪種機制」時，依序問三個問題：

1. 這是持續流動的資料嗎？是 → 話題。
2. 是不是很快就能回答的一次性請求？是 → 服務。
3. 是不是耗時且需要監控進度的任務？是 → 動作。

這個判斷看似簡單，卻是 ROS 系統設計品質的關鍵。用錯機制（例如用服務執行耗時

的導航）會導致系統阻塞、難以除錯——這是實作時值得留意的細節。

量一量：話題與服務的代價差在哪

表 28-3 比較了三種通訊機制的特性，其中最關鍵的一欄是「是否阻塞」。這一欄看起來只是個技術細節，但它對系統設計的影響比其他所有欄位加起來還大。本小節把它量出來。

實驗方法：用真實的行程間通訊（兩個獨立的作業系統行程，以管線連接），傳送 4 KB 的訊息。話題模式是「發了就走」，不等回應；服務模式是「送出請求後等待回覆」，並讓伺服端花掉不同長度的處理時間。量測呼叫端實際被擋住多久，以及每秒能完成幾次。

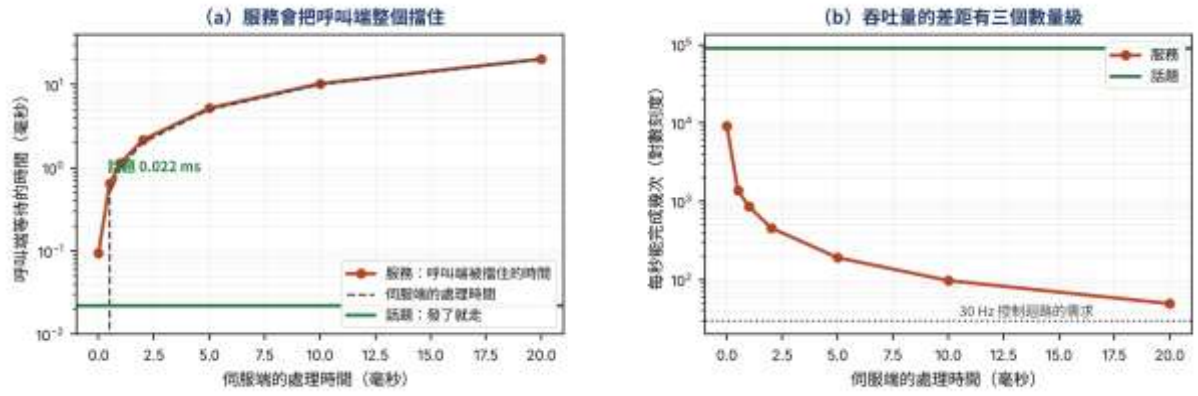


圖 28-2　話題與服務的延遲與吞吐量實測

（a）的綠線是話題模式：延遲 0.022 毫秒，而且不論伺服端在忙什麼都不會改變——因為發送端根本不等。紅線是服務模式，它幾乎完全貼著灰色虛線（伺服端的處理時間）：伺服端花 5 毫秒，呼叫端就被擋 5.2 毫秒；花 20 毫秒，就被擋 20.3 毫秒。

這條線看起來理所當然，卻是最容易被忽略的陷阱。很多同學會在控制迴路裡呼叫一個服務去「問一下目前的地圖」或「請求規劃一條路徑」，然後納悶為什麼機器人的反應變得遲鈍——因為整個迴路在等那個服務回來。

（b）把它換算成吞吐量，縱軸是對數刻度。話題模式每秒可以送 89,862 則；服務模式即使伺服端完全不做事也只有 9,101 次（差了十倍，純粹來自一來一回的往返成本）；伺服端花 20 毫秒時，只剩每秒 49 次。灰色點線標出 30 Hz 控制迴路的需求——伺服端的處理時間只

要超過約 30 毫秒，服務就跟不上了。

這就是「動作」這個機制存在的理由。動作把「送出請求」與「等待結果」拆開：呼叫端送出目標之後立刻回到自己的迴路，伺服端則週期性地回報進度，完成時再送出結果。它本質上是用話題的非阻塞特性，包裝出服務的「有來有回」語意。

三個選用的判準

依圖 28-2 的量測，選擇通訊機制時可以問三個問題：

一、下游需不需要回覆？不需要就用話題——它快了十倍以上，而且不會把上游卡住。感測資料、位姿、速度指令全部屬於這一類。

二、需要回覆，但很快就能回來嗎？若伺服端的處理時間遠小於控制週期（例如查一個參數、切換一個模式），服務是最簡單的選擇。

三、需要回覆，而且要花很久？那就必須用動作。導航到目標點、執行一段掃描、充電對接——這些動輒數秒到數分鐘的任務，用服務會把整個系統鎖死。

28.4 rosbag 資料錄製與重播

rosbag2 是 SLAM 開發最重要的工具之一，它讓「錄一次、分析無數次」成為可能。

為什麼 rosbag 如此重要

- 可重現性：同一段資料反覆重播，能公平比較不同演算法或參數——第 15 章實作 15-1 的「Cartographer vs Gmapping 對照實驗」正需要這個能力。
- 離線開發：資料錄好後，不需要機器人也能開發演算法。這對硬體有限的教學環境極為實用。
- 除錯：出問題時把資料錄下來，可以反覆重播找出原因，不必在現場一次次重跑實驗。
- 資料集建立：第 26.5 節建議的「自製場域資料集」，正是以 rosbag 錄製的。

MCAP：新一代的記錄格式

rosbag2 的儲存格式近年有重要演進。MCAP（Message Capture）是來自 Foxglove 的記錄容器格式，自 ROS 2 Jazzy 起成為 rosbag2 的預設儲存格式。

它的特性包含：經過索引以支援快速隨機存取、內嵌訊息定義因而具自我描述性，並支援 ROS、protobuf、JSON、flatbuffers 等多種格式。

對同學而言，「自我描述」這一點特別重要：記錄檔本身就包含了訊息格式的定義，因此即使在沒有安裝對應訊息套件的電腦上，也能正確讀取——這大幅改善了資料交換與長期保存的可行性。

錄製的實務要點

1. 錄什麼：SLAM 開發至少要錄感測器原始資料（/scan 或影像）、IMU、里程計、以及 TF 座標轉換。TF 常被遺漏，但缺少它重播時什麼都對不上。
2. 不要錄什麼：輸出結果（地圖、估測軌跡）通常不需要錄，因為重播時會重新產生。錄了反而佔空間並可能造成混淆。
3. 時間標記：確認所有資料的時間戳記正確——這是第 5、10、27 章反覆強調的工程細節，在重播時尤其關鍵。
4. 檔案大小：影像與點雲資料量龐大，一分鐘可能就數 GB。錄製前先估算並確認儲存空間充足。

重播的常見問題

實務上常見的三個問題值得同學留意：ROS 版本不符——在某個版本錄製的資料可能無法在另一版本順暢重播；缺少 TF 轉換——許多資料集缺少轉換資訊，導致 RViz 視覺化失效；以及話題名稱不符——若你的演算法預期 /scan 但資料中是 /lidar_scan，就必須做話題重新對應。

第三點特別實用：重播時可以「改名」，讓錄好的資料餵給預期不同話題名稱的演算法，

不必修改程式碼。這是整合他人資料集時的常用技巧。

量一量：錄一小時要多大

28.4 節說明了 rosbag 的價值，但沒談它的代價。這一節把代價算出來，因為「硬碟滿了」是實驗現場最常見的意外之一。

實驗方法：產生逼真的合成感測資料（不是隨機雜訊——隨機資料不會壓縮，會嚴重低估壓縮率），包含平滑牆面加不連續的 2D 光達掃描、16 線的 3D 點雲、帶紋理與幾何結構的 RGB 影像、平滑的深度圖，以及 IMU。量測每種資料的原始位元率，以及用一般壓縮（zlib）與影像專用編碼（JPEG）之後的位元率。

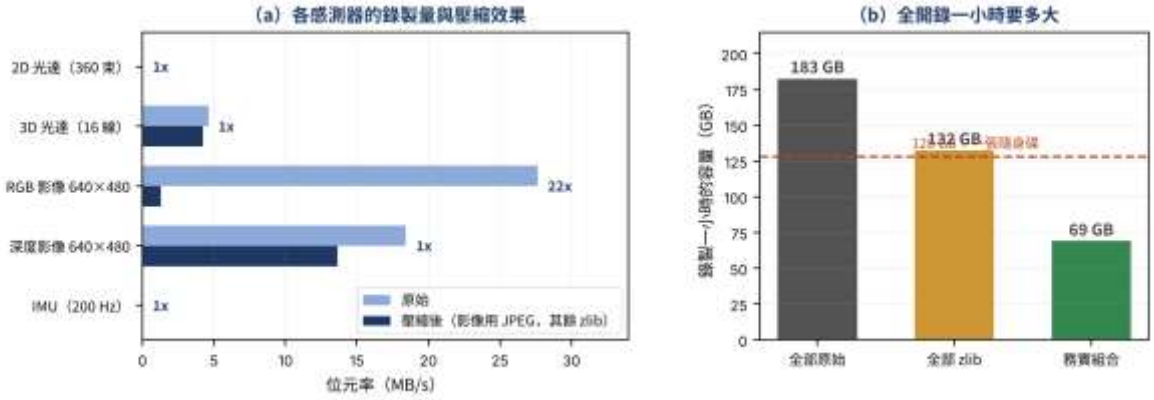


圖 28-3　各感測器的錄製量與壓縮效果

（a）第一個發現是量級的懸殊。2D 光達每秒只有 0.01 MB，IMU 也是 0.01 MB——這兩種資料可以錄上好幾天都不成問題。而 RGB 影像是每秒 27.65 MB、深度影像 18.43 MB、3D 點雲 4.61 MB。單一台 RGB-D 相機就佔掉了全部資料量的九成以上。

第二個發現更重要：一般壓縮對感測資料幾乎沒用。3D 點雲的壓縮率只有 1.1 倍、深度影像 1.4 倍、RGB 影像 1.5 倍。原因是這些資料本質上是高熵的浮點數或接近隨機的低位元，zlib 找不到可以利用的重複樣式。IMU 甚至出現壓縮後變大的情況（0.8 倍），因為訊息太小，壓縮格式的標頭反而佔了便宜。

真正有效的是領域專用的編碼。同一批 RGB 影像用 JPEG（品質 85）壓縮，位元率從 27.65

MB/s 降到 1.28 MB/s——22 倍。這個對比說明了一件事：壓縮的效果取決於編碼器懂不懂資料的結構，而不是取決於壓縮等級調多高。

（b）把它換算成實際的容量。所有感測器全開、完全不壓縮，錄一小時是 183 GB；全部套上 zlib 是 132 GB，只省下 28%；而「影像用 JPEG、其餘用 zlib」的務實組合是 69 GB，只有原始的 38%。紅色虛線是 128 GB——一支常見隨身碟的容量。也就是說，不做取捨的話，一支隨身碟連一小時都錄不滿。

錄製前的三個決定

依圖 28-3 的數字，規劃錄製時應該先做三個決定。

一、哪些話題真的需要錄？多數 SLAM 的離線分析只需要光達、IMU 與里程計——那加起來不到每秒 5 MB，一小時只有 17 GB。影像除非要做視覺 SLAM 或語意標註，否則不必全程錄。

二、影像要用什麼格式？壓縮影像話題（JPEG）與原始影像話題差了二十倍。若是為了之後跑特徵點法，JPEG 品質 85 以上通常足夠；若是要做光度法（第 20.2 節的直接法），就必須錄原始影像。

三、要不要降頻？30 fps 的影像對多數離線分析是浪費，10 fps 往往就夠了，這一刀直接砍掉三分之二。

28.5　RViz、Gazebo、Foxglove

本節介紹三個貫穿第八篇的工具：視覺化、模擬與除錯分析。

RViz2：看見機器人的世界

RViz2 是 ROS 2 內建的三維視覺化工具，也是本書從第 10 章開始就反覆提到的除錯利器。它能同時顯示：

- 感測器資料：光達掃描點、點雲、影像。

- 座標框架：TF 樹的各個座標軸（第 10 章「地圖歪了先看 TF」的檢查方法）。
- SLAM 輸出：佔據格地圖、估測軌跡、粒子雲（第 11 章案例 11-2）、關鍵幀與共視圖（第 22 章）。
- 導航資訊：規劃路徑、代價地圖、目標點（第 29 章）。

近年 RViz2 也持續改進，例如加入基於正規表示式的 TF 過濾器讓你能依名稱顯示或隱藏座標轉換、話題頻率會自動顯示在狀態列、以及可訂閱 point_cloud_transport 話題等功能。

Gazebo：沒有機器人也能實驗

Gazebo 提供物理模擬與感測器模擬，讓同學能在虛擬環境中完整驗證 SLAM 演算法。本書從第 8 章就一再建議「先模擬、再便宜硬體、後追求精度」，Gazebo 正是這條路徑的第一站。

- 模擬什麼：機器人的運動學與動力學、光達與相機的量測（含雜訊）、環境的幾何與碰撞。
- 教學價值：每位同學都能有一台「自己的機器人」，不受硬體數量限制；可以任意設計場景（長走廊、玻璃隔間）來重現第 13、14 章討論的各種困難情境；也能安全地做危險實驗。
- 版本搭配：ROS 2 從 Jazzy 起，官方會為每個 ROS 版本推薦特定的 Gazebo 版本，並提供 Gazebo 供應商套件讓使用者能快速安裝上手。安裝時務必依官方建議的搭配，以免相容性問題。

Foxglove：更強大的除錯與分析

Foxglove Studio 是由前自駕車公司工程師於 2021 年創辦的公司所開發的多協定視覺化工具，同時支援桌面、網頁與雲端，並支援 ROS 1、ROS 2、MCAP、MAVLink、MQTT 與原始 WebSocket，提供 3D、繪圖、影像、狀態轉換、地圖等多種面板。

它之所以受到重視，是因為 RViz2 雖然功能足夠但有其限制：它依賴 ROS，難以檢視非 ROS 的系統；記錄檔分析能力較弱；難以協作或分享（只能靠截圖與錄影）；且沒有網頁或平板的支援。

- 對教學的價值：網頁版讓同學不必安裝就能檢視資料；能分享連結給老師或同學，方便討論與批改。
- 對研究的價值：強大的繪圖與時間序列分析，適合做第 22.7 節的軌跡誤差分析。
- 建議：RViz2 用於即時的三維視覺化（執行時看機器人在做什麼），Foxglove 用於事後的資料分析（找出哪裡出了問題）。兩者互補而非取代。

28.6 GitHub 開源專案閱讀方法

SLAM 領域的一大幸運，是幾乎所有重要系統都開源。但面對一個有數萬行程式碼的專案，同學常不知從何下手。本節提供一套系統性的閱讀方法。

五個步驟

1. 先讀論文，再讀程式：這是最重要的建議。ORB-SLAM3、Cartographer、LOAM 都有對應的論文，先理解演算法架構（如本書第 15、18、22、23 章所做的），再看程式碼會事半功倍。直接跳進程式碼往往會迷失在細節中。
2. 讀 README 與文件：確認相依套件、支援的平台版本、以及執行範例的指令。特別注意硬體需求——第 18 章案例 18-3 提過，許多同學卡關的原因是硬體規格不符，而這在文件中往往寫得清清楚楚。
3. 先跑起來，再讀程式：用官方提供的資料集把系統跑通，建立成功經驗。能跑之後再讀程式碼，你會有具體的參照——「剛剛畫面上那條軌跡，是這段程式產生的」。
4. 從進入點往下追：找到 main 函式或 ROS 節點的進入點，順著資料流往下讀。不要企圖一次讀懂全部，而是追蹤「一幀資料進來後，依序經過哪些函式」。

5. 加輸出訊息驗證理解：在關鍵位置加上輸出訊息，執行後觀察它們的觸發時機與頻率。第 22 章實作 22-1 的延伸挑戰正是這個方法——追蹤 ORB-SLAM2 在什麼情況下切換三種追蹤策略。

閱讀時的重點對照

讀 SLAM 專案時，建議依本書的架構去找對應的模組：

- 前端在哪：特徵提取或掃描匹配的程式（第 13、20 章）。
- 後端在哪：呼叫最佳化函式庫的程式（第 12 章的 g2o、Ceres、GTSAM）。
- 閉環在哪：地點辨識與驗證的程式（第 22、23 章）。
- 資料結構：關鍵幀、地圖點、位姿圖如何定義（第 22.6 節）。

把程式碼的模組對應到本書的章節，你會發現理論與實作的對應關係非常清楚——這也是本書刻意以「架構」而非「數學細節」為主軸的原因。

善用社群資源

- Issues：專案的 issue 區是寶庫。你遇到的問題，很可能別人已經問過並得到解答。搜尋錯誤訊息往往能直接找到答案。
- Fork 與分支：原始專案可能已停止維護，但社群常有維護中的分支（第 22.7 節提過的建議）。留意 star 數較多、近期有更新的 fork。
- 貢獻回饋：若你解決了一個未被記錄的問題，把解法寫成 issue 或 pull request 回饋社群。這是參與開源生態最直接的方式，也是很好的履歷素材。

28.7 從高中積木式實作到大學原始碼實作

本章最後，規劃一條從高中到大學、乃至研究所的完整學習路徑。這也是全書教學設計的總綱。

階段	目標	主要工具	本書章節
一：觀察體驗	理解 SLAM 在做什麼	模擬器、掃地機器人觀察	第 1–3、16 章
二：積木式操作	會用現成系統建圖導航	Gazebo、啟動檔、參數調整	第 14、15、29 章
三：資料與分析	會錄資料、評估品質	rosbag2、Python、evo	第 13.7、22.7 節
四：讀原始碼	理解系統如何實現	GitHub、除錯工具	第 22、23 章
五：修改與創新	能改良或整合系統	C++、CMake、演算法設計	第 30、31 章

表 28-4　SLAM 學習的五個階段

各階段的教學建議

- 階段一（高中適用）：不碰程式碼，重點是建立直覺。用模擬器操作、觀察家中掃地機器人（第 16.7 節）、做紙上活動（第 12、16、25 章的實作）。目標是讓同學「知道 SLAM 在解什麼問題、為什麼困難」。
- 階段二（高中進階/大學部）：使用現成系統，學會啟動、設定參數、觀察結果。重點是理解參數的意義與影響，而非撰寫程式。第 14、15 章的實作正是設計在這個層次。
- 階段三（大學部）：開始寫程式——但寫的是分析與工具程式，而非 SLAM 核心。錄製資料、計算誤差、繪製圖表。這個階段培養的是「用資料說話」的工程素養。
- 階段四（大學部後期/專題）：閱讀開源系統的原始碼，理解理論如何落實為程式。此時前面三階段的經驗會成為理解的支架。
- 階段五（專題/研究生）：修改既有系統或整合多個系統以解決特定問題。第 31 章的專題與第 32 章的 AI 整合，都屬於這個層次。

三個教學心法

1. 先成功、再深入：讓同學在每個階段都有可展示的成果。太早要求讀原始碼，容易造成挫折而放棄。
2. 刻意示範失敗：本書每一章都討論「常見失敗」，是有意的設計。看到系統在什麼情況下壞掉，比看它順利運作學到更多——第 14.6 節、第 26.5 節的挑戰測試都是這個精神。
3. 環境問題與學習內容分開：如 28.1 節建議，用容器或統一環境把安裝問題降到最低。課堂時間應該花在 SLAM，而不是排除相依性錯誤。

承先啟後

本章把工具鏈的三個代價量出來了：語言方面，同一段掃描配準核心，純 Python 比 C++ 慢 147 倍，在 30 Hz 迴路中會超出時間預算 63%，但 NumPy 在大批量時只比 C++ 慢 1.5 倍，在 30 Hz 下只佔 10% 的預算；通訊方面，話題的延遲是 0.022 毫秒、吞吐 89,862 則 / 秒，而服務會把呼叫端擋住伺服端的整段處理時間（伺服端 20 毫秒，呼叫端就等 20.3 毫秒，吞吐掉到每秒 49 次）；錄製方面，全部感測器不壓縮錄一小時是 183 GB，而一般壓縮只省 28%，只有影像專用編碼能帶來 22 倍的差距。

環境與工具至此備齊。下一章，我們把第四篇的 2D SLAM 理論真正跑起來：從硬體組裝、TF 設定、里程計發布，到 Gmapping 與 Cartographer 建圖、AMCL 定位、以及完整的導航堆疊。第 29 章將是本書第一個從頭到尾的完整系統實作。

28.8　工具鏈的三個實務原則

本章量了三件事，每一件都對應一個可以直接套用的原則。

一、先量測，再優化

圖 28-1 顯示 NumPy 在 30 Hz 控制迴路中只佔 10% 的時間預算。這表示大多數「我應該改寫成 C++ 嗎」的糾結是多餘的。正確的順序是：先用 Python 寫出能跑的版本，用計時器

量出哪一段真的慢，再只改寫那一段。

這個原則聽起來老生常談，但在 SLAM 專題裡特別重要，因為瓶頸的位置常常違反直覺。很多人以為瓶頸在矩陣運算，實際上往往在資料的複製、序列化，或是第 28.3 節那種被服務擋住的等待。

二、阻塞比慢更危險

圖 28-2 的核心訊息是：一個「慢」的模組只是拖慢自己，一個「阻塞」的呼叫卻會拖垮整個迴路。前者可以靠加快硬體解決，後者不行——它是架構問題。

這與第 33.8 節的「錯開執行」、第 34.5 節的「非同步設計」是同一個原則，也是為什麼本書在談每一個系統時都要先問「這一層的更新頻率是多少」。不同頻率的模組之間，介面必須是非阻塞的。

三、資料是會滿出來的

圖 28-3 顯示不做取捨時一小時就是 183 GB。這個數字對規劃實驗有直接的意義：一天的實驗若錄滿八小時，就是 1.5 TB。而這些資料還要備份、要傳輸、要長期保存。

務實的做法是分層：平時只錄輕量的話題（光達、IMU、里程計、位姿），需要做視覺分析時才開啟影像，而且降頻與壓縮。這呼應了第 19.6 節巡檢機器人的地圖維護與第 27 章的頻寬排序——在資源有限的系統裡，「選擇要留下什麼」本身就是一項設計工作。

量到的事	數字	實務原則
純 Python 的代價	比 C++ 慢 147 倍，30 Hz 下超出預算 63%	即時迴路至少要向量化
NumPy 的代價	大批量時只比 C++ 慢 1.5 倍	多數情況不需要改寫 C++
服務的阻塞	伺服端花 20 ms，呼叫端就被擋 20.3 ms	長任務一律用動作

話題與服務的吞吐	89,862 對 9,101 次 / 秒	不需回覆就用話題
錄製的資料量	全開一小時 183 GB	先選話題、再壓縮、必要時降頻
壓縮的效果	zlib 只有 1.1–1.5 倍，JPEG 有 22 倍	用懂資料結構的編碼器

表 28-5　本章三個實驗的實務對照

公式或流程

一、環境建置的完整流程

圖 28-4 從零到能跑 SLAM 的環境建置流程

① 確認版本對應：選定 ROS 2 發行版 → 查出對應的 Ubuntu 版本（表 28-1）

↓

② 安裝 Ubuntu：原生 / 虛擬機 / 容器（依用途選擇，28.1 節）

↓

③ 安裝 ROS 2：依官方文件加入軟體來源並安裝桌面版（含 RViz2）

↓

④ 安裝模擬器：依該 ROS 版本官方推薦的 Gazebo 版本

↓

⑤ 建立工作空間：建立目錄 → 放入套件原始碼 → colcon 建置

↓

⑥ 載入環境設定（source）→ 驗證：啟動範例節點，確認能通訊

↓

⑦ 安裝 SLAM 套件：slam_toolbox / Cartographer / Nav2 等

↓

⑧ 冒煙測試：啟動模擬器與 SLAM，在 RViz2 中看到地圖成長

第八步「冒煙測試」是關鍵的檢查點：若能看到地圖逐步長出來，表示整條工具鏈都通了，可以放心進入第 29 章的實作。若卡住，依 27.1 至 28.2 節的常見陷阱逐項排查。

二、通訊機制的選用判斷

圖 28-5　話題、服務、動作的選用決策

這個功能需要什麼樣的通訊？

↓

① 是持續流動的資料嗎？（感測器、狀態、控制指令）

├─ 是 → 【話題】發布訂閱，非同步，一對多

└─ 否 → 進入 ②

↓

② 是很快就能回答的一次性請求嗎？（查詢、設定）

├─ 是 → 【服務】請求回應，呼叫端等待

└─ 否 → 進入 ③

↓

③ 是耗時且需監控進度的任務嗎？（導航、掃描任務）

└─ 是 → 【動作】可回報進度、可取消

三、rosbag 錄製與重播的完整流程

圖 28-6　資料錄製到分析的工作流程

【錄製階段】

① 確認要錄的話題：感測器資料 + IMU + 里程計 + TF（勿漏 TF）

② 估算檔案大小與儲存空間

③ 執行錄製 → 產生 MCAP 格式的記錄檔

↓

【重播階段】

④ 檢查話題名稱是否與演算法預期相符，必要時做重新對應

⑤ 重播資料 → 同時啟動 SLAM 節點

⑥ 在 RViz2 中即時觀察，或以 Foxglove 事後分析

↓

【分析階段】

⑦ 輸出估測軌跡 → 與真值比較 → 計算 ATE / RPE（第 22.7 節）

⑧ 調整參數 → 回到 ⑤ 重播同一份資料 → 公平比較

注意第八步：因為重播的是同一份資料，不同參數的比較才是公平的。這正是 rosbag 對研究與教學最大的價值——把「感測資料的隨機性」這個變因固定下來。

案例

案例 28-1　半堂課的代價：版本不符的連鎖反應

這是幾乎每個 SLAM 課程都會發生的場景，值得同學引以為戒。

狀況是這樣的：同學下載了某個開源 SLAM 專案，依 README 執行編譯，結果出現一長串錯誤訊息。上網搜尋、逐一嘗試各種解法、安裝了幾個看起來相關的套件——兩個小

時過去，問題不但沒解決，系統還被裝壞了。

根源往往極其單純：專案是為某個 ROS 2 版本寫的，而同學裝的是另一個版本。相依的函式庫版本不同，連鎖反應下產生數十個看似無關的錯誤。

這個案例的教訓有三：

- 第一，先確認版本對應。開始編譯前，先花五分鐘確認專案支援的 ROS 版本與你的環境是否相符——這五分鐘能省下兩小時。
- 第二，錯誤訊息要從第一個看起。編譯錯誤常有連鎖效應，後面數十個錯誤可能都源於第一個。從最後往回看是常見的誤區。
- 第三，容器是好朋友。用容器封裝環境，裝壞了就重建，不會污染主系統，也能確保全班環境一致（28.1 節的建議）。

更深一層的啟示是：工程實作中，「環境問題」佔用的時間往往遠超預期。這不是同學能力不足，而是這個領域的常態。有經驗的工程師與初學者的差別，不在於不會遇到問題，而在於能快速判斷問題的類別並找到對應的解法。

案例 28-2　從 rosbag 到論文：一份資料的多重價值

rosbag 的價值，在一個真實的研究流程中展現得最清楚。

假設一位同學要做「不同 SLAM 系統在系館走廊的表現比較」的專題。若沒有 rosbag，他必須：為每個系統各跑一次實體實驗——但每次實驗的路線、速度、光線、人流都不完全相同，比較結果毫無說服力；而且每次都要借用機器人、佈置場地，耗時費力。

有了 rosbag，流程完全不同：錄製一次高品質的資料（含影像、IMU、光達、TF），之後所有系統都用同一份資料測試。這帶來四個好處：

- 公平比較：所有系統面對完全相同的輸入，差異純粹來自演算法。
- 可重現：別人拿到你的資料，能重現你的結果——這是科學研究的基本要求。

- 效率：參數調整可以反覆進行，不需重跑實驗。一個下午能測試數十組參數。
- 長期價值：這份資料明年、後年的學生仍能使用。第 26.7 節建議的「錄製並公開在地場域資料集」，正是這個思路。

具體的建議是：每一屆的專題生都貢獻一份自己場域的資料，累積下來就是一個有價值的在地資料集。這對後續研究、教學示範、甚至與其他學校交流，都是實質的資產。

案例 28-3　工具的演進：從 rosbag 到 MCAP、從 RViz 到 Foxglove

本章介紹的工具鏈，近年有顯著的演進，值得同學留意這個趨勢。

記錄格式方面，MCAP 已成為事實上的新標準，取代了原本的 rosbag 格式，並自 ROS 2 Jazzy 起成為 rosbag2 的預設儲存格式。它的關鍵優勢是自我描述——記錄檔內嵌訊息定義，不需額外的套件就能解讀。

視覺化方面，Foxglove 補足了 RViz2 的限制：RViz2 依賴 ROS、難以檢視非 ROS 系統、記錄檔分析能力弱、難以協作分享、且無網頁或平板支援。Foxglove 則同時支援桌面、網頁與雲端，並能處理 ROS 1、ROS 2、MCAP、MAVLink、MQTT 等多種協定。

這個演進背後有一個值得注意的模式：推動改變的往往不是 ROS 核心團隊，而是產業界的實務需求。Foxglove 由前自駕車公司的工程師創辦，而他們正是把自駕車開發中的痛點——大量資料的分析、跨團隊協作、非 ROS 系統的整合——轉化為工具。

對同學的啟示是：工具會不斷演進，不必執著於某個特定工具。重要的是理解「這個工具解決什麼問題」——當你清楚問題的本質，就能快速上手新工具，也能判斷什麼時候該換工具。這與本書一貫的主張相同：理解原理比記憶操作重要。

實作活動

實作 28-1　建置 SLAM 開發環境並完成冒煙測試

目的：完整建置可用的 ROS 2 開發環境，熟悉三種通訊機制與 rosbag 工具，並完成第一次

模擬建圖。本實作是第 29、30 章的必要前置作業。

第一部分：環境建置（全體）

1. 依表 28-1 選定 ROS 2 版本與對應的 Ubuntu 版本，記錄你的選擇與理由。
2. 依 28.1 節選擇安裝方式（原生 / 虛擬機 / 容器），完成 Ubuntu 與 ROS 2 桌面版的安裝。
3. 驗證安裝：啟動 ROS 2 內建的範例節點（如 talker 與 listener），確認兩者能正常通訊。
4. 建立工作空間：建立目錄結構，用 colcon 建置一個空的工作空間，並載入環境設定。
5. 安裝模擬器與 TurtleBot3 套件，啟動模擬環境，確認能在畫面中看到虛擬機器人。

第二部分：通訊機制探索（全體）

1. 列出目前系統中所有活躍的話題，找出 /scan、/odom、/tf 等 SLAM 相關的話題。
2. 檢視 /scan 話題的訊息內容，對照第 8.3 節的 LaserScan 格式，找出 angle_min、angle_increment、ranges 等欄位。
3. 觀察話題的發布頻率，記錄光達、里程計、TF 各自的頻率，思考為什麼它們不同。
4. 列出可用的服務與動作，找出與導航相關的動作介面（第 29 章會用到）。
5. 用 Python 撰寫一個最簡單的節點：訂閱 /scan，計算並印出「當前最近障礙物的距離與方向」。這是第 8 章實作 8-1 延伸挑戰的 ROS 版本。

第三部分：rosbag 錄製與重播（全體）

1. 遙控模擬機器人在環境中移動，同時用 rosbag2 錄製 /scan、/odom、/tf 與 /imu（若有），觀察產生的檔案格式與大小。
2. 停止模擬器，單獨重播錄好的資料，在 RViz2 中顯示光達掃描與 TF，確認能正常重現。

3. 話題重新對應練習：重播時把 /scan 改名為 /laser_scan，觀察原本訂閱 /scan 的節點是否還能運作，理解 28.4 節提到的重新對應技巧。
4. 刻意不錄 TF 再重播一次，觀察 RViz2 中會發生什麼——親身驗證 28.4 節「勿漏 TF」的提醒。

第四部分：冒煙測試——第一次建圖（全體）

1. 啟動模擬器與 SLAM 套件（建議用 slam_toolbox 或 Cartographer），在 RViz2 中顯示地圖、光達與 TF。
2. 緩慢遙控機器人繞行環境一圈，觀察佔據格地圖如何逐步長出來（第 13 章的理論在此具象化）。
3. 儲存地圖，並用第 13.7 節的方法評估品質：牆是否筆直？迴路是否閉合？
4. 把整個流程寫成一個啟動檔，讓下次能一鍵啟動模擬器、SLAM 與 RViz2。

討論問題

1. 你選擇了哪個 ROS 2 版本？依 28.1 節的建議，這個選擇對「找教學資源」與「長期維護」各有什麼利弊？
2. 光達、里程計、TF 三者的發布頻率各是多少？為什麼它們的頻率不同？（提示：第 7、13 章的感測器特性）
3. 不錄 TF 的資料重播時發生了什麼？這印證了第 10 章的哪個觀念？
4. 依圖 28-2 的決策流程，若要實作「讓機器人執行一次完整的房間掃描任務，過程中可查詢進度也可中止」，應該用哪種通訊機制？為什麼？
5. 第一次建圖的結果如何？若不理想，依第 14.6 節的失敗模式，你認為最可能的原因是什麼？

延伸挑戰（大學部）

其一：容器化你的開發環境——撰寫一個容器設定檔，封裝完整的 ROS 2 環境與所需的

SLAM 套件，讓同學能一鍵取得一致的環境。這對教學助教與專題團隊極為實用，也是現代軟體工程的必備技能。其二：開源專案閱讀練習——選一個本書介紹過的開源 SLAM 專案（建議從 slam_toolbox 開始，程式規模適中），依 28.6 節的五個步驟閱讀，並繪製一張「資料如何從輸入流到輸出」的流程圖。把你的流程圖與本書對應章節的架構圖比較，看看理論與實作的對應關係。

延伸閱讀

1. ROS 2 官方文件（docs.ros.org）.（安裝指引、教學與 API 文件，實作的第一手參考）
2. REP-2000: ROS 2 Releases and Target Platforms(ros.org/reps).（版本與平台對應的官方規範，28.1 節的依據）
3. S. Macenski et al., "Robot Operating System 2: Design, Architecture, and Uses in the Wild," Science Robotics, 2022.（ROS 2 設計理念的權威介紹）
4. Gazebo 官方文件之 Installing Gazebo with ROS(gazebosim.org).（ROS 與 Gazebo 版本搭配的官方建議）
5. MCAP 官方文件（mcap.dev）與 Foxglove 文件（docs.foxglove.dev）.（新一代記錄格式與視覺化工具）
6. slam_toolbox 開源專案（GitHub）.（ROS 2 主流的 2D SLAM，程式規模適中，適合作為原始碼閱讀的入門）
7. ROBOTIS TurtleBot3 e-Manual(emanual.robotis.com).（教學平台的完整實作指引，含模擬與實體機器人）

小結

1. ROS 2 與 Ubuntu 的版本是綁定的，裝錯會造成連鎖的相依性錯誤；ROS 2 每年 5 月發版、偶數年為 LTS，教學建議選用資源最豐富的成熟 LTS 版本，且全班應統一版本。安裝方式有原生、虛擬機與容器三種，教學階段建議用容器以確保環境一致。
2. Python 適合快速原型與資料分析、C++ 適合 SLAM 核心與高頻運算；CMake 管理 C++ 的編譯，看懂 find_package 與 target_link_libraries 就能解決多數編譯問題；colcon 依相依關係建置工作空間，建置後務必載入環境設定。
3. 三種通訊機制各有適用：話題用於持續流動的資料（感測器、狀態、指令）、服務用於快速的一次性請求、動作用於耗時且需監控進度的任務（如導航）；用錯機制會導致系統阻塞與除錯困難。
4. rosbag2 讓「錄一次、分析無數次」成為可能，支撐可重現性、離線開發、除錯與資料集建立；MCAP 自 ROS 2 Jazzy 起成為預設格式，具索引與自我描述特性；錄製時勿漏 TF，重播時留意版本、TF 與話題名稱三個常見問題。
5. RViz2 用於即時三維視覺化、Gazebo 提供無需硬體的模擬（版本須依官方推薦搭配）、Foxglove 補足 RViz2 在記錄檔分析、協作分享與非 ROS 系統上的限制；三者互補而非取代。
6. 閱讀開源專案的五步驟：先讀論文再讀程式、讀 README 確認相依與硬體需求、先跑起來再讀程式、從進入點順資料流往下追、加輸出訊息驗證理解；並善用 issue 區與社群維護的分支。
7. 學習路徑分五階段：觀察體驗、積木式操作、資料與分析、讀原始碼、修改與創新；教學心法為先成功再深入、刻意示範失敗、以及把環境問題與學習內容分開。

環境與工具至此備齊，同學已經有能力把前面所有理論真正跑起來。下一章，我們把第四篇的 2D SLAM 從頭到尾實作一次：硬體組裝與 TF 設定、里程計發布、Gmapping 與 Cartographer 建圖、AMCL 定位，最後接上完整的導航堆疊，讓機器人真正自己走到目標點。這將是本書第一個完整的端到端系統實作。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 29 章

2D 光達小車建圖實作

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

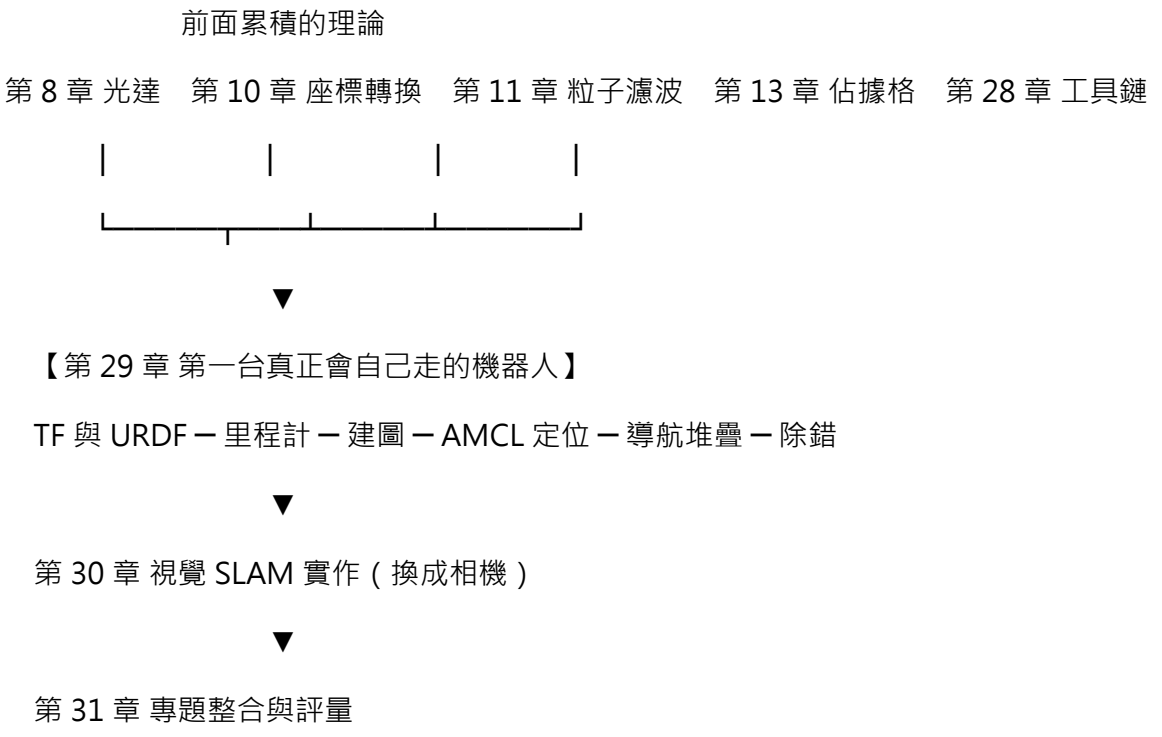
本章在全書的位置

這是全書第一個端到端的系統實作——從輪子轉動到機器人自己走到目標點，中間每一個環節都要親手接起來。

本章與前面各章最大的不同，在於它處理的是「接線的地方」。演算法本身在第 13、14、15 章都講過了，真正會卡住專題的是座標轉換設錯、里程計參數沒校正、以及系統不動時不知道該從哪裡查起。這些問題的共同點是：它們不會報錯，只會讓結果「看起來怪怪的」。

本章的三張圖就是為了讓「怪怪的」變成「可以量的」：圖 29-2 示範里程計校正的標準方法，並說明為什麼一定要順時針與逆時針各走一次；圖 29-3 把四種常見的座標轉換錯誤建出來的地圖並排，當成一張症狀對照表；圖 29-4 則實際跑一次全域定位，看粒子要幾步才收斂。

圖 29-0　第 29 章的連結地圖



學習目標

前二十七章的理論與工具至此全部備齊。本章是本書第一個從頭到尾的完整系統實作：從硬

體與座標設定開始，經過建圖、定位，最後讓機器人自己走到目標點。學完本章之後，同學應能夠達成下列目標：

8. 說明 2D SLAM 小車的硬體組成與各元件的角色。
9. 撰寫 URDF 機器人描述檔並正確設定 TF 座標樹。
10. 由輪速編碼器計算並發布里程計資料。
11. 執行 2D SLAM 建圖並儲存地圖檔。
12. 說明 AMCL 定位的設定要點與常見問題。
13. 說明 Nav2 導航堆疊的架構與一次導航任務的完整資料流。
14. 依系統性的順序排除導航系統的故障。

核心概念

中文名詞	英文名詞	概念說明
機器人描述檔	URDF	以 XML 描述機器人的連桿、關節與感測器安裝位置。
連桿與關節	Link / Joint	URDF 的基本元素：剛體與連接它們的關節。
狀態發布器	robot_state_publisher	依 URDF 發布各連桿之間 TF 變換的節點。
差速驅動	Differential Drive	以左右兩輪速度差控制方向的驅動方式。
里程計	Odometry	由輪速積分推算的位姿，提供 odom→base_link 變換。
代價地圖	Costmap	在佔據格上加註通行成本的導航用地圖。
膨脹層	Inflation Layer	把障礙物向外擴張安全距離的代價地圖層。
全域規劃器	Global Planner	在靜態地圖上規劃起點到目標的完整路徑。
控制器	Controller	把路徑轉為速度指令的局部規劃元件。

中文名詞	英文名詞	概念說明
行為樹	Behavior Tree	Nav2 用來組織導航任務流程的架構。
復原行為	Recovery Behavior	導航失敗時嘗試脫困的動作，如原地旋轉、後退。
生命週期節點	Lifecycle Node	有明確啟動狀態轉換的 ROS 2 節點類型。
速度指令	cmd_vel	送給底盤的線速度與角速度指令。

29.1　硬體：小車底盤、馬達、編碼器、2D 光達

本節說明一台能跑 SLAM 的 2D 小車需要哪些元件，以及選型時的考量。

五大元件

- 底盤與馬達：最常見的是差速驅動——左右兩個驅動輪加一到兩個萬向輪。它結構簡單、控制直觀（第 10.3 節的 2D 位姿正好對應），是教學平台的標準選擇。
- 編碼器：裝在馬達軸或輪軸上，計算輪子轉了多少。這是里程計的資料來源（29.3 節），也是第 13.4 節「里程計預測、光達校正」中不可或缺的一半。
- 2D 光達：第 8 章介紹過的旋轉式雷射掃描器，提供 /scan 話題。教學用的入門級型號量測範圍約 8 至 12 公尺，對室內建圖已足夠。
- 微控制器：負責馬達驅動與編碼器讀取，並與上層電腦通訊。它處理的是毫秒級的即時控制，不適合放在單板電腦上做。
- 單板電腦：執行 ROS 2 與 SLAM 演算法。這個「微控制器負責即時控制、單板電腦負責運算」的分工，正是第 25 章案例 25-2 討論過的雙晶片架構在教學平台上的體現。

選型的三個考量

4. 光達安裝高度與遮蔽：光達必須有清晰的 360 度視野。若機身或其他元件擋住部分角度，那些方向就永遠是盲區。安裝時務必實際檢查掃描是否被遮擋。
5. 編碼器解析度：解析度太低，里程計的精度就差。一般教學平台的編碼器解析度足夠，但若使用自製底盤，需確認每圈的脈衝數足以分辨小幅度的移動。
6. 運算能力：2D SLAM 對運算的需求不高（相較第五、六篇的 3D 與視覺 SLAM），常見的單板電腦即可勝任。若要同時跑 Nav2 導航，則需留意記憶體是否充足。

三種平台選擇

- 現成教學平台：整合完整、文件齊全、有社群支援，能讓同學快速進入 SLAM 本身而非硬體除錯。適合課程時間有限的情況。
- 自組平台：成本較低、學習深度更高——同學能親手處理每一個環節，對系統的理解遠比使用現成平台深刻。適合專題與有充裕時間的課程。
- 純模擬：完全不需硬體，但能完整體驗 28.2 至 29.7 節的所有流程。第 28 章建議的「先模擬、再硬體」路徑，在此章尤其適用——先在模擬中把整條管線跑通，再搬到實體車上，能省下大量除錯時間。

29.2 TF 設定與 URDF

第 10 章強調過「地圖歪了先看 TF」，本節說明如何正確建立座標樹。這是整個實作最容易出錯、也最需要謹慎的環節。

URDF：機器人的骨架描述

URDF（統一機器人描述格式）以 XML 描述機器人的物理結構：有哪些剛體（連桿）、它們如何連接（關節）、以及各自的尺寸與相對位置。

對 SLAM 而言，URDF 最重要的功能是描述感測器的安裝位置——也就是第 5、10 章反覆提到的外參。若 URDF 中寫的光達位置與實際安裝差了幾公分或幾度，建出的地圖就會模糊或錯位。

典型的座標樹結構

第 10.7 節介紹過 ROS 的標準框架階層，在實體小車上的具體形式為：

圖 29-1　2D SLAM 小車的完整 TF 樹

map（地圖座標系）　全域固定、不漂移、會跳變

↓ 由 SLAM 或 AMCL 發布

odom（里程計座標系）　連續平滑、長期漂移

↓ 由里程計節點發布（29.3 節）

base_footprint（機器人在地面的投影，選用）

↓

base_link（機器人本體中心）

├──→ laser（光達）　　↖

├──→ imu_link（IMU）　　由 URDF 與 robot_state_publisher

├──→ left_wheel　　依固定的安裝關係發布

└──→ right_wheel　　↙

誰負責發布哪一段

這是初學者最常混淆的地方，務必弄清楚：

- map → odom：由 SLAM 系統（建圖時）或 AMCL（導航時）發布。官方文件明確指出，這個變換是 Nav2 系統的主要需求之一。
- odom → base_link：由里程計系統發布。里程計系統的角色就是提供這個變換，而其來源可以是光達、雷達、輪速編碼器、視覺慣性里程計或 IMU；目標是提供一個基於機器人運動的平滑連續局部座標系。
- base_link → 各感測器：由 robot_state_publisher 依據 URDF 提供，它會發布 base_link 到各感測器的變換。

常見的 TF 錯誤

7. 座標樹斷裂：某一段沒有節點發布，導致 map 到 laser 的完整鏈路斷開。RViz 中會顯示「找不到變換」的錯誤。
8. 外參填錯：光達的安裝位置或朝向填錯。症狀是掃描點與實際環境錯位——把機器人放在牆邊，若 RViz 中的掃描點與牆的位置對不上，就是這個問題。
9. 重複發布：兩個節點同時發布同一段變換，造成衝突與跳動。這在整合多個套件時容易發生。
10. 時間戳記問題：變換的時間戳記與感測資料不同步，導致高速移動時明顯錯位（第 10.6 節提過時間戳記的重要性）。

驗證的方法

- 視覺化檢查：在 RViz 中同時顯示 TF 座標軸與光達掃描，確認掃描點與機器人的相對位置合理。
- 產生 TF 樹圖：用工具產生座標樹的圖形，確認結構完整、沒有斷裂或重複。
- 靜態驗證：把機器人靜置在已知位置（例如緊靠牆角），檢查掃描點是否落在正確的距離上。

29.3 里程計資料發布

里程計是 2D SLAM 的另一半基石。本節說明它如何從編碼器讀值變成 ROS 的里程計訊息。

從編碼器到位姿

差速驅動小車的里程計計算，是第 3 章航位推算與第 10.3 節座標轉換的直接應用：

11. 讀取編碼器：取得左右輪在這個時間間隔內各轉了多少。
12. 換算輪子位移：轉動圈數乘上輪子周長，得到左右輪各自走了多遠。
13. 計算機體運動：由左右輪位移的平均值得到前進距離、由兩者的差除以輪距得到轉角（公式見本章公式一）。

14. 更新位姿：依第 10.3 節的 2D 位姿複合公式，把相對運動累加到當前位姿，並記得處理角度環繞。
15. 發布：發布里程計訊息（含位姿與速度），同時發布 odom → base_link 的 TF 變換。

校正：精度的關鍵

第 14.6 節提過「輪子打滑與里程計失準」是建圖失敗的首要原因，而其中最常被忽略的是參數校正。兩個關鍵參數是：

- 輪徑：若設定值與實際不符，直線行走的距離就會系統性偏差，導致整張地圖等比例放大或縮小。校正方法是讓小車直走一段固定距離（例如 3 公尺），比較里程計讀值與捲尺實測值。
- 輪距：兩個驅動輪的中心距離。若設錯，轉彎角度就會偏差，這對建圖的影響比輪徑更嚴重——因為角度誤差會透過後續的移動被放大（第 7.3 節討論過姿態誤差的放大效應）。校正方法是讓小車原地轉 10 圈，比較里程計累計角度與 3600 度的差異。

這兩項校正應該在第一次建圖之前完成。許多同學花數小時調 SLAM 參數，卻沒發現問題出在未校正的里程計——這是實作中最常見的冤枉路。

里程計的品質要求

里程計的目標是提供一個平滑而連續的局部座標系，而全域定位系統會更新相對於全域座標系的變換以補償里程計的漂移。

請注意「平滑而連續」這個要求：里程計不需要絕對準確（漂移會被 SLAM 修正），但必須平滑不跳變。若里程計因為程式錯誤或通訊丟包而出現跳躍，SLAM 的運動預測就會失效，建圖必然失敗。

進階：融合 IMU

純輪速里程計的角度估計容易因打滑而失準。實務上常以濾波器融合輪速與 IMU 的角

速度（第 7、11 章），得到更穩健的里程計。ROS 2 生態中可用 robot_localization 套件的 EKF 節點來提供 odom → base_link 變換，這在地面不平或容易打滑的場域特別有幫助。

量一量：里程計參數不準會怎樣

29.3 節說里程計要校正，但沒說不校正會差多少、也沒說怎麼知道是哪個參數錯了。本小節用標準的方形路徑測試把兩件事一起回答。

差動輪里程計只有兩個關鍵參數：輪半徑與輪距。它們各自會造成不同性格的誤差：輪距設太小，機器人以為自己轉得比實際多，於是每次轉彎都轉不夠；左右輪直徑不等，機器人直行時會偏向一側，但它的里程計以為自己走直線。麻煩的是——光看一次實驗，這兩種錯誤看起來一模一樣。

實驗方法是經典的方形路徑測試：命令機器人依「自己的里程計」走一個 2 公尺見方的正方形回到起點，然後量測它實際停在哪裡。關鍵在於順時針與逆時針各走一次。

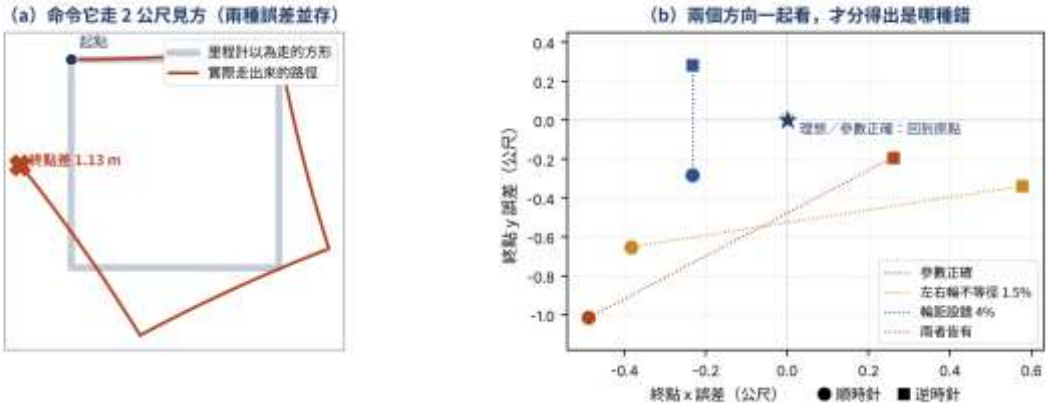


圖 29-2　方形路徑測試：里程計誤差的指紋

（a）畫出一台同時有兩種誤差的小車（輪距設定值偏小 4%、右輪實際比左輪大 1.5%）順時針走方形的結果。灰色粗線是里程計相信自己走的路徑——一個標準的正方形；紅線是它實際走出來的路徑。每個轉角都轉不夠，而且直線段微微偏彎，四個邊累積下來，終點與起點差了 1.13 公尺。在一個 2 公尺見方的場地裡，這個誤差超過邊長的一半。

（b）是本圖的重點。四種情況各跑順時針（圓點）與逆時針（方塊）兩次，把終點誤差畫

在同一張圖上。參數正確時兩點都落在原點（誤差 0.001 公尺）。

輪距設錯 4%（藍色）的兩點落在 x 相同、y 正負對稱的位置（−0.233, −0.281）與（−0.233, +0.281）。這個「同 x、鏡像 y」的排列，就是純輪距誤差的指紋——因為轉彎角度的誤差在兩個方向上是對稱的。

左右輪不等徑 1.5%（橘色）則完全不同：兩點落在（−0.384, −0.652）與（+0.577, −0.339），位置差很遠而且沒有明顯的對稱關係。因為直行時的偏彎方向不隨行進方向改變，但轉彎的順序改變了它累積的方式。

兩種誤差並存時（紅色），順時針的終點誤差是 1.13 公尺，逆時針卻只有 0.32 公尺。這正是為什麼只跑一個方向會被騙：如果同學只試了逆時針，看到 0.32 公尺可能覺得「還可以接受」，完全不知道另一個方向會差到 1.13 公尺。

校正的順序

依圖 29-2(b) 的指紋，校正可以分兩步進行。

第一步先處理左右輪不等徑：調整左右輪的半徑比，直到順時針與逆時針的終點落在「對原點大致對稱」的位置為止。此時殘留的就主要是輪距誤差。

第二步調整輪距：若轉彎轉不夠（終點偏向路徑內側），把輪距設定值調大；反之調小。每次調整後重跑兩個方向，直到兩點都靠近原點。

一個實務提醒：校正結果會隨地面材質改變（地毯與磁磚的打滑程度不同），也會隨輪胎磨損而漂移。因此這個測試值得在每學期開始、或換場地時重做一次——它只需要十分鐘，卻能省下後面所有「地圖為什麼會歪」的除錯時間。

29.4　2D SLAM 建圖

前置作業完成後，終於可以建圖了。本節說明流程與工具選擇。

工具選擇

第 14、15 章介紹過 Gmapping 與 Cartographer，而在現代 ROS 2 環境中，還有第三個常用選擇。Nav2 官方把 SLAM Toolbox 列為預設的 SLAM 演算法，用於定位並產生靜態地圖。

工具	理論基礎	特點	適用
slam_toolbox	圖優化（第 12 章）	ROS 2 預設、參數簡潔、支援續建地圖	教學首選
Cartographer	圖優化 + 子地圖	閉環強、支援 3D、參數較多	大場域、需閉環
Gmapping	粒子濾波（第 11 章）	概念直觀、ROS 1 時代主流	理解粒子濾波

表 29-1 三種 2D SLAM 工具的比較

教學建議：先用 slam_toolbox 建立成功經驗（參數少、容易上手），再用 Cartographer 體驗閉環的威力（第 15 章實作 15-1），最後可用 Gmapping 對照理解粒子濾波與圖優化的差異。

建圖流程

16. 啟動硬體：機器人驅動、光達、里程計節點全部啟動，確認 /scan 與 /odom 都有資料。
17. 檢查 TF：確認座標樹完整（29.2 節的驗證方法）。這一步不可跳過。
18. 啟動 SLAM：啟動選定的 SLAM 節點。要確認它有提供 map → odom 變換並發布 /map 話題。
19. 啟動 RViz：顯示地圖、掃描、TF 與機器人模型，以便即時觀察。

20. 緩慢遙控建圖：第 14.6 節強調過，「開慢一點」是新手成功率最大的變數。轉彎時尤其要慢。
21. 評估品質：依第 13.7 節檢查牆是否筆直、迴路是否閉合、有無重影。
22. 儲存地圖：用 nav2_map_server 儲存，會產生 .yaml 與 .pgm 兩個檔案來表示地圖。

地圖檔的兩個檔案

- .pgm：灰階影像檔，就是佔據格地圖的視覺化——黑色為障礙、白色為自由空間、灰色為未知（第 13.1 節的三態）。可以用一般看圖軟體開啟檢視。
- .yaml：描述檔，記錄影像檔名、解析度（每像素幾公尺）、原點座標、以及判定佔據與自由的機率門檻。

這兩個檔案就是第 14.5 節提過的地圖產物。日後導航時由 map_server 載入，供 AMCL 定位與路徑規劃使用。

量一量：座標轉換設錯時，地圖長什麼樣

29.2 節設定了座標轉換樹，29.7 節把它列為除錯檢查表的第一項。但「座標轉換設錯」到底會長什麼樣子？本小節刻意設錯四種常見的情況，把建出來的地圖並排，當成一張症狀對照表。

實驗設定是一間 10×7 公尺的室內場景，機器人沿橢圓軌跡繞一圈，用 240 束的 2D 光達建佔據格地圖。真實的光達裝在車體中心，而系統「以為」的外參被刻意設錯。

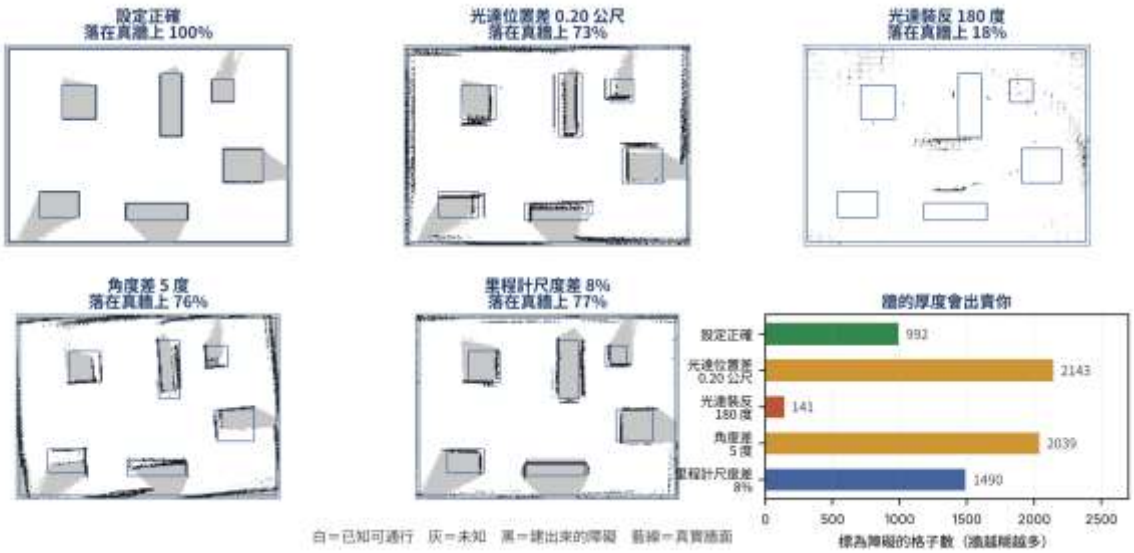


圖 29-3 座標轉換設錯的四種症狀

設定正確時，牆面是乾淨的一條線，標為障礙的格子有 992 個，其中 100% 落在真實牆面上。這是基準。

光達位置差 0.20 公尺時——這是最常見的錯誤，通常來自「量測安裝位置時少量了法蘭的厚度」或「URDF 裡填的是光達外殼位置而非光學中心」。地圖看起來仍然像那間房間，牆也還在該在的地方，但障礙格數從 992 暴增到 2,143（2.2 倍），只有 73% 落在真實牆面上。症狀是：牆變厚了，而且是雙層的。

角度差 5 度的症狀幾乎一樣（2,039 格、76%）。這說明一件重要的事：從地圖的外觀，你分不出是位置錯還是角度錯——只能分辨出「有錯」。要分辨是哪一種，得回頭檢查座標轉換樹的數值。

光達裝反 180 度是最戲劇性的一種。這種錯誤在實務上並不罕見——光達的安裝方向與它的座標系定義不一定一致。結果是所有量測都被投影到房間外面，地圖上只剩 141 個障礙格，而且只有 18% 落對位置。這種錯誤反而最容易發現，因為地圖根本不成形。

里程計尺度差 8% 則是另一種類型——它不是座標轉換的錯，而是 29.3 節的校正沒做好，但症狀同樣表現在地圖上：1,490 格、77% 正確。牆同樣變厚，但厚的程度隨離起點的距離而增加。

把這五張圖放在一起，可以歸納出一條很實用的診斷規則：地圖的牆有多厚，就是系統總誤差有多大。一張乾淨的單線牆代表所有環節都對；雙層或模糊的牆代表某處有系統性偏差；而完全不成形的地圖，反而通常是最容易找的那種錯。

一個不用寫程式的檢查法

把機器人放在定點，不要移動它，只讓它原地旋轉一圈並建圖。

如果外參正確，旋轉時所有量測都會落在同一條牆線上，地圖是乾淨的。如果光達的安裝位置填錯，旋轉時量測會在地圖上畫出一個圓環——圓環的半徑就是你填錯的距離。

這個檢查只需要三十秒，卻能一次驗證圖 29-3 裡最常見的那兩種錯誤，而且圓環的半徑直接告訴你差多少。建議列入每次換硬體後的標準流程。

29.5　AMCL 定位

地圖建好之後，日常運行不需要再建圖，只需在已知地圖中定位——這就是第 3、11 章介紹過的 AMCL。

AMCL 是什麼

AMCL 是一套機率式的 2D 移動機器人定位系統，實作了自適應（或 KLD 取樣）的蒙地卡羅定位方法，以粒子濾波追蹤機器人在已知地圖上的位置。它已不再是 SLAM 演算法，因為它不建立環境地圖。

演算法的每次迭代，會比較地圖的樣貌與當前的雷射掃描，只保留那些讓資料與地圖最吻合的位置。這正是第 11 章粒子濾波「預測—秤重—重採樣」循環的實現。

AMCL 與 SLAM 的分工

項目	SLAM（建圖階段）	AMCL（導航階段）
是否建圖	是	否，使用既有地圖
地圖來源	即時建立	由 map_server 載入
運算負擔	較高	較低
提供的 TF	map → odom	map → odom
使用時機	首次進入新環境	日常運行

表 29-2 SLAM 與 AMCL 的分工

注意兩者都提供 map → odom 變換——這正是第 10.7 節設計的巧妙之處：對下游的導航系統而言，不論是 SLAM 還是 AMCL 在提供定位，介面完全相同，因此可以無縫切換。

關鍵參數

- 粒子數：與第 14.3 節的討論相同，多則穩健但耗運算。AMCL 的「自適應」特性能依據不確定性動態調整粒子數——不確定時多用粒子，收斂後減少。
- 運動雜訊模型：描述里程計有多不準的參數。其中控制旋轉對機器人旋轉估計影響的參數，若調高會讓系統對旋轉運動更謹慎、假設更大的不確定性；調低則對旋轉估計更有信心，但若感測器不夠準確會有風險。
- 更新門檻：機器人移動或轉動多少才執行一次濾波更新。設定較小的角度門檻（例如 0.05 弧度而非預設的 0.2，約 11.5 度）能讓機器人在較少的旋轉後就更新，適合緩慢或精確的移動；調高則減少更新次數以節省運算。
- 變換容許時間：發布的變換被視為有效的時間長度，預設 1 秒是合理值；調高有助於處理通訊延遲但可能使用過時的位置，調低則確保資料較新但可能造成抖動。

初始位姿：最常見的問題

AMCL 啟動時需要一個初始位姿的猜測。若沒有給，粒子會撒滿整張地圖，可能需要機器人移動一段距離才能收斂，甚至收斂到錯誤位置（第 11 章的多峰問題）。

- 手動設定：在 RViz 中用「2D Pose Estimate」工具點選機器人的大致位置與朝向。這是最常用的做法。
- 固定起點：讓機器人總是從同一個位置（例如充電站）啟動，並在設定中寫死初始位姿。這是第 27.5 節提過的實務技巧。
- 視覺標記輔助：用 AprilTag 提供絕對位姿（第 24.3 節），自動完成初始化。

量一量：全域定位要幾步才收斂

29.5 節說明了 AMCL 的原理與它與 SLAM 的分工。這一節實際跑一次全域定位——機器人完全不知道自己在哪，從均勻散布的粒子開始，看它要走幾步才能確定位置。

實驗設定：同樣的 10×7 公尺場景，已有先驗地圖。4,000 個粒子隨機散布在所有可通行的位置上，朝向也完全隨機。機器人沿一條約 5.5 公尺的路徑移動，每一步以 60 束的光達量測更新權重並重取樣。

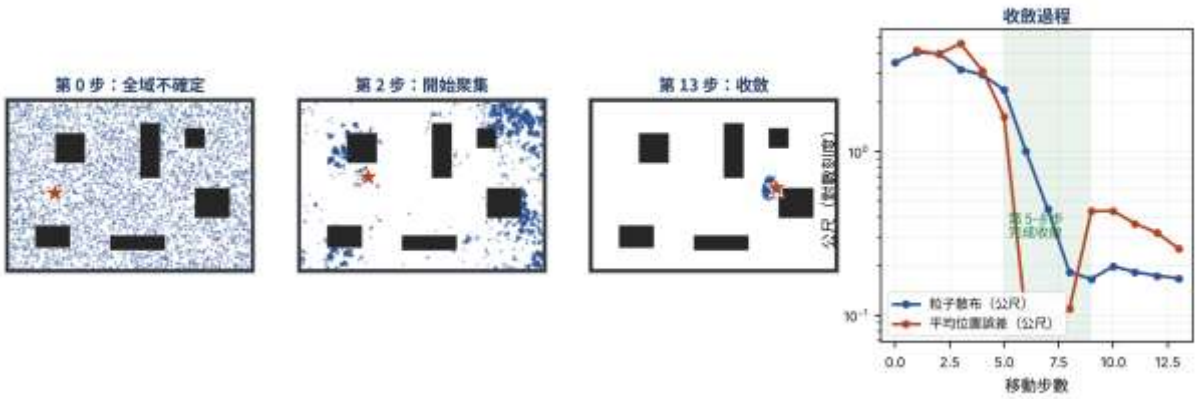


圖 29-4　AMCL 全域定位的收斂過程

第 0 步時粒子鋪滿整個可通行區域，散布是 3.49 公尺——系統對自己的位置一無所知。第 2 步時粒子已經開始聚成幾團，散布降到約 3.0 公尺；這些團代表「幾個看起來都說得通的可能位置」。到第 13 步，粒子收斂成一小團，散布 0.17 公尺，平均位置誤差 0.26 公尺。

右邊的收斂曲線顯示一個值得注意的現象：前四步幾乎沒有進展（散布甚至一度略為上升），真正的收斂集中在第 5 到 8 步。原因是機器人在前幾步只看到走廊的一小段，而場景裡有好

幾處長得差不多；直到它移動到能同時看見兩面不同牆的位置，多個假設才被一次排除。

這個「要走一段路才定得出來」的特性，對專題操作有直接的意義：啟動 AMCL 之後不要期待它立刻正確，先用遙控讓機器人在原地附近走動幾公尺，等粒子雲收斂了再交給導航堆疊。很多「導航一啟動就亂衝」的狀況，原因只是定位還沒收斂就下了目標點。

實作時踩過的一個坑

本實驗第一次跑的時候，粒子確實收斂了，卻收斂到錯誤的位置。

原因有兩個，都很典型。其一是運動更新寫錯了：里程計給的是世界座標的位移，但每個粒子的朝向都不同，必須先把位移轉到各粒子自己的機體座標再套用。直接把世界座標位移加到所有粒子上，等於假設它們朝向都一樣。

其二是沒有淘汰落在牆裡或地圖外的粒子。那些粒子的似然值是從被裁切的距離場查出來的，數值虛高，結果整團粒子一路飄出房間。

這兩個錯誤都不會讓程式當掉，只會讓定位「看起來在動但就是不對」——正是 29.7 節所說的、最難查的那一類問題。

29.6　Navigation Stack 導航

最後一步：讓機器人自己走到目標點。這是第 3.4 節導航系統架構的實際落地。

Nav2 的整體架構

Nav2 是 ROS 2 的導航堆疊，由多個模組協同運作。一個導航目標從送出到執行，會流經行為樹導航器、規劃器、控制器、代價地圖、復原行為、速度指令，最後到達底盤。

圖 29-5 一次導航任務的完整資料流

① 使用者送出目標（在 RViz 點選，或由程式以動作介面送出）

↓

② 行為樹導航器：依據行為樹決定執行流程

↓

③ 全域規劃器：在全域代價地圖上規劃完整路徑

（輸入：靜態地圖 + AMCL 提供的當前位姿）

↓

④ 控制器：依局部代價地圖，把路徑轉為速度指令

（局部代價地圖即時納入光達偵測到的動態障礙）

↓

⑤ 發布 /cmd_vel → 底盤執行 → 機器人移動

↓

⑥ 若卡住或路徑失效 → 觸發復原行為（原地旋轉、後退、清除代價地圖）

↓

⑦ 抵達目標 → 回報成功；失敗則回報失敗原因

代價地圖：導航的核心資料結構

代價地圖套件運用感測器資訊，以佔據格的形式提供機器人環境的表示；格子儲存 0 到 254 之間的成本值，代表通過該區域的代價。成本 0 表示格子是自由的，254 表示致命佔據，而介於兩者之間的值則被導航演算法用來像位能場一樣引導機器人遠離障礙物。

代價地圖以多個圖層實作，每個圖層有特定功能，共同貢獻於格子的整體成本。常見的圖層包括：

- 靜態層：載入建好的地圖（29.4 節的產物）。
- 障礙層：即時納入光達偵測到的障礙，包含地圖上沒有的動態物體。
- 膨脹層：把障礙物向外擴張一圈安全距離。這一層極為重要——沒有它，規劃出的路徑會緊貼牆面，機器人必定擦撞。膨脹半徑應設為機器人半徑加上安全餘裕。

全域與局部的分工

- 全域代價地圖：涵蓋整張地圖，以靜態地圖為主，供全域規劃器算出完整路徑。更新頻率低。
- 局部代價地圖：只涵蓋機器人周圍數公尺，以即時感測為主，供控制器閃避動態障礙。更新頻率高。

這個「全域慢而完整、局部快而即時」的分工，同學應該覺得熟悉——它與第 15 章 Cartographer 的雙子系統、第 18 章 LOAM 的雙層架構、第 26 章無人機的分層設計，是同一個系統設計模式。

29.7　系統性除錯：當機器人不動的時候

本章最實用的一節。導航系統元件眾多，出問題時若沒有系統性的排查順序，很容易亂槍打鳥浪費時間。

排查的正確順序

建議的除錯順序是：依序檢查生命週期狀態、TF、地圖伺服器、AMCL、代價地圖、規劃器輸出、控制器輸出，最後才是底盤運動。

更重要的是一個常被忽略的原則：規劃器與控制器的調參，應該等到生命週期、TF、定位、地圖與代價地圖都健康之後才進行。

這個順序背後的邏輯是：下游的問題往往源於上游。若 TF 有問題，再怎麼調規劃器參數都沒用。同學務必養成「由上游往下游排查」的習慣。

圖 29-6 導航系統的除錯檢查表

① 生命週期狀態：所有 Nav2 節點是否都已啟動並進入 active 狀態？

↓ 通過

② TF:map → odom → base_link → laser 的鏈路是否完整？

↓ 通過

③ 地圖：map_server 是否成功載入？RViz 中看得到地圖嗎？

↓ 通過

④ 定位：AMCL 是否收斂？粒子雲是否聚集在正確位置？

↓ 通過

⑤ 代價地圖：全域與局部代價地圖是否正確顯示？障礙是否有膨脹？

↓ 通過

⑥ 規劃器：送出目標後，是否有規劃出路徑？

↓ 通過

⑦ 控制器：是否有發布 /cmd_vel？數值是否合理？

↓ 通過

⑧ 底盤：收到 /cmd_vel 後輪子是否轉動？方向是否正確？

常見症狀與對應的檢查點

症狀	最可能的原因	檢查順序
完全無反應	節點未啟動或未進入 active	①
RViz 顯示變換錯	TF 斷裂或外參錯誤	②

症狀	最可能的原因	檢查順序
誤		
機器人位置亂跳	AMCL 未收斂或初始位姿錯誤	④
規劃不出路徑	目標在障礙上、代價地圖膨脹過大	⑤⑥
規劃出路徑但不動	控制器問題、/cmd_vel 未送達底盤	⑦⑧
一直撞牆	膨脹半徑太小、光達外參錯誤	②⑤
原地打轉	角度控制參數不當、里程計角度失準	29.3 節校正

表 29-3　常見故障症狀與檢查順序

每個階段都要留下證據

一個好的實作習慣是：每個檢查點都應該產出可驗證的證據——一段指令輸出、RViz 中一個看得見的座標框架、一個地圖檔、一台移動中的模擬機器人、一份乾淨的程式碼倉庫，或一段簡短的展示錄影。

這個習慣有三重價值：除錯時能快速定位問題在哪一段；報告與專題時有具體的成果可展示；而且累積下來就是一份完整的學習歷程。建議同學從本章開始就養成這個習慣——第 31 章的專題評量正需要這些證據。

至此，同學已經完成第一個從感測器到自主導航的完整系統。下一章我們把感測器換成相機，實作 RGB-D 與雙目的視覺 SLAM:RealSense 相機設定、相機校正、點雲產生、RTAB-Map 與 ORB-SLAM3 實作，並比較視覺 SLAM 與光達 SLAM 在同一場域的表現差異。

29.8　把三張圖變成一份開機檢查表

本章的三個實驗各自回答一個問題，合起來正好構成一套每次開機都該跑一遍的檢查流程。

順序	檢查	方法	合格標準
1	座標轉換	原地旋轉一圈建圖	牆是單線，不出現圓環
2	里程計	順逆時針各走一次 2 公尺方形	兩次終點都在 5 公分內
3	建圖	繞場一圈	障礙格數接近基準值，牆不變厚
4	定位	在已知地圖上做全域定位	5 至 10 步內粒子散布小於 0.3 公尺
5	導航	下一個 3 公尺外的目標點	能抵達且不碰撞

表 29-4　開機檢查表

這份檢查表的順序不是隨意排的，而是依「錯誤會往下游傳播」的方向排列。座標轉換錯了，里程計校正的結果就沒有意義；里程計不準，建出來的地圖就會歪；地圖歪了，定位就收斂不了；定位不準，導航一定失敗。從下游開始查，會浪費大量時間在症狀上。

每一項該花多久

第一項三十秒、第二項十分鐘、第三項五分鐘、第四項兩分鐘、第五項兩分鐘——整套跑完不到二十分鐘。相較於專題後期花三天找「機器人為什麼會撞牆」，這是極划算的投資。

更重要的是，這份檢查表把「感覺怪怪的」變成了可以記錄的數字。障礙格數、終點誤差、粒子散布——這三個數字每次都記下來，就能看出系統是不是在退化（輪胎磨損、感測器鬆動、地面變髒）。這與第 19.6 節巡檢機器人的地圖維護是同一個思路：會長期運行的系統，需要可追蹤的健康指標。

給教學現場的建議

對帶實作課的老師，這份檢查表還有一個額外的用途：它可以當成評分的客觀依據。與其評

「機器人有沒有跑起來」（受運氣影響很大），不如評「這五項檢查的數字是多少」——前者是二元的，後者是連續的，也更能區分出學生真正理解到哪一層。

公式或流程

一、差速驅動的里程計計算

設在一個取樣週期內，左右輪各走了 d_L 與 d_R（由編碼器脈衝數乘上每脈衝距離得到），輪距為 b，則機體的運動為：

$$\Delta s = (d_R + d_L) / 2 \quad (\text{前進距離})$$

$$\Delta\theta = (d_R - d_L) / b \quad (\text{轉角})$$

再依第 10.3 節的 2D 位姿複合公式更新位姿：

$$x \leftarrow x + \Delta s \cdot \cos(\theta + \Delta\theta / 2)$$

$$y \leftarrow y + \Delta s \cdot \sin(\theta + \Delta\theta / 2)$$

$$\theta \leftarrow \text{normalize}(\theta + \Delta\theta)$$

式中取 θ + Δθ / 2 是為了用「這段運動的平均朝向」來計算位移，比直接用起始朝向精確。normalize 則是第 10.3 節強調的角度環繞處理，不可省略。

注意 Δθ 的分母是輪距 b——這解釋了為什麼 29.3 節說「輪距設錯對建圖的影響比輪徑更嚴重」：輪距誤差直接造成角度誤差，而角度誤差會被後續移動放大。

二、每脈衝距離的換算

編碼器輸出的是脈衝數，換算為距離需要：

$$\text{每脈衝距離} = (\pi \times D) / (N \times G)$$

其中 D 為輪徑、N 為編碼器每圈脈衝數、G 為減速比（若編碼器裝在馬達軸上）。這個常數是里程計精度的基礎，務必以實測校正而非只依規格書。

三、輪徑與輪距的校正

圖 29-7　里程計參數校正流程

【輪徑校正】

① 讓小車沿直線行走一段固定距離（建議 3 公尺以上）

② 以捲尺量測實際距離 L_real，讀取里程計距離 L_odom

③ 修正：新輪徑 = 舊輪徑 × （L_real / L_odom）

④ 重複驗證，直到誤差小於 1%

【輪距校正】

① 讓小車原地旋轉 10 圈（順時針與逆時針各做一次）

② 讀取里程計累計角度 θ_odom，理論值為 3600 度

③ 修正：新輪距 = 舊輪距 × （θ_odom / 3600°）

④ 重複驗證，直到誤差小於 2%

【順序】先校輪徑、再校輪距；兩者都完成後才開始建圖

四、代價地圖的膨脹半徑

膨脹層的關鍵參數是膨脹半徑，它決定障礙物向外擴張多遠：

$$膨脹半徑 \geq 機器人外接圓半徑 + 安全餘裕$$

設定過小，規劃出的路徑會緊貼牆面導致擦撞；設定過大，狹窄通道會被完全封死而規劃不出路徑（表 29-3 的常見症狀）。實務上先設為機器人半徑的 1.2 至 1.5 倍，再依場域的通道寬度微調。

五、從零到自主導航的完整流程

圖 29-8　2D SLAM 小車的完整實作流程

【階段一：硬體與座標】

① 組裝硬體 → ② 撰寫 URDF → ③ 驗證 TF 樹完整

↓

【階段二：里程計】

④ 實作編碼器讀取與里程計計算 → ⑤ 校正輪徑與輪距（圖 29-4）

↓

【階段三：建圖】

⑥ 啟動 SLAM → ⑦ 緩慢遙控建圖 → ⑧ 評估品質 → ⑨ 儲存地圖

↓

【階段四：定位】

⑩ 載入地圖 → ⑪ 啟動 AMCL → ⑫ 設定初始位姿 → ⑬ 確認收斂

↓

【階段五：導航】

⑭ 啟動 Nav2 → ⑮ 調整代價地圖 → ⑯ 送出目標 → ⑰ 觀察執行

↓

【階段六：除錯與調校】依圖 29-3 的順序排查與優化

案例

案例 29-1　三公分的代價：一個未校正的輪徑

這是課堂上真實發生過、也極具代表性的案例。

一組同學的建圖結果總是「怪怪的」：地圖看起來完整，牆線也算筆直，但把地圖上的教室尺寸與實際量測比較，發現地圖上的教室比實際大了約 8%。更奇怪的是，他們花了整個下午調整 SLAM 的各種參數，結果毫無改善。

問題出在輪徑。他們使用的輪子標稱直徑 65 公釐，但實際裝上後因為橡膠輪胎受壓變形，有效直徑只有約 60 公釐。里程計以 65 公釐計算，因此「以為」自己走得比實際遠，建出的地圖就被等比例放大。

這個案例的教訓有三層：

- 第一，規格書不等於實際值。輪胎變形、製造公差、安裝誤差都會造成差異，必須實測校正（29.3 節與公式三）。
- 第二，系統性誤差不會被 SLAM 修正。SLAM 能修正隨機的漂移（靠掃描匹配與閉環），但對「每一步都固定放大 8%」的系統性誤差無能為力——因為所有資料都一致地錯，系統無從察覺。
- 第三，調參之前先檢查基礎。這正是 29.7 節強調的排查順序：上游的問題，調下游的參數永遠解決不了。

建議教師在課程一開始就要求同學完成里程計校正並記錄數據——這半小時能省下後續數小時的困惑。

案例 29-2　從模擬到實體：預期之外的落差

第 28 章建議「先模擬、再實體」，但同學常會遇到一個挫折：在 Gazebo 中跑得完美的系統，搬到實體小車上卻問題百出。這個落差來自哪裡？

主要有四個來源：

- 感測器雜訊：模擬中的光達雖然可加雜訊，但真實光達還有第 8 章談過的各種問題——玻璃穿透、黑色物體反射微弱、量測距離的隨機跳動。這些在模擬中往往未被完整建模。
- 里程計品質：模擬中的輪速通常是理想的；真實世界有打滑、地面材質變化、輪胎變形（案例 29-1）、以及通訊丟包造成的資料斷續。
- 時間同步：模擬中所有節點共用同一個模擬時鐘，時間戳記完美一致；真實系統中，微控制器、單板電腦、感測器各有時鐘，同步問題會浮現（第 5、10、27 章反覆強調的工程細節）。
- 環境複雜度：模擬場景通常乾淨簡單；真實教室有桌椅腳（第 8 章的角解析度問題）、玻璃窗、走動的人。

但這不表示模擬沒有價值——恰恰相反。模擬的價值在於：它讓你先確認「邏輯與架構是對的」，把 TF 設定、話題連接、參數配置等問題先解決掉。搬到實體時，剩下的就只是「真實世界的物理問題」，問題範圍大幅縮小。

務實的建議是：模擬中通過的系統，實體上仍要預留一到兩倍的除錯時間。這不是失敗，而是工程的常態。

案例 29-3　為什麼機器人規劃出路徑卻不動

這是導航實作中最令人困惑的症狀之一：RViz 中明明看到一條漂亮的綠色路徑，機器人卻紋風不動。

依 29.7 節的排查順序，問題必然在第七或第八個檢查點——控制器或底盤。常見的原因包括：

23. 控制器沒有發布速度指令：可能是控制器節點未啟動、或參數設定錯誤導致它認為「無法執行這條路徑」。檢查方法是直接監看 /cmd_vel 話題有無資料。

24. 速度指令發布了但底盤沒收到：話題名稱不符（第 28.4 節提過的重新對應問題）、或底盤驅動節點未訂閱。
25. 底盤收到了但不動：速度指令的數值太小，低於馬達的啟動門檻；或馬達驅動器未啟用、電源未開。
26. 速度限制設得太保守：Nav2 的最大速度參數設得過小，或加速度限制過嚴，使機器人「動得太慢看起來像不動」。

這個案例的價值在於它示範了系統性除錯的威力：同樣的症狀，若隨機猜測可能花數小時；依循「監看 /cmd_vel → 判斷是控制器還是底盤 → 再往下細查」的順序，通常十分鐘內就能定位。

這也呼應第 28.6 節閱讀開源專案的方法——順著資料流追蹤。無論是讀程式碼還是除錯系統，「follow the data」都是最可靠的策略。

實作活動

實作 29-1　從建圖到自主導航的完整實作

目的：完成本書第一個端到端的系統實作。依圖 29-5 的六個階段循序進行，每階段都產出可驗證的證據。本實作可在模擬中完成，有硬體者建議先模擬再實體。

階段一：座標系統設定（全體）

5. 若使用現成平台，檢視其 URDF，找出 base_link 與 laser 之間的變換，並與實際安裝位置核對。若自組平台，則撰寫自己的 URDF。
6. 啟動系統並產生 TF 樹圖，確認 map → odom → base_link → laser 鏈路完整無斷裂。
7. 外參驗證：把機器人靜置在距離牆面已知距離處（例如 1 公尺），在 RViz 中檢查掃描點是否落在正確位置。若有偏差，回頭修正 URDF。

8. 刻意破壞實驗：故意把 URDF 中光達的位置改錯 20 公分，觀察 RViz 中掃描點如何錯位——親身理解外參的重要性。改回正確值後繼續。

階段二：里程計校正（有硬體者必做，模擬者可略）

9. 依圖 29-4 執行輪徑校正：直線行走 3 公尺，比較里程計與捲尺實測值，修正參數後重驗。記錄修正前後的誤差百分比。
10. 依圖 29-4 執行輪距校正：原地旋轉 10 圈，比較累計角度與 3600 度，修正輪距後重驗。
11. 平滑度檢查：遙控機器人移動，觀察里程計輸出是否連續平滑、有無跳變（29.3 節的品質要求）。

階段三：建圖（全體）

12. 啟動 slam_toolbox 建圖，緩慢遙控機器人繞行場域一圈，在 RViz 中觀察地圖成長。
13. 依第 13.7 節評估地圖品質：牆是否筆直？迴路是否閉合？量測地圖上的關鍵尺寸與實際比較，計算誤差百分比。
14. 速度對照實驗：分別以「很慢」與「較快」的速度各建一次圖，比較品質差異，驗證第 14.6 節「開慢一點」的建議。
15. 儲存地圖，檢視產生的 .yaml 與 .pgm 兩個檔案，用看圖軟體開啟 .pgm 觀察佔據格的三態表示。

階段四：定位（全體）

16. 啟動 map_server 載入剛才儲存的地圖，確認 RViz 中能看到地圖。
17. 啟動 AMCL，在 RViz 中顯示粒子雲。觀察未設定初始位姿時，粒子如何散布全圖。
18. 用 RViz 的位姿估計工具設定初始位姿，遙控機器人移動數步，觀察粒子雲如何收斂——這是第 11 章案例 11-2 的親身重現。

19. 綁架測試：把機器人（或模擬中的機器人）瞬移到地圖上的另一個位置，觀察 AMCL 是否能重新收斂，以及花了多久。

階段五：自主導航（全體）

1. 啟動 Nav2，在 RViz 中顯示全域與局部代價地圖，觀察障礙物的膨脹效果。
2. 用 RViz 的目標點工具送出一個導航目標，觀察規劃出的路徑，以及機器人如何沿路徑移動。
3. 動態避障測試：在機器人行進途中放置一個障礙物（模擬中可加入物件），觀察局部代價地圖如何更新、機器人如何繞行。
4. 膨脹半徑實驗：分別把膨脹半徑設為機器人半徑的 0.8 倍、1.2 倍、2.5 倍，觀察路徑的變化與是否出現「規劃不出路徑」的情況（公式四的討論）。
5. 復原行為觀察：把機器人困在死角或送出無法抵達的目標，觀察 Nav2 觸發哪些復原行為。

階段六：除錯練習（大學部）

1. 人為製造故障：依表 29-3 選三種症狀，分別人為製造（例如關閉某個節點、改錯話題名稱、把膨脹半徑設到極大），再依圖 29-3 的順序排查，記錄從發現症狀到定位問題所花的時間。
2. 建立你自己的檢查表：依本次實作的經驗，整理一份適合你的平台的除錯檢查表，供日後與同學使用。

討論問題

6. 外參改錯 20 公分後，RViz 中的掃描點如何錯位？若不修正就建圖，地圖會出現什麼問題？
7. （有硬體者）輪徑校正前後的誤差各是多少？這個誤差若不修正，對地圖尺寸的影響有多大？（對照案例 29-1）
8. 慢速與快速建圖的品質差多少？從第 13.3 節掃描匹配的原理解釋這個差異。

9. 綁架測試中 AMCL 花了多久收斂？若地圖中有兩個外觀相似的區域，你預期會發生什麼？（提示：第 11 章的多峰分布）
10. 膨脹半徑過大時，發生了什麼？過小時呢？你的場域最合適的值是多少？
11. （大學部）在除錯練習中，依系統性順序排查比隨機猜測快多少？這對你日後的工程習慣有什麼啟示？

延伸挑戰（大學部）

其一：完整的系館場域導航系統——在系館的一個樓層完成建圖，設定數個具名的目標點（如「實驗室門口」「電梯口」「茶水間」），並撰寫一個 Python 節點，讓使用者輸入地點名稱後機器人自動導航前往。這實際上是第 24 章語意導航的簡化版，也是第 31 章專題的良好起點。其二：里程計融合實驗——若平台有 IMU，設定 robot_localization 的 EKF 節點融合輪速與 IMU，比較「純輪速里程計」與「融合後里程計」在快速轉彎與地面打滑情況下的差異，量化 IMU 帶來的改善。這是第 7、11 章理論的直接應用。

延伸閱讀

8. Nav2 官方文件（docs.nav2.org）.（架構說明、設定指引與教學，本章主要依據）
9. Nav2 官方文件之 Setup Guides:Setting Up Sensors、Mapping and Localization.（從零建立機器人的完整步驟）
10. slam_toolbox 開源專案（GitHub: SteveMacenski/slam_toolbox）.（ROS 2 預設的 2D SLAM，程式規模適中適合閱讀）
11. S. Macenski et al., "The Marathon 2: A Navigation System," IROS, 2020.（Nav2 的官方論文，說明設計理念）
12. ROS 2 官方文件之 URDF 教學（docs.ros.org）.（機器人描述檔的撰寫方法）
13. ROBOTIS TurtleBot3 e-Manual 之 SLAM 與 Navigation 章節.（教學平台的完整實作指引）

14. robot_localization 套件官方文件.（以 EKF 融合多感測器提供里程計，29.3 節的進階內容）

小結

8. 2D SLAM 小車由底盤馬達、編碼器、2D 光達、微控制器與單板電腦組成；微控制器負責毫秒級即時控制、單板電腦負責 SLAM 運算，這個分工與第 25 章的雙晶片架構思想一致。
9. URDF 描述機器人結構與感測器安裝位置（外參）；TF 樹的分工是：SLAM 或 AMCL 發布 map→odom、里程計系統發布 odom→base_link、robot_state_publisher 依 URDF 發布 base_link 到各感測器。常見錯誤為斷裂、外參填錯、重複發布與時間戳記不同步。
10. 里程計由編碼器讀值經差速驅動公式計算並發布；輪徑與輪距必須實測校正，輪距誤差因直接造成角度誤差而影響更嚴重；里程計不需絕對準確但必須平滑連續，可用 EKF 融合 IMU 提升穩健度。
11. slam_toolbox 是 Nav2 的預設 SLAM 演算法、參數簡潔適合教學；建圖流程須先確認 TF 完整，緩慢遙控並持續評估品質；地圖以 .yaml 與 .pgm 兩檔儲存。
12. AMCL 以粒子濾波在已知地圖中定位、不建立地圖，與 SLAM 同樣提供 map→odom 變換因而可無縫切換；關鍵參數包含粒子數、運動雜訊模型、更新門檻與變換容許時間；初始位姿必須設定，否則可能收斂緩慢或錯誤。
13. Nav2 的導航目標流經行為樹導航器、規劃器、控制器、代價地圖、復原行為與速度指令；代價地圖以 0 至 254 的成本值表示，0 為自由、254 為致命佔據，並以靜態層、障礙層、膨脹層等多層疊加。
14. 除錯必須依系統性順序：生命週期、TF、地圖、定位、代價地圖、規劃器、控制器、底盤；規劃器與控制器的調參應等到上游都健康之後才進行；每個檢查點都應留下可驗證的證據。

承先啟後

本章把三個最容易卡住專題的環節量化了：里程計方面，輪距設錯 4% 加上左右輪不等徑 1.5%，走一個 2 公尺見方的方形就會偏 1.13 公尺，而且順時針與逆時針的誤差差了三倍多（1.13 對 0.32 公尺）——只跑一個方向會被騙；座標轉換方面，光達位置差 0.20 公尺或角度差 5 度，都會讓障礙格數從 992 增加到兩千多、正確率掉到七成多，而且兩者的症狀從外觀分不出來；定位方面，全域定位的粒子散布從 3.49 公尺收斂到 0.17 公尺，但真正的收斂發生在第 5 到 8 步——啟動後必須先走一段路，定位才會穩。

本章完成了第一個從感測器到自主導航的端到端系統。同學應該能感受到：前面二十七章的理論，在這一章全部匯流成一台真正會動的機器人——第 8 章的光達、第 10 章的座標轉換、第 11 章的粒子濾波、第 12 章的圖優化、第 13 章的佔據格、第 15 章的 SLAM 系統、第 28 章的工具鏈，缺一不可。

下一章，我們把感測器換成相機。RealSense 與 ZED 等相機的設定、相機校正、RGB-D 點雲產生、RTAB-Map 與 ORB-SLAM3 的實作，以及最重要的——視覺 SLAM 與光達 SLAM 在同一場域的對照比較。第 30 章將讓同學親身驗證第 8、9 章反覆討論的感測器互補性。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 30 章

RGB-D / 雙目相機 SLAM 實作

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

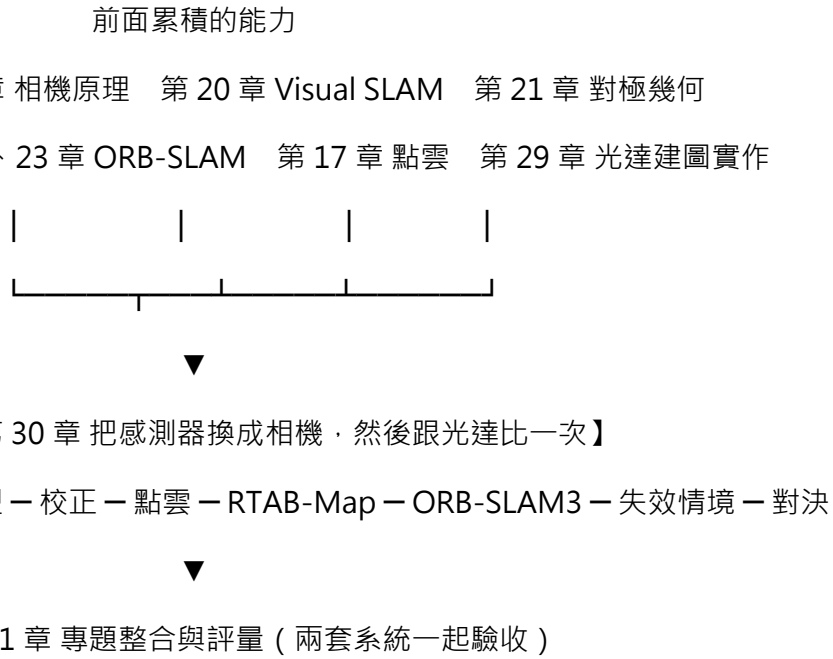
本章在全書的位置

這是第八篇的最後一章技術實作，也是全書唯一一次讓兩套感測器在同一個場域、走同一條路徑、正面比一次。

上一章的光達小車，困難在「接線」——座標轉換、里程計參數、一層壓一層的系統整合。本章換成相機之後，困難換了一種性質：相機不像光達那樣自帶光源、也不像光達那樣直接量距離。它的深度是「算」出來的，而凡是算出來的東西，就會在某些條件下算得很準、在另一些條件下算得離譜，而且常常不會告訴你它算錯了。

本章的三張圖，就是要把這三種「條件」量出來：圖 30-1 實測雙目深度誤差怎麼隨距離成長，順便回答「有效範圍五公尺」這句話到底是什麼意思；圖 30-2 把深度與彩色沒對齊的後果畫出來，並指出它的嚴重程度與距離的關係正好和深度誤差相反；圖 30-4 則是本書醞釀了二十幾章的對決——貼滿海報、白牆、玻璃、昏暗四種房間，光達與視覺各跑一圈。

圖 30-0　第 30 章的連結地圖



學習目標

上一章我們用光達完成了完整的建圖與導航。本章把感測器換成相機，實作視覺 SLAM——

並在最後做一件本書醞釀已久的事：在同一個場域，讓光達與視覺兩套系統正面對決。學完本章之後，同學應能夠達成下列目標：

15. 比較 RealSense、ZED、OAK-D 三類深度相機的原理與適用場合。
16. 完成相機校正並驗證內參與畸變係數的正確性。
17. 由 RGB 與深度影像產生彩色點雲，並理解對齊的重要性。
18. 執行 RTAB-Map 建圖並儲存三維地圖。
19. 執行 ORB-SLAM3 並完成軌跡評測。
20. 規劃室內場景的測試流程，並辨識視覺 SLAM 的典型失效情境。
21. 在同一場域比較視覺 SLAM 與光達 SLAM 的表現差異。

核心概念

中文名詞	英文名詞	概念說明
深度影像	Depth Image	每個像素記錄該點距離的影像。
深度對齊	Depth Alignment	把深度影像對齊到彩色影像的座標系。
彩色點雲	Colored Point Cloud	每個三維點帶有 RGB 顏色的點雲。
相機資訊	CameraInfo	ROS 中攜帶內參與畸變係數的訊息。
視覺里程計	Visual Odometry	由影像估計相機運動，提供 odom→base_link。
RGB-D 里程計	RGBD Odometry	RTAB-Map 提供的以 RGB-D 影像估計位姿的模組。
外觀式建圖	Appearance-based Mapping	以影像外觀相似度偵測閉環的建圖方法。
記憶體管理	Memory Management	RTAB-Map 控制地圖規模以維持即時性的機制。
資料庫檔	Database File	RTAB-Map 儲存地圖與影像的檔案格式。

中文名詞	英文名詞	概念說明
定位模式	Localization Mode	載入既有地圖只做定位、不再擴充地圖。
全域快門	Global Shutter	全畫面同時曝光，適合運動中的視覺。
主動紅外投射	IR Projector	投射紋理圖案以協助立體匹配的裝置。
近似同步	Approximate Sync	容許不同話題時間戳記略有差異的同步方式。

30.1 RealSense / ZED / OAK-D 相機介紹

第 9.3 節介紹過三種取得深度的方式，本節聚焦實際可用的產品與選型考量。

三大類深度相機

- RealSense 系列：採主動紅外雙目（第 9.3 節）。它以改良的硬體與軟體達成室內外皆可用的十公尺以上量測範圍，並具備全域快門與寬視野，對機器人與無人機應用相當有用。部分型號內建 IMU，可直接支援視覺慣性 SLAM（第 23 章）。
- ZED 系列：採被動雙目，不投射紅外線，因此在戶外強光下仍可運作（這正好補上 RealSense 的弱點）。它一般能提供比 RealSense 更佳的深度精度與更寬的視野，但代價是需要 GPU 運算——ZED 的深度計算倚賴 CUDA，必須搭配 NVIDIA 顯示卡。
- OAK-D 系列：雙目加上內建的 AI 推論晶片。它能在相機端直接執行物件偵測等神經網路，把運算從主機卸載出去——這與第 25 章邊緣運算的思想一致，對運算資源有限的平台特別有價值。

選型的四個考量

27. 使用環境：室內為主可用 RealSense；需要戶外強光下運作則考慮 ZED 或被動雙目。第 9.3 節提過主動紅外投射會被陽光淹沒。
28. 運算資源：ZED 需要 NVIDIA GPU;RealSense 的深度由相機內部處理，對主機需求較低；OAK-D 甚至能把 AI 推論也搬到相機上。
29. 是否需要 IMU：若要做視覺慣性 SLAM（第 23 章），選內建 IMU 的型號能省去外部 IMU 的同步與外參校正麻煩。
30. 有效範圍：以 RealSense D435 為例，它的量測範圍達 10 公尺，但在 5 公尺範圍內精度較佳——實務上常把有效範圍設為 5 公尺。這呼應第 9.6 節「深度誤差隨距離平方成長」的討論。

類型	深度原理	強光下	運算需求	特色
RealSense	主動紅外雙目	較弱	低（相機內處理）	部分型號內建 IMU
ZED	被動雙目	可用	需 NVIDIA GPU	精度與視野較佳
OAK-D	雙目 + AI 晶片	可用	極低（相機端推論）	可在相機上跑神經網路

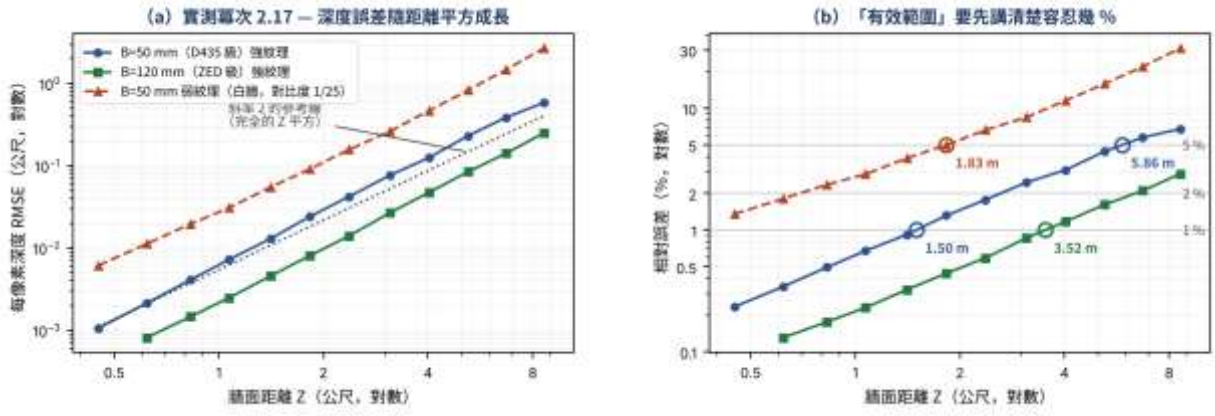
表 30-1　三類深度相機的比較

在往下談校正與點雲之前，先把選型這一節最關鍵、卻也最常被含糊帶過的一句話量出來：「有效範圍五公尺」。

前面說 D435 的量測範圍達十公尺、但五公尺內精度較佳。這句話其實不完整——精度從來不是在某個距離突然「變差」，它是連續惡化的，所以「有效範圍」這四個字只有在講清楚「容忍幾 % 的誤差」之後才有意義。下面這組數字是實際跑出來的，不是套公式。

實驗這樣做：用隨機斑點紋理產生一面傾斜的牆，依雙目幾何算出每個像素應有的視差，渲染出左右兩張 640×480 影像，加上標準差 2.5 灰階的感測雜訊，再交給 OpenCV 的 StereoSGBM 去匹配，最後把它算出來的視差換回深度，和真值逐像素相減。三組條件：基線 50 mm（D435 等級）、基線 120 mm（ZED 等級）、以及同樣 50 mm 但把紋理對比度降到二十五分之一（模擬白牆）。

圖 30-1　雙目深度誤差的實測成長：距離平方、基線、紋理三者的作用



圖（a）是對數座標。三條線都是直的，而且斜率量出來是 2.17、2.19、2.05——這就是教科書上「深度誤差隨距離平方成長」那句話的實測版本。要注意的是這個平方關係並非來自任何人為設定：程式裡沒有任何一行寫過 Z 的平方，它是雙目幾何（深度與視差成反比）加上匹配誤差自然長出來的。基線 50 mm 的深度 RMSE 從 0.45 m 處的 1.1 公釐，一路長到 8.6 m 處的 58 公分，相差五百多倍。

圖（b）把同樣的數字換成相對誤差，「有效範圍」這句話就有了可以回答的形式。以 50 mm 基線、強紋理來說：要求 1 % 的相對精度，有效範圍只有 1.50 公尺；放寬到 2 % 是 2.61 公尺；放寬到 5 % 才終於接近課本上常說的五公尺（實測 5.86 公尺）。所以「D435 的有效範圍五公尺」這句話，隱含的其實是「可以接受 5 % 誤差」——在五公尺處那是 25 公分，用來避障綽綽有餘，用來做三維量測則完全不行。

第二條線回答了選型那一節的另一個問題：基線加大到底買到什麼。120 mm 基線（2.4 倍）在 1 % 精度下的有效範圍是 3.52 公尺，恰好是 50 mm 的 2.35 倍。這不是巧合：相對誤差正比於 Z / B，所以在固定精度要求下，有效距離與基線成正比。要把有效範圍加倍，最直接的辦法就是把兩顆鏡頭拉開一倍——這也是為什麼戶外用的雙目相機都又寬又大。

第三條線是本章後面「五種典型失效情境」的第一個量化證據。同一顆相機、同一個距離，只是把紋理對比度降到二十五分之一，五公尺處的誤差就從 4.4 % 惡化到 15.6 %，差了 3.5 倍；1 % 的有效範圍更是直接掉到一公尺以內。但真正要注意的不是誤差變大，而是下面這

件事。

白牆不會給你「無效」，它會給你一個錯得很離譜的數字

弱紋理那一組，SGBM 回報的有效像素比例仍然是 99.5 %。

也就是說，面對白牆時相機並沒有拒絕作答，它照樣填滿整張深度圖，只是填進去的值在五公尺處平均錯了 80 公分。

這是視覺感測器最危險的性格：光達打到吸光的黑色物體會直接回報無效點，你一眼就看得出來；相機卻會「有自信地答錯」。

實務上的對策是不要只看深度圖有沒有洞，而要看驅動提供的信心度或視差有效遮罩，並對同一面牆重複量測看數值穩不穩。

教學上的建議

對大學與高中的教學而言，建議優先考慮內建 IMU 的 RealSense 型號：它價格相對親民、ROS 2 驅動成熟、社群資源豐富、且不需額外的 GPU。等到專題階段需要戶外運作或更高精度時，再考慮 ZED。

30.2 相機校正

第 9.2 節說明過相機校正的原理，本節聚焦實作要點——這是視覺 SLAM 的第一道關卡，也是精度的基礎。

出廠參數 vs 自行校正

現代深度相機出廠時已經過校正，並會透過 ROS 的 CameraInfo 訊息提供內參與畸變係數。對多數應用而言，直接使用出廠參數即可。

但在三種情況下，建議自行校正：

- 相機受過碰撞或鏡頭被調整過。

- 使用一般的網路攝影機或自組的雙目系統（沒有出廠參數）。
- 對精度要求極高的研究應用，想確認出廠參數的準確性。

校正流程回顧

依第 9.2 節的張氏棋盤格法：

31. 列印棋盤格並貼在硬板上（務必保持平整，彎曲會嚴重影響校正精度）。
32. 從不同角度、距離拍攝 15 至 20 張，涵蓋畫面各區域——尤其別忘了邊角，徑向畸變在邊角最嚴重。
33. 執行校正程式，取得內參矩陣 K 與畸變係數。
34. 檢查重投影誤差，應小於 1 像素；若偏大，增加影像數量或重拍。
35. 驗證：對含直線的場景做去畸變，確認彎曲的直線被拉直。

ROS 中的校正工具

ROS 提供 camera_calibration 套件，能以互動方式完成校正：對著棋盤格移動相機，工具會即時提示涵蓋度（左右、上下、遠近、傾斜），達標後即可計算並儲存參數。這比自己寫程式方便得多，是實作首選。

雙目的額外校正

雙目相機除了各自的內參，還需要兩相機之間的相對位姿（外參）——這決定了基線長度，直接影響深度計算（第 9.6 節的公式 $Z = f \cdot b / d$）。雙目校正同樣可用 camera_calibration 套件完成，但需要同時拍攝左右影像。

常見的校正陷阱

- 棋盤格不平整：貼在紙板或曲面上會導致校正結果錯誤。務必用硬質平板。
- 影像涵蓋不足：只在畫面中央拍攝，邊角的畸變係數就估不準。務必讓棋盤格出現在畫面的各個角落。

- 解析度不一致：校正時與實際使用時的影像解析度必須相同，否則內參的像素單位會不匹配。
- 忘記更新：校正完成後，務必確認 SLAM 系統實際讀取的是新的參數檔，而非舊的或預設值。

30.3 RGB-D 點雲產生

有了 RGB 影像、深度影像與相機內參，就能產生彩色點雲——這是視覺 SLAM 的核心資料形式，也是第 17 章點雲處理技術的來源之一。

從深度影像到三維點

原理是第 9.1 節針孔投影模型的逆運算。已知某像素的座標與該點的深度，即可反推出它的三維座標（公式見本章公式一）。對整張深度影像的每個像素執行這個運算，就得到一片點雲。

再把對應像素的 RGB 顏色賦予每個點，就成為彩色點雲——這正是第 17.6 節點雲顏色化的簡化版：因為 RGB 與深度來自同一台相機，不需要處理跨感測器的外參與時間同步問題。

關鍵前提：深度對齊

這裡有一個初學者常忽略的細節：彩色相機與深度感測器在物理上位於不同位置，因此同一個像素座標在兩張影像中對應的並非同一個空間點。

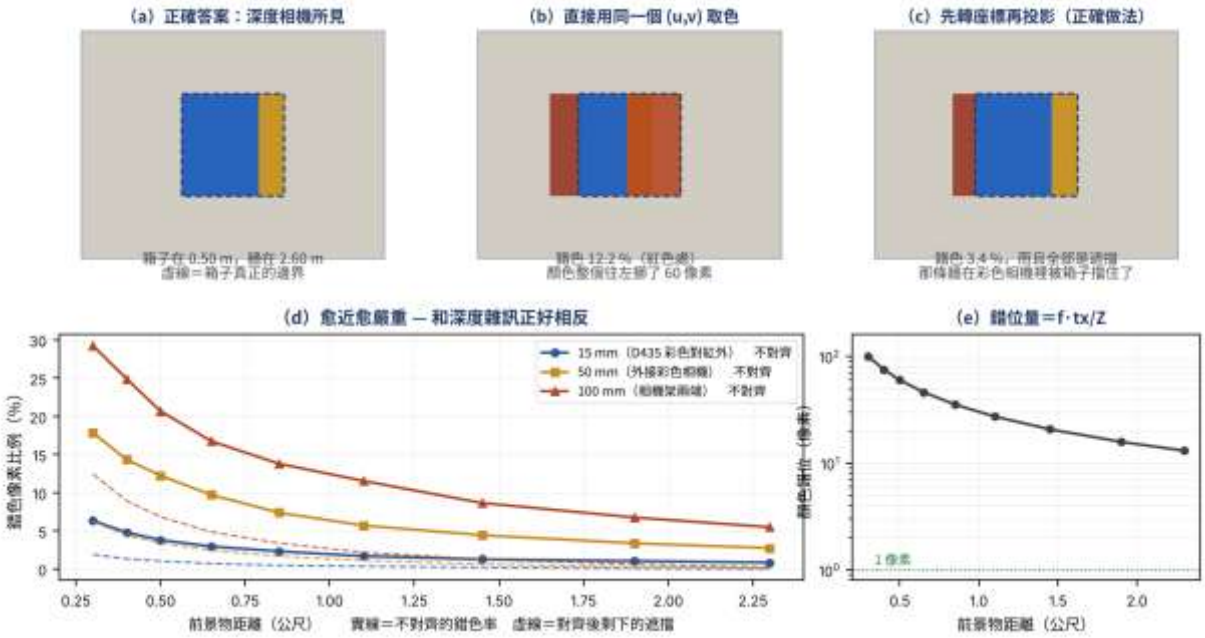
解決方法是深度對齊——把深度影像投影到彩色相機的視角，使兩者的像素一一對應。實務上這對應到相機驅動中的對齊設定，產生的話題會是「對齊到彩色影像的深度影像」；RTAB-Map 也正是訂閱這個對齊後的深度話題。

若忘了開啟對齊，產生的點雲會出現顏色與幾何錯位——牆的顏色跑到窗戶上，這與第 17.6 節提過的外參錯誤症狀相同。

「忘了開啟對齊，點雲就會顏色與幾何錯位」這句話，聽起來像是美觀問題。把它量出來之後會發現它不只是美觀問題，而且它的嚴重程度與距離的關係，和剛剛圖 30-1 的結論正好相反。

實驗場景很簡單：一面 2.6 公尺遠的牆，前方放一個箱子，箱子正面漆藍色、右側漆橘色，牆是米白色。深度感測器與彩色鏡頭在物理上錯開 tx。上色分兩種做法：正確做法是把深度像素反投影成三維點、轉到彩色相機座標、再用彩色內參投影回去取色；錯誤做法則是直接拿同一組 (u, v) 去彩色影像取色。

圖 30-2　深度沒對齊到彩色，彩色點雲會錯成什麼樣



圖（a）是正確答案，虛線框標出箱子真正的邊界。圖（b）是直接用同一個 (u, v) 取色的結果，紅色就是上錯顏色的像素。可以清楚看到整塊顏色往左挪了 60 個像素：箱子右側有一條帶狀區域被塗上了牆的顏色，而箱子左邊的牆面則被塗成了箱子的藍色。錯位量有一條很乾淨的公式，就是圖（e）畫的 f·tx / Z——這個例子裡是 600 × 0.050 ÷ 0.50 = 60 像素。整張畫面有 12.2 % 的像素上錯了顏色。

圖（c）是先轉座標再投影的正確做法。錯色率從 12.2 % 掉到 3.4 %——但沒有掉到零，而剩下的那 3.4 % 值得單獨講一下。

把遮擋單獨算出來之後發現：對齊之後剩下的錯色像素，100 % 都落在遮擋區，非遮擋的殘餘是 0.00 %。原因是幾何上的必然——靠近箱子邊緣的那一條牆，在彩色相機的視角裡被箱子擋住了，彩色影像上根本沒有它的顏色可取。這條帶子無論對齊做得多好都補不回來，它是兩顆鏡頭分處兩地這件事本身的代價。這也解釋了為什麼實務上的彩色點雲在物體邊緣總有一圈顏色可疑的點：那不是你的程式寫壞了。

圖（d）是本節最重要的一張。三條實線是三種鏡頭間距（15 mm 的 D435 彩色對紅外、50 mm 與 100 mm 的外接彩色相機）在不同物距下的錯色率，虛線是對齊後剩下的遮擋下限。三條線都是「愈近愈嚴重」：D435 那條在 0.30 公尺處錯色 6.3 %，到 2.30 公尺只剩 0.84 %；100 mm 那條在 0.30 公尺處更高達 29.3 %，也就是將近三分之一的點顏色是錯的。

兩種誤差往相反方向拉，所以沒有一個「最佳距離」

圖 30-1 說：深度誤差隨距離平方成長，所以愈遠愈糟。

圖 30-2 說：顏色錯位量是 f·tx / Z，所以愈近愈糟。

以 D435 為例，從 0.5 公尺退到 2.3 公尺，深度 RMSE 大約放大 38 倍，錯色率卻縮小到四分之一。

兩者的方向相反，代表「把相機拿近一點比較準」這個直覺只對了一半：它對深度對，對顏色錯。

真正的結論是——先決定這份點雲要拿來做什麼。要做幾何量測就離遠一點不行、離近一點顏色就不能信；要做語意或材質辨識則反過來。兩個都要的話，唯一的解法是把對齊確實打開，把兩條曲線中可以消掉的那一條消掉。

ROS 中的資料流

圖 30-3 RGB-D 相機到點雲的資料流

相機驅動節點

├──→ /color/image_raw 彩色影像

├──→ /aligned_depth_to_color/image_raw 對齊後的深度影像

├──→ /color/camera_info 內參與畸變係數

└──→ /imu（若相機內建） 慣性資料

↓

點雲產生節點（或 SLAM 系統內部）

依公式一把每個像素反投影為三維點，並賦予顏色

↓

└──→ /points PointCloud2 格式的彩色點雲（第 8.3 節）

資料量的現實

一張 640×480 的深度影像會產生約 30 萬個點，以 30 幀每秒計算，每秒就是 900 萬個點——這正是第 17.2 節必須降採樣的原因。實務上會先做體素降採樣，再交給後續處理。

另一個實務考量是有效範圍的裁切：如 30.1 節提過，深度相機在較遠處的精度快速下降，因此常設定一個最大有效距離（例如 5 公尺），超過的點直接捨棄——保留不可靠的遠點，反而會拉低整體精度。

30.4 RTAB-Map 實作

RTAB-Map 是 RGB-D SLAM 最常用的開源系統，也是本章的實作主角。

RTAB-Map 是什麼

RTAB-Map（Real-Time Appearance-Based Mapping）是一套基於增量式外觀閉環

偵測器的 RGB-D 圖優化 SLAM 方法，設計為可通用於室內與室外環境。

它使用相機接收到的深度影像執行圖優化 SLAM，產生稠密的彩色點雲與相機里程計；它也能以詞袋與特徵偵測器即時完成閉環。它是一套支援 RGB-D 相機的圖優化 SLAM 框架，同時運用視覺外觀與深度資訊進行三維建圖與閉環偵測。

對照本書的架構，同學應該能立刻定位它：圖優化是第 12 章、詞袋閉環偵測是第 20、22 章、稠密點雲是第 17 章——RTAB-Map 是這些技術的整合實現。

三項實用特性

- 可用外部里程計：它允許你傳入自己的里程計話題，而非由它自行產生——若你有更精確的里程計（例如第 29 章的輪速里程計或融合 IMU 的結果），可直接使用。這對輪型機器人特別有價值。
- 可融合雷射：你也能使用自己的 2D 或 3D 雷射掃描來改善地圖品質。這意味著 RTAB-Map 能同時吃視覺與光達資料——正是第 8、9 章反覆強調的感測器互補在實作上的體現。
- 多種地圖格式：它相容於八叉樹與體素等資料結構，這些對三維規劃與多地圖網格化的效率都優於原始點雲。這呼應第 17.5 節三種地圖表示的討論。

RGB-D 里程計

RTAB-Map 套件也提供名為 RGBD 里程計的視覺里程計，用來提供位姿資訊與 odom→base_link 變換，在室內運作已展現可靠的結果。

這一點很重要：即使機器人沒有輪速編碼器（例如手持相機掃描），RTAB-Map 也能自己產生里程計，完整運作。這讓它成為「一台相機就能做 SLAM」的實用方案。

執行流程

36. 啟動相機驅動，務必開啟深度對齊（30.3 節）。
37. 確認三個必要話題有資料：彩色影像、對齊後的深度影像、相機資訊。

38. 啟動 RTAB-Map，指定上述話題名稱與座標框架。首次執行建議加上「啟動時刪除資料庫」的參數，避免載入舊地圖造成混淆。
39. 在 RViz 或 RTAB-Map 自己的視覺化介面中觀察：即時點雲、相機軌跡、以及閉環偵測的發生。
40. 緩慢移動相機掃描整個空間，注意保持特徵豐富的視野。
41. 完成後儲存地圖（RTAB-Map 以資料庫檔案儲存，內含影像、特徵與位姿圖）。
42. 日後可切換到定位模式，載入既有地圖只做定位、不再擴充——這與第 29.5 節 AMCL 的角色相同。

關鍵參數

- 同步方式：若彩色與深度影像的時間戳記完全一致，可用精確同步；若來自不同來源或有微小差異，則需啟用近似同步，否則系統會因等不到配對而完全不動。這是實作中常見的卡關點。
- 最大深度：設定有效範圍上限（30.3 節），超過的點捨棄。
- 記憶體管理：RTAB-Map 有機制控制地圖規模以維持即時性——當地圖過大時，較舊的部分會被移到「長期記憶」中，只在需要時取回。這讓它能處理大範圍場景。

30.5 ORB-SLAM3 實作

第 23 章完整剖析過 ORB-SLAM3 的原理，本節聚焦實作要點。

與 RTAB-Map 的定位差異

面向	RTAB-Map	ORB-SLAM3
主要輸出	稠密彩色點雲地圖	稀疏特徵點地圖
ROS 整合	原生支援，開箱即用	需社群封裝或自行整合
適合任務	三維重建、導航地圖	高精度定位、軌跡評測

面向	RTAB-Map	ORB-SLAM3
視覺化	點雲模型直觀	特徵點與關鍵幀
本書章節	本章	第 23 章原理、本章實作

表 30-2 RTAB-Map 與 ORB-SLAM3 的定位比較

簡言之：要「看得見的三維模型」用 RTAB-Map；要「精確的軌跡與定位」用 ORB-SLAM3。兩者不是替代關係，而是不同任務的工具。

實作要點

43. 編譯：相依套件與第 22.7 節相同（OpenCV、Eigen3、Pangolin）。注意版本相容性，建議參考社群維護的分支或使用容器（第 28.1 節的建議）。
44. 組態檔：準備 YAML 檔，填入 30.2 節校正得到的內參與畸變係數、影像尺寸、以及 ORB 特徵參數（每幀特徵數、金字塔層數）。
45. 模式選擇：依相機能力選擇單目、雙目、RGB-D，以及是否啟用慣性模式（需相機內建 IMU）。
46. IMU 初始化：若使用慣性模式，啟動後務必做帶加減速與旋轉的運動，否則初始化不會成功（第 23.4 節反覆強調的運動激勵）。
47. 觀察：在 Pangolin 視窗中觀察特徵點、關鍵幀、共視圖與閉環——第 22.6 節的資料結構在此具象化。
48. 軌跡輸出與評測：輸出估測軌跡，以 evo 工具計算 ATE 與 RPE（第 22.7 節）。單目模式記得先做尺度對齊。

三種模式的實測比較

同學可在同一場域依序測試三種模式，親身驗證第 20.6 節表 20-3 的理論比較：

- 單目：初始化最困難（需足夠視差，第 21.5 節）、軌跡無絕對尺度、但硬體最簡單。

- RGB-D：初始化最容易（第一幀就有深度）、有絕對尺度、地圖最稠密，但受限於深度相機的有效範圍與強光。
- 雙目慣性：最穩健、精度最佳（第 23.7 節提過的毫米級成績），但需要相機支援與較多的設定。

30.6 室內場景測試

本節說明如何設計有教學價值的測試，特別是「刻意尋找失效邊界」的實驗。

基本測試流程

49. 環境準備：選擇紋理豐富、光線均勻的空間開始（例如有書架、海報的教室），先建立成功經驗。
50. 緩慢掃描：手持相機或裝在小車上，以緩慢平穩的速度移動，涵蓋整個空間。
51. 閉環測試：繞行一圈回到起點，觀察閉環是否觸發、地圖是否被拉正（第 12、15 章的震撼畫面在此重現）。
52. 品質評估：量測地圖上的關鍵尺寸與實際比較；檢查牆面是否「變厚」（第 17 章案例 17-1 的三維品質指標）。

五種典型失效情境

這是本節最有價值的部分——刻意製造失敗，理解系統的邊界。

20. 白牆與弱紋理：對著大面積白牆掃描，觀察特徵點如何歸零、系統如何跟丟。這是第 9、20 章反覆提過的「白牆詛咒」，在此親身驗證。
21. 快速運動：這是視覺 SLAM 最常見的失效原因。移動或轉動太快會中斷成功的點雲匹配序列，導致系統跟丟。同學可逐步提高移動速度，找出系統開始失敗的臨界點。
22. 光線變化：從明亮處走進昏暗處，或正對窗戶造成過曝，觀察追蹤是否中斷。這是第 9.5 節「依賴光線」的具體呈現。

23. 玻璃與鏡面：對著落地窗或鏡子掃描，觀察深度資料出現什麼異常。這與第 8 章光達的玻璃問題是同一類困難。
24. 動態物體：讓人在畫面中走動，觀察是否影響定位。第 24.4 節提過可用物件偵測剔除動態物體的特徵，此處先觀察不處理時的後果。

一個實務上的重要提醒

關於資料錄製，有一個值得注意的經驗：相較於錄製資料再離線處理，線上即時建圖有一個優勢——RTAB-Map 在追蹤失敗時，能藉由把相機移回上次成功追蹤的位置來恢復里程計。

這個「人在迴路中」的即時修正能力，是離線處理錄好的資料所沒有的。因此教學上建議：先以線上方式建立成功經驗與手感，理解「相機要怎麼移動系統才不會跟丟」；等到需要做演算法比較時，再錄製資料以確保公平性（第 28.4 節 rosbag 的價值）。

掃描的手感

視覺 SLAM 有一個光達 SLAM 沒有的特點：操作者的手法會顯著影響結果。幾個實用建議：

- 轉彎時放慢：直線移動時特徵變化平緩，轉彎時視野快速改變，最容易跟丟。
- 保持特徵在視野中：不要突然把相機轉向空白牆面，而應沿著有紋理的方向移動。
- 避免只旋轉不平移：純旋轉無法三角測量（第 21.5 節、案例 21-1），單目模式尤其致命。
- 回頭確認：繞行時偶爾回頭看一下走過的區域，有助於觸發閉環並鞏固地圖。

30.7 視覺 SLAM 與光達 SLAM 比較

這是本書醞釀已久的一節：第 8、9 章比較過感測器特性，第 29 章實作了光達 SLAM，本章實作了視覺 SLAM——現在讓兩者在同一場域正面對決。

實測比較的設計

公平比較需要控制變因：

- 同一場域、同一條路線：最好用同一台機器人同時搭載光達與相機，一次移動同時錄下兩種資料。
- 相同的評估指標：軌跡誤差（ATE）、地圖品質（第 13.7 節的 2D 指標與第 17 章的牆厚度）、以及系統穩健度（跟丟次數）。
- 多種環境：這是關鍵——單一環境的比較會誤導，必須涵蓋各自的優勢與劣勢場景。

四種環境的預期表現

環境	光達 SLAM	視覺 SLAM	原因
明亮有紋理的教室	良好	良好	兩者都在舒適區
昏暗的機房	良好	失效	光達自帶光源、相機需環境光
大面積白牆走廊	尚可	失效	光達看幾何、相機找不到特徵
玻璃帷幕大廳	失效	尚可	雷射穿透玻璃、相機看得到反光與框

表 30-3　兩種 SLAM 在四種環境的預期表現

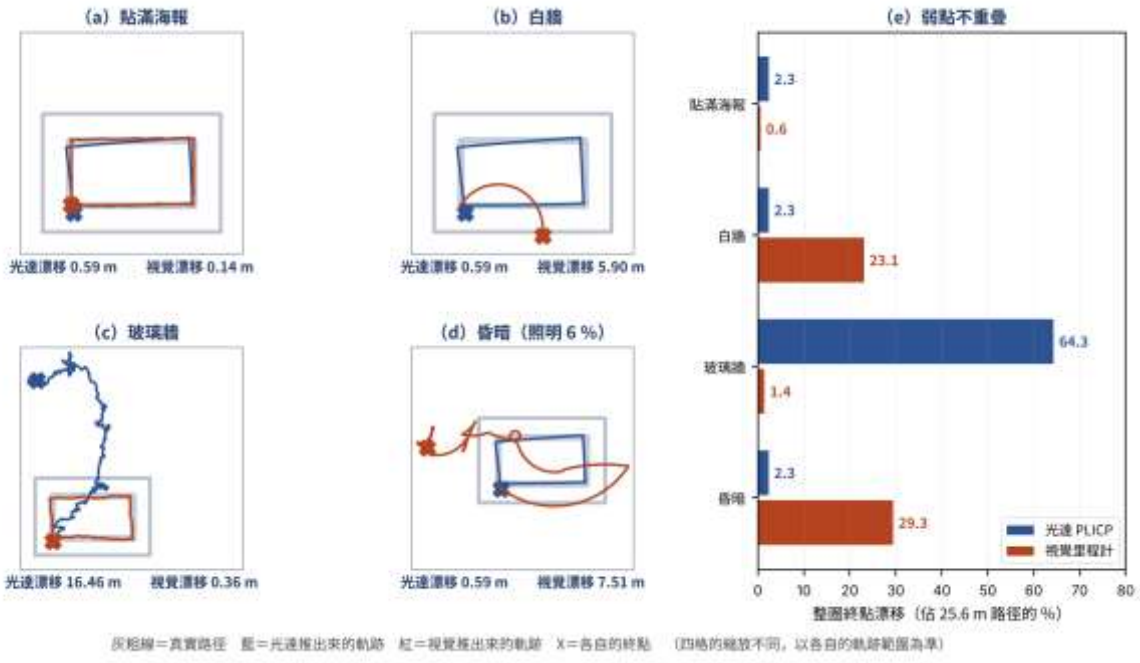
請特別注意最後一列——這是唯一視覺勝過光達的情境，也最能說明「互補」的真義：兩者的弱點不重疊，因此組合起來能涵蓋更多場景。

上面那張表是「預期表現」。本節把它真的跑一次。

實驗設計刻意做得很乾淨：四間一模一樣的 12 × 8 公尺房間，機器人沿同一條貼牆的矩形路徑走一圈（全長 25.6 公尺），每一步同時取得光達掃描與相機觀測。四間房的差別只有

牆面性質：貼滿海報（紋理 1.00、照明 1.00）、白牆（紋理 0.04）、玻璃（85 % 的雷射光束直接穿透，未穿透者有一半發生鏡射、回報錯誤距離）、以及昏暗（紋理 1.00，但照明只有 6 %）。光達走點對線 ICP 做掃描匹配，視覺則由兩幀共同看到的地標方位角與距離解相對位姿。

圖 30-4　同一場域的對決：四種環境、兩套感測器、同一條路徑



先看光達。四間房裡有三間的終點漂移完全相同，都是 0.59 公尺。這不是三次獨立實驗恰好吻合——而是因為光達的模擬根本沒有讀取紋理與照明這兩個參數，三次執行在位元層級是同一回事。把這件事講白了其實正是重點：所謂「光達在昏暗處表現良好」，精確的說法不是它「克服」了黑暗，而是黑暗對它根本不存在。雷射自己帶光源，關燈與否和它完全無關。

光達唯一垮掉的是玻璃房：漂移 16.46 公尺，佔路徑全長的 64.3 %，終點已經飛到房間外面去了。圖（c）裡那條藍線在半途整個斷開往上衝，就是掃描匹配失去參考之後的典型樣子。每幀可用的光達點從 306 個掉到 38 個，少了八成七——剩下的那些還混著鏡射回來的假距離。這一格非常值得在課堂上停下來看：它說明光達的失效不是「精度下降」，而是「突然什麼都不知道」。

再看視覺。白牆房漂移 5.90 公尺、昏暗房 7.51 公尺，而且 342 步裡分別有 340 步與 316 步判定跟丟——也就是幾乎整趟都在用等速模型硬推。兩間房的平均可用觀測分別只有 1.3 個與 2.6 個地標，連解一個三自由度位姿的最低需求都達不到。注意白牆與昏暗這兩種情況在相機眼中其實是同一件事：對比度等於紋理乘上照明，紋理 0.04 配滿照明，和紋理 1.00 配 6 % 照明，算出來的對比度分別是 0.04 與 0.06，難怪結果如此接近。這個觀察在實務上很有用：白牆上貼海報和把燈打亮，是同一種補救措施的兩種形式。

玻璃房則反過來，視覺只漂移 0.36 公尺，比光達好了 45 倍。原因在模擬裡也寫得很明白：玻璃對雷射是透明的，對相機卻不是——窗框、汙漬、反光、貼紙都是可以追蹤的特徵。這一格是整本書裡唯一一次視覺大幅勝過光達，也是「互補」這兩個字最具體的證據。

每步最準的那一個，整圈不一定最準

貼滿海報那一間房有個反直覺的結果值得留意：光達的每步相對位姿 RMSE 是 0.42 公分，視覺是 1.86 公分，光達準了 4.4 倍；但整圈下來光達漂移 0.59 公尺，視覺只有 0.14 公尺，反而是視覺贏。

原因是兩者的誤差性質不同。掃描匹配在每一步都往同一個方向偏一點點（因為相鄰兩幀的光束打在不同的表面點上），這種系統性偏差會一路累積；視覺的誤差比較大但比較隨機，走一圈之後正負相抵掉不少。

這件事對專題評分很重要：用「每幀誤差」來評斷一套里程計是不夠的，一定要跑閉環、看回到起點差多少。但閉環落差也有它的盲區——里程計的尺度偏差會讓整張地圖等比例放大或縮小，而迴路照樣完美閉合，所以閉環落差必須與捲尺量到的尺寸誤差一起看（第 31.8 節有這件事的實測）。第 31 章的評分標準就是照這兩項互補的原則訂的。

其他維度的比較

比較維度	光達 SLAM	視覺 SLAM
成本	較高	較低
地圖資訊	精確幾何，無語意	含顏色與語意線索
運算需求	2D 較低、3D 較高	較高（影像處理）
深度精度	公分級且穩定	隨距離平方衰減
對操作手法	較不敏感	敏感（30.6 節的手感）
後續應用	導航、避障	語意理解、AR、三維重建

表 30-4　兩種 SLAM 的其他維度比較

融合：最務實的答案

比較的結論不是「哪個比較好」，而是「如何組合」。30.4 節提過 RTAB-Map 支援同時使用視覺與雷射資料，這正是融合的實作途徑。

- 光達提供精確幾何與穩定的里程計，在昏暗與弱紋理處撐住系統。
- 相機提供顏色、語意與閉環辨識能力，在玻璃環境與需要理解場景時發揮作用。
- IMU（第 7、23 章）在兩者都短暫失效時提供支撐。

這正是本書從第二篇就反覆強調的核心信念：沒有全能的感測器，只有恰當的組合。走過二十九章，同學應該能親手驗證這句話——而不只是接受它。

不過這場對決還沒打完——剛才那場比賽的規則，其實暗中偏袒了其中一方。下面兩節先把規則改公平，再談這個領域接下來要往哪裡走。

把禮物收回去：如果特徵會配錯呢

上面那場對決對視覺其實是有優待的——程式裡用地標編號直接配對，等於假設特徵匹配永遠不會出錯。真實的 ORB 描述子匹配在重複紋理、視角變化大或動態物體出現時，一成到兩成的錯配是家常便飯。把這份禮物收回去，結果會變成下面這樣。

表 30-5　把錯配率加進去之後，視覺里程計的整圈漂移（公尺）

環境	錯配 0 %	錯配 10 %	錯配 20 %	錯配 35 %	同一間房的光達
貼滿海報	0.14	0.52	1.22	2.50	0.59
白牆	5.90	5.90	5.90	5.90	0.59
玻璃牆	0.36	1.10	3.10	5.50	16.46
昏暗	7.51	7.51	7.98	17.32	0.59

第一列是最該注意的一列。在零錯配的理想世界裡，貼滿海報的房間是視覺大勝（0.14 對 0.59）；但只要加進 10 % 的錯配，視覺就掉到 0.52 公尺，和光達的 0.59 公尺幾乎打平；錯配到 20 % 就反過來輸了。換句話說，前面那句「有紋理的環境視覺比較準」成立的前提是特徵匹配要夠乾淨，而這正是 ORB-SLAM 花那麼多力氣在 RANSAC、詞袋驗證與共視圖一致性檢查上的理由（第 22 章）。

第二、四列幾乎不隨錯配率改變，原因也很實際：白牆與昏暗的房間裡平均只有一兩個可用地標，連「配錯」的機會都沒有——系統早就跟丟了。錯配率只有在特徵夠多的時候才是個問題；特徵不夠的時候，問題在更前面。

第三列則說明了一個令人安心的事實：即使錯配高達 20 %，玻璃房裡的視覺（3.10 公尺）仍然遠勝光達（16.46 公尺）。互補性不是建立在視覺很完美的假設上，而是建立在兩者的失效原因毫不相干這件事上。

30.8　2024–2026：視覺 SLAM 正在被重寫

本章教的 RTAB-Map 與 ORB-SLAM3 是目前產線與課堂上最可靠的選擇，這一點近期內不會改變。但同一時間，視覺 SLAM 的研究主線已經搬到了兩個新的地方，而且兩者都快到了會影響選型的程度。這一節把幾個具體的數字放在一起，讓同學知道自己正站在什麼位置。

一、地圖的長相變了：從點雲到 3D 高斯

傳統的稠密視覺 SLAM 把地圖存成點雲或體素。2024 年起，3D Gaussian Splatting 開始取代它們：地圖變成數百萬顆帶顏色與不透明度的橢球，可以用光柵化即時算繪出照片級的畫面。

SplaTAM 是第一個把這件事做進 SLAM 迴圈的代表作。在 Replica 資料集上，它的相機軌跡誤差（ATE RMSE）平均 0.36 公分，勝過 Point-SLAM 的 0.52 公分、ESLAM 的 0.63 公分與 NICE-SLAM 的 1.06 公分，算繪速度在 876×584 解析度下可達每秒 400 幀。但在真實感測器拍攝的 TUM-RGBD 上，它的 5.48 公分卻輸給 ORB-SLAM2 的 1.98 公分——論文自己也指出，在這個基準上以特徵為基礎的方法依然勝過稠密方法。這個對比很能說明本章選擇教 ORB-SLAM3 的理由：論文榜上的第一名，換個資料集就不是第一名了。

速度方面的進展更快。2025 年底發表的 Gaussian-plus-SDF SLAM 在 Azure Kinect 的真實序列上跑到每秒 150 幀以上，而同期的高斯式 SLAM 多半還在每秒 20 幀以下；它用的高斯數量少了約一半，最佳化迭代次數少了約 75 %。

二、不校正也能跑：視覺基礎模型進入 SLAM

本章 30.2 節花了不少篇幅談相機校正，因為內參錯了後面全錯。2025 年 CVPR 的 MASt3R-SLAM 直接挑戰了這個前提：它用一個預訓練的三維重建基礎模型取代傳統的特徵匹配與三角化，在「不知道相機內參」的情況下也能做稠密 SLAM。

數字是這樣的：在 RTX 4090 上約每秒 15 幀。已知內參時，TUM-RGBD 的 ATE RMSE 平均 0.030 公尺、7-Scenes 0.047 公尺，都略勝 DROID-SLAM 的 0.038 與 0.049；完全不知道內參時分別退到 0.060 與 0.066 公尺，仍與許多需要校正的方法相當。代價也很清楚：換到 RTX 3080 Ti 就掉到每秒 5 到 7 幀，顯示記憶體要 9 到 11 GB，而且 EuRoC 這種快速運動的序列上（0.041）仍輸給 DROID-SLAM（0.022）。

同一條路線上的另一半是單目深度估測。Depth Anything V2 與 2025 年的 Video-Depth-

Anything 讓一般的 RGB 相機也能輸出具備時間一致性的深度。以後者的公制版本為例，大型模型在 NYUv2 上的 δ1 達 0.908、KITTI 上 0.910；最小的模型只有 2,840 萬個參數，在 A100 上半精度推論延遲 7.5 毫秒。不過要留意它的即時串流模式仍有明顯代價：ScanNet 上的 δ1 從離線的 0.926 掉到 0.836。

這對同學的專題意味著什麼

不要急著把 RTAB-Map 換掉。上面這些方法共同的特徵是：論文裡的數字很漂亮，但都需要一張高階顯示卡，而且多半還沒有成熟的 ROS 2 封裝與長期維護承諾。

可以現在就用上的是單目深度估測：它能當作深度相機失效時的備援（例如白牆或超出有效範圍的區域），也能讓沒有深度相機的隊伍先把流程跑起來。

要寫進專題報告的是比較：用本章的方法在自己的場域量一次 ATE，再把它和論文上的數字並排，然後解釋為什麼不一樣。這件事的學習價值遠高於把最新的模型跑起來。

三、硬體這一端也在變

30.1 節的選型建議建立在一個假設上：RealSense 便宜、ROS 2 驅動成熟、社群資源豐富。這個假設在 2025 年 7 月有了變化——RealSense 自 Intel 獨立成為新公司，取得 5,000 萬美元投資（來自 Intel Capital 與聯發科創新基金），Intel 保留少數股權與一席董事。

對教學現場而言，這件事短期是好消息：新公司有 130 名員工、三千多家客戶，並且明確把自主移動機器人的視覺 SLAM 列為投資方向，新推出的 D555 支援乙太網路供電，並內建一顆 5 TOPS 的自研 AI 晶片，走的正是第 25 章邊緣運算那條路。但長期的維護承諾與供貨穩定度還需要時間驗證，因此在規劃多年期的課程或專題設備時，建議不要把整個流程綁死在單一廠牌的 SDK 上——盡量讓節點只依賴標準的 ROS 2 影像與 CameraInfo 介面，這樣換相機時改的是驅動，不是整套系統。

公式或流程

一、由深度影像反投影為三維點

這是第 9.1 節針孔投影模型的逆運算。設某像素座標為（u, v）、該像素的深度值為 d、相機內參為 fx、fy、cx、cy，則對應的三維點座標為：

$$Z = d \,/\, scale$$

$$X = (u - cx) \times Z \,/\, fx$$

$$Y = (v - cy) \times Z \,/\, fy$$

其中 scale 是深度縮放因子——深度影像常以毫米為單位的整數儲存，而點雲以公尺為單位，因此需要除以 1000。這個縮放因子填錯是初學者常見的錯誤，症狀是「點雲的尺度完全不對」（例如整個房間變成一公釐大）。

把這個運算套用到深度影像的每個像素，再賦予對應的 RGB 顏色，就得到彩色點雲（30.3 節）。

二、深度品質與有效範圍

第 9.6 節推導過雙目深度誤差隨距離平方成長：

$$\Delta Z \approx (Z^2 \,/\, (f \cdot b)) \times \Delta d$$

這解釋了 30.1 節提到的實務做法——雖然相機標稱十公尺量測範圍，但常把有效範圍設為五公尺。以此式估算：距離加倍，誤差變四倍。若在 2 公尺處誤差 1 公分，到 10 公尺處誤差就可能達 25 公分，已不足以支撐精確建圖。

實務建議：依你的精度需求，用這條式子反推可用的最大距離，並在系統中設定為深度裁切門檻。

三、視覺 SLAM 實作的完整流程

圖 30-5　RGB-D 視覺 SLAM 的完整實作流程

【階段一：相機準備】

① 安裝相機驅動 → ② 確認內參（出廠或自行校正，30.2 節）

③ 開啟深度對齊 → ④ 驗證彩色與深度影像的像素對應正確

↓

【階段二：資料驗證】

⑤ 檢查三個必要話題：彩色影像、對齊深度、相機資訊

⑥ 在 RViz 中顯示點雲，確認幾何與顏色都正確

↓

【階段三：SLAM 執行】

⑦ 啟動 RTAB-Map 或 ORB-SLAM3，注意同步方式設定

⑧ 緩慢掃描，觀察軌跡、地圖與閉環

↓

【階段四：評估與比較】

⑨ 儲存地圖 → ⑩ 量測誤差（尺寸比對、牆厚度、ATE）

⑪ 與光達 SLAM 在同場域比較（30.7 節）

四、視覺 SLAM 的故障排除順序

與第 29.7 節的導航除錯相同，視覺 SLAM 也應依系統性順序排查：

圖 30-6 視覺 SLAM 的除錯檢查表

① 相機驅動：三個必要話題都有資料嗎？頻率正常嗎？

↓

② 內參：CameraInfo 的數值合理嗎？（fx、fy 應相近；cx、cy 約為影像寬高的一半）

↓

③ 深度對齊：彩色與深度是否對齊？點雲的顏色有沒有錯位？

↓

④ 深度縮放：點雲的尺度正確嗎？量一個已知物體驗證（公式一的 scale）

↓

⑤ 同步：若系統完全不動，檢查是否需要啟用近似同步

↓

⑥ 特徵：當前畫面偵測到多少特徵點？太少表示環境紋理不足

↓

⑦ 運動：移動是否過快？是否只旋轉未平移？（30.6 節的手感）

↓

⑧ 參數：確認上游都健康後，才開始調 SLAM 參數

案例

案例 30-1 動得快一點就跟丟：視覺 SLAM 最常見的挫折

這是同學實作視覺 SLAM 時幾乎必定遇到的狀況，值得完整說明。

典型情境是這樣的：同學拿著深度相機在實驗室慢慢移動，系統運作良好，點雲逐步建

立。但只要稍微加快移動或轉動速度，畫面上的座標軸就開始亂飄，點雲停止記錄，終端顯示追蹤品質為零。

這個現象在社群中被反覆討論。有使用者描述：若非常緩慢地移動或旋轉相機，地圖能建立但尺寸不對；若稍微加快一點，相機的 TF 就開始隨機亂飄，點雲停止記錄。官方文件也明確警告：移動或轉動太快會中斷成功的點雲匹配序列，導致系統跟丟。

原因是第 26.4 節討論過的三重打擊在此重現：快速運動造成動態模糊使特徵提取失敗、特徵軌跡縮短使匹配困難、而深度相機的幀率限制也使前後幀差異過大，超出配準的收斂範圍（第 17.3 節 ICP 需要好初值）。

對策有三個層次：

- 操作層面：放慢速度，尤其轉彎時（30.6 節的手感建議）。這是最直接有效的一招。
- 設定層面：社群提出的一個解法是分別使用相機的兩個紅外線影像，以紅外線組態而非深度組態執行 SLAM——這相當於改用雙目模式，能提供更穩定的特徵追蹤。
- 系統層面：加入 IMU 融合（第 23 章）。IMU 能在視覺短暫失效時提供運動預測，大幅提升快速運動下的穩健度——這正是第 7.6 節「IMU 填補視覺失效空隙」的價值。

這個案例的教學價值在於：它讓同學親身體會「論文上的精度數字，是在特定條件下取得的」。同一套系統，慢速掃描能達公分級精度，快速移動則直接失效——理解這個落差，是負責任地使用技術的前提。

案例 30-2　三公尺長的房間變成三公釐：深度縮放的陷阱

這是一個看似荒謬、卻極為常見的錯誤，幾乎每個第一次自己寫點雲產生程式的同學都會遇到。

症狀是：點雲產生出來了，形狀看起來也對，但整個場景小得離譜——在 RViz 中要把

視角拉到極近才看得見，而量測的尺寸完全不合理。

原因就是公式一中的深度縮放因子。深度影像為了節省空間，通常以 16 位元整數儲存，單位是毫米；而 ROS 的點雲以公尺為單位。若程式中忘了除以 1000，整個場景就會縮小一千倍——三公尺長的房間變成三公釐。

這個錯誤的診斷方法極為簡單：量測點雲中一個已知尺寸的物體（例如一張 A4 紙或桌子的寬度），若數值差了三個數量級，就是縮放因子的問題。

更廣泛的啟示是：單位錯誤是工程中最常見也最容易避免的錯誤之一。第 10.3 節提過弧度與角度的混用、本章的毫米與公尺、第 5 章的經緯度與公尺——養成「每個數值都標註單位」的習慣，能省下無數除錯時間。這個習慣在寫程式、寫報告、看規格書時都同樣受用。

案例 30-3　同一場域的對決：當光達與相機各自失效

第 30.7 節提出的對照實驗，在大學課堂上有極高的教學價值。這裡描述一次典型的實測結果。

實驗設計：同一台小車同時搭載 2D 光達與 RGB-D 相機，沿相同路線走過四個區域——明亮的教室、昏暗的機房、貼滿白色隔音板的走廊、以及玻璃帷幕的中庭。

結果相當戲劇性：

- 教室：兩者都表現良好，地圖清晰、軌跡準確。同學此時可能會覺得「兩者差不多」。
- 昏暗機房：視覺 SLAM 在進入後數秒內跟丟，特徵數歸零；光達則完全不受影響，因為它自帶雷射光源（第 8 章）。
- 白牆走廊：視覺 SLAM 再次失效（白牆詛咒）；光達雖然也面臨走廊的結構退化問題（第 13.6 節），但至少能持續運作。

- 玻璃中庭：局勢逆轉——光達的雷射穿透玻璃，在地圖上畫出根本不存在的空間；相機反而能看到玻璃的反光、窗框與框外的景物，建出較合理的地圖。

這個實驗把第 8、9 章的理論從「表格上的比較」變成「親眼看到的失效」。同學普遍反映，做完這個實驗後，對感測器選型的理解遠比讀十遍表格深刻。

更重要的是後續討論：既然兩者的弱點不重疊，那麼組合起來會如何？這自然引導到 RTAB-Map 同時使用視覺與雷射的融合實作，也呼應第 27 章多感測器與多機器人協作的思想。建議教師務必安排這個實驗，它是第八篇最有說服力的一堂課。

實作活動

實作 30-1　視覺 SLAM 實作與跨感測器比較

目的：完成 RGB-D 視覺 SLAM 的完整實作，並在同一場域與第 29 章的光達 SLAM 進行對照比較。這是第八篇的高潮實驗，也是第 31 章專題的重要基礎。

第一部分：相機設定與點雲產生（全體）

25. 安裝深度相機驅動，確認彩色影像、深度影像與相機資訊三個話題都有資料，並記錄各自的發布頻率。
26. 檢視 CameraInfo 中的內參數值，依圖 30-3 的第二步驗證合理性（fx 與 fy 是否相近？cx、cy 是否約為影像寬高的一半？）。
27. 開啟深度對齊，在 RViz 中顯示彩色點雲，檢查顏色與幾何是否正確對應。刻意關閉對齊再看一次，觀察錯位現象。
28. 尺度驗證：在點雲中量測一個已知尺寸的物體（A4 紙、桌面寬度），確認數值正確——親身避開案例 30-2 的陷阱。
29. （進階）自己寫程式：依公式一，從深度影像與內參產生點雲，不使用現成節點。故意把 scale 設錯，重現案例 30-2 的現象。

第二部分：RTAB-Map 建圖（全體）

30. 啟動 RTAB-Map，指定正確的話題名稱。首次執行加上刪除舊資料庫的參數。
31. 在教室中緩慢移動相機掃描，觀察即時建立的彩色點雲與相機軌跡。
32. 閉環測試：繞行一圈回到起點，觀察閉環偵測是否觸發、地圖是否被拉正。
33. 速度實驗：逐步加快移動速度，找出系統開始跟丟的臨界速度，重現案例 30-1。記錄這個速度值。
34. 儲存地圖，並切換到定位模式重新啟動，驗證系統能在既有地圖中定位（對照第 29.5 節的 AMCL）。

第三部分：失效情境探索（全體，本章特色）

依 30.6 節的五種情境，逐一測試並記錄系統行為：

35. 白牆測試：把相機對準大面積白牆，記錄特徵點數量的變化，以及系統跟丟前能撐多久。
36. 光線測試：從明亮處走入昏暗處，記錄追蹤中斷的時機；再對著窗戶造成過曝，觀察結果。
37. 玻璃測試：掃描玻璃窗或鏡子，觀察深度資料出現什麼異常（缺失？錯誤的深度值？）。
38. 動態物體測試：讓一位同學在畫面中走動，觀察是否影響定位精度與地圖品質。
39. 純旋轉測試：原地旋轉相機（不平移），觀察系統的反應，驗證第 21.5 節的三角測量限制。

第四部分：跨感測器比較（核心實驗，大學部）

3. 場域選擇：選出四個特性不同的區域——明亮有紋理處、昏暗處、大面積白牆處、玻璃處（參考表 30-3）。

4. 資料採集：以同一台小車同時搭載光達與相機，沿相同路線移動，同時錄製兩種資料（第 28.4 節的 rosbag）。
5. 分別執行：用同一份資料，分別執行第 29 章的光達 SLAM 與本章的視覺 SLAM。
6. 量化比較：記錄四個區域中各自的跟丟次數、地圖品質、以及（若有真值）軌跡誤差，填入你自己的比較表。
7. 融合嘗試：設定 RTAB-Map 同時使用視覺與雷射資料（30.4 節），觀察是否能涵蓋兩者各自失效的區域。

討論問題

12. 關閉深度對齊後，點雲的顏色如何錯位？錯位的方向與相機的物理結構有什麼關係？
13. 你的系統在多快的移動速度下開始跟丟？這個速度對「裝在移動機器人上」意味著什麼？
14. 五種失效情境中，哪一種最容易發生？哪一種後果最嚴重？為什麼？
15. （核心問題）在四個區域的比較中，光達與視覺各在哪些區域勝出？這是否符合表 30-3 的預期？有沒有出乎意料的結果？
16. 融合視覺與雷射後，系統的表現如何？它是否真的涵蓋了兩者各自的弱點？融合帶來什麼新的代價（運算、設定複雜度）？

延伸挑戰（大學部）

其一：大學場域三維數位分身——用 RTAB-Map 掃描一間實驗室或一段走廊，產生完整的彩色三維模型，並依第 17.5 節的方法轉為八叉樹或網格格式輸出。這份成果可用於系所展示、空間管理，也是第 31 章專題二的直接產物。其二：視覺與光達融合的量化研究——設計一套嚴謹的實驗，在多種環境中比較「純光達」「純視覺」「兩者融合」三種配置的表現，以 ATE 與跟丟次數量化，並撰寫成一份完整的研究報告。這個題目層次完整、成果具體，是極佳的專題主軸，也與第八篇強調的「用資料說話」相呼應。

延伸閱讀

15. M. Labbé and F. Michaud, "RTAB-Map as an Open-Source Lidar and Visual SLAM Library for Large-Scale and Long-Term Online Operation," Journal of Field Robotics, 2019.（RTAB-Map 的權威論文，含視覺與雷射融合的說明）
16. RTAB-Map 官方網站與 ROS 2 套件文件（introlab.github.io/rtabmap）.（參數說明、教學與各種相機的設定範例）
17. C. Campos et al., "ORB-SLAM3," IEEE T-RO, 2021.（第 23 章的系統，本章的實作對象之一）
18. Intel RealSense ROS 2 套件文件與 SLAM 教學 Wiki(GitHub: IntelRealSense/realsense-ros).（相機設定、深度對齊與 IMU 融合的官方指引）
19. ROS 2 camera_calibration 套件文件.（相機校正的互動式工具，30.2 節的實作基礎）
20. L. Keselman et al., "Intel RealSense Stereoscopic Depth Cameras," CVPR Workshops, 2017.（深度相機的原理與規格，第 9、30 章共同參考）
21. evo 軌跡評測工具（GitHub: MichaelGrupp/evo）.（ATE 與 RPE 計算，跨感測器比較的量化工具）

小結

15. 三類深度相機各有定位：RealSense 採主動紅外雙目、室內表現佳且部分型號內建 IMU;ZED 採被動雙目、戶外可用且精度較佳但需 GPU;OAK-D 內建 AI 晶片可在相機端推論。選型應考量環境、運算資源、是否需 IMU 與有效範圍。
16. 現代深度相機出廠已校正並透過 CameraInfo 提供參數，但在碰撞後、使用自組系統或高精度需求時應自行校正；校正的常見陷阱包含棋盤格不平整、影像涵蓋不足、解析度不一致與忘記更新參數。

17. 由深度影像反投影產生點雲是針孔模型的逆運算，關鍵前提是深度對齊——彩色與深度感測器物理位置不同，未對齊會造成顏色與幾何錯位；深度縮放因子填錯會使場景尺度差三個數量級。

18. RTAB-Map 是基於增量式外觀閉環偵測的 RGB-D 圖優化 SLAM，產生稠密彩色點雲與 RGB-D 里程計；它能使用外部里程計、融合 2D 或 3D 雷射掃描、並支援八叉樹與體素等結構，是視覺與光達融合的實作途徑。

19. RTAB-Map 與 ORB-SLAM3 定位不同：前者輸出稠密彩色地圖適合三維重建與導航，後者輸出稀疏特徵地圖適合高精度定位與軌跡評測；三種模式（單目、RGB-D、雙目慣性）的難易與精度差異可實測驗證。

20. 視覺 SLAM 的五種典型失效為白牆弱紋理、快速運動、光線變化、玻璃鏡面與動態物體；移動或轉動太快會中斷點雲匹配序列導致跟丟，而線上建圖相較離線處理的優勢是能靠人為把相機移回上次成功位置來恢復追蹤。

21. 光達與視覺的弱點不重疊：昏暗與白牆處光達勝出、玻璃處視覺勝出，這正是互補的真義；融合（如 RTAB-Map 同時使用視覺與雷射）是最務實的答案，印證了本書「沒有全能的感測器，只有恰當的組合」的核心信念。

22. 評估一套里程計或 SLAM，單一指標不夠：每幀誤差看不出累積漂移，而閉環落差看不出里程計的尺度偏差——整張地圖等比例放大或縮小時，迴路照樣完美閉合。閉環落差要與捲尺量到的尺寸誤差一起看，兩者的盲區才不會重疊（第 31.8 節有這件事的實測數據）。

第八篇的技術實作到這裡告一段落。同學已經親手建立了兩套完整的 SLAM 系統，並且在同一個場域裡量出了它們各自在哪裡失效、失效時是什麼樣子、以及為什麼那些弱點不會同時出現。這比讀任何一張比較表格都更深刻——因為當你親眼看過光達在玻璃房裡把軌跡甩出房間外、也看過相機在白牆前整趟都在跟丟，「沒有全能的感測器，只有恰當的組合」這句話就不再是一句口號。

下一章，我們把所有能力整合成完整的專題：從高中的教室 2D 建圖、大學部的三維空間掃描、掃地機器人路徑分析、機器狗校園巡檢、視覺導航無人機，到多感測器融合定位，並提出涵蓋地圖品質、定位誤差、系統穩定性、報告表達與程式碼品質的完整評分標準。第 31 章將是本書實作的總驗收。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 31 章

SLAM 專題與評量

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程　/　高中 AI 機器人社團課程

本章在全書的位置

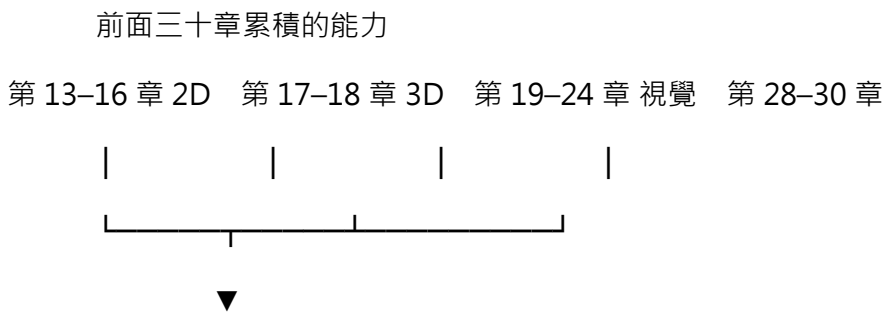
這是第八篇的最後一章，也是全書實作部分的總驗收。

前面三十章，我們一路從「SLAM 是什麼」走到「我親手把它跑起來了」：第一篇到第三篇建立觀念、感測器與數學基礎，第四篇到第七篇依序處理 2D 建圖、3D 光達、視覺 SLAM 與微型平台，第 28 章備妥開發環境，第 29 章讓光達小車真的動起來，第 30 章把相機與光達放在同一個場域正面比了一次。

本章要處理的是一個不一樣的問題：這些能力要怎麼組成一件「作品」，而這件作品要怎麼被評斷好壞。這兩件事看似是課程行政，其實是工程訓練裡最難教、也最容易被跳過的部分——多數同學的專題不是敗在技術，而是敗在題目太大、時程沒有緩衝、以及最後交出一份沒有數據的報告。

所以本章有三張用實驗算出來的圖，分別回答三個常被含糊帶過的問題：圖 31-1 回答「不同策略的效率差多少」，用的是專題三的覆蓋規劃；圖 31-2 回答「時程該留多少緩衝」，用的是四個里程碑的蒙地卡羅模擬；圖 31-3 回答「你用的評估方法本身可不可靠」——這一張的結論可能會讓不少同學重新檢視自己的評估方式。

圖 31-0　第 31 章的連結地圖



學習目標

走過三十章的理論與實作，同學已經具備完整的 SLAM 能力。本章是本書實作的總驗收：六個難度遞增的專題，一套五向度的評分標準，以及三組量化實驗，用來說明「怎麼做」與「怎麼被評」背後的理由。它既是課程的結業設計，也是同學規劃自己研究方向的地圖。學完本章之後，同學應能夠達成下列目標：

1. 依自身的能力與資源，選擇合適難度的專題並規劃執行流程。
2. 完成教室 2D 地圖建構專題，並量化評估地圖品質。
3. 完成三維空間掃描專題，產出可展示的數位模型。
4. 設計並執行掃地機器人路徑分析與覆蓋率評估，並以實驗說明不同策略的效率差距。
5. 規劃機器狗校園巡檢與視覺導航無人機的系統架構。
6. 設計多感測器融合定位的實驗與比較方法。
7. 依五向度評分標準自我檢核，並準備專題報告與展示。

8. 判斷一個評估方法是否足以支撐結論，並說明閉環落差、尺寸誤差與 ATE 各自能看出什麼、看不出什麼。

9. 以機率的觀點規劃里程碑與緩衝，並估算專題準時完成的機會。

核心概念

中文名詞	英文名詞	概念說明
專題導向學習	Project-Based Learning	以完成真實任務為核心的學習方式。
評分標準	Rubric	以明確向度與等級描述評分依據的工具。
絕對軌跡誤差	ATE	估計軌跡與真值對齊後的位置誤差（第 22.7 節）。
相對位姿誤差	RPE	固定間隔內相對運動的誤差，反映局部漂移。
閉環落差	Loop Closure Gap	繞一圈回到原點時，地圖上起終點的距離。
覆蓋率	Coverage Rate	實際走過的面積佔可行區域的比例（第 16 章）。
重複率	Redundancy Rate	實際行走距離與理論最短距離的比值。
地圖品質指標	Map Quality Metric	牆面厚度、尺寸誤差、閉環一致性等評估項目。
系統穩定性	Robustness	長時間運行不跟丟、能從失敗恢復的能力。
可重現性	Reproducibility	他人能依你的紀錄重現相同結果的程度。
成果證據	Evidence	可驗證的產出：地圖檔、軌跡圖、錄影、程式碼。
技術債	Technical Debt	為求快速而累積的、日後需償還的程式品質

		問題。
專題里程碑	Milestone	把長期專題切分成可檢核的階段性目標。
緩衝	Schedule Buffer	為吸收工期不確定性而刻意預留的時間。

表 31-1　本章的核心名詞

專題總覽：如何選擇

本章提供六個專題，難度由淺入深。它們不是彼此獨立的選項，而是一條可以循序漸進的路徑——後面的專題往往以前面的成果為基礎。

專題	主題	難度	適合對象	主要依據章節
一	教室 2D 地圖建構	★	高中、大學部入門	第 13–15、28 章
二	三維空間掃描	★★	大學部	第 17、30 章
三	掃地機器人路徑分析	★★	高中進階、大學部	第 16 章
四	機器狗校園巡檢	★★★★	專題生	第 18、24、27 章
五	視覺導航無人機	★★★★	專題生	第 23、26 章
六	多感測器融合定位	★★★★★	專題生、研究生	第 5–7、11、29 章

表 31-2　六個專題的難度與適用對象

選擇專題的三個判準

- 資源：有哪些硬體？有多少時間？若只有電腦，專題一與三可完全以模擬完成；專題四、五需要實體平台。
- 能力：對照第 28.7 節的五階段學習路徑，誠實評估自己目前在哪一階。挑戰高一階的專題最能成長，跳兩階則容易挫折。
- 興趣與延續性：專題最好能延續成長期的研究方向。若你對 AI 有興趣，專題四、五

能自然銜接第 32 章的語意 SLAM 與第 34 章的語言導航。

共通的執行原則

- 切分里程碑：把專題拆成三到五個可檢核的階段，每階段都有可展示的成果。這能避免「做到最後才發現方向錯了」。
- 留下證據：第 29.7 節強調過，每個階段都應產出可驗證的證據——地圖檔、誤差數據、錄影、程式碼倉庫。這既是除錯的依據，也是報告的素材。
- 先模擬再實體：第 28、29 章反覆建議的路徑。在模擬中把流程與參數調通，能省下大量實體除錯時間。
- 記錄失敗：失敗的嘗試與其原因，往往比成功的結果更有價值。專題報告中若能誠實呈現「我試過什麼、為什麼不行、最後怎麼解決」，遠比只報告成功結果更有說服力。

31.1　專題一：教室 2D 地圖建構

這是最基礎也最重要的專題。它涵蓋 SLAM 的完整循環，而且成果直觀可見——適合作為所有同學的第一個專題。

專題目標

以 2D 光達建立一間教室（或一段走廊）的佔據格地圖，並量化評估其品質。進階目標是完成建圖後的自主導航。

三個階段

- 階段一——模擬建圖：在模擬環境中完成建圖流程，熟悉工具鏈與參數（第 28、29 章）。產出：模擬地圖檔與操作紀錄。
- 階段二——實體建圖：在真實教室以小車建圖。務必先完成里程計校正（第 29.3 節）。產出：地圖檔、建圖過程錄影、品質評估報告。

- 階段三——自主導航：載入地圖，以 AMCL 定位並用 Nav2 導航到指定目標點（第 28.5、29.6 節）。產出：導航錄影、成功率統計。

必做的量化評估

- 尺寸誤差：量測地圖上教室的長寬，與捲尺實測比較，計算誤差百分比。目標是小於 3%。
- 閉環一致性：繞行一圈回到起點，量測地圖上起點與終點的落差。
- 目視檢查：依第 13.7 節檢查牆是否筆直、有無重影與斷裂。
- 導航成功率（若做階段三）：設定五個目標點，各嘗試三次，統計成功抵達的比例與平均耗時。

這四項要一起做，不能只挑一項。理由在 31.8 節會以實驗說明：尺寸誤差與閉環落差各自只能看見一種錯誤，只量其中一項，有很大的機會把一張錯了 3% 的地圖判成滿分。

進階挑戰

- 演算法比較：用同一份錄製資料（rosbag），分別執行 slam_toolbox、Cartographer 與 Gmapping，比較地圖品質與運算負擔（第 15 章實作 15-1）。
- 困難場域：挑戰長走廊或空曠大廳，重現第 13.6 節的困境，並嘗試提出對策（增加特徵、調整參數、融合 IMU）。
- 參數研究：系統性地改變粒子數、解析度、更新門檻等參數，量化其對品質與效能的影響，繪製成圖表。

評量重點

這個專題的評量重點不在「有沒有建出地圖」，而在「能不能說清楚地圖為什麼好或不好」。同學應能指出誤差的來源（里程計？掃描匹配？環境結構？），並提出改善的依據。

> **常見的三個起步錯誤**

一、沒有校正里程計就直接建圖。第 29.3 節量過：輪徑誤差 2% 在繞行一圈後就足以讓地圖產生可見的扭曲，而且這種扭曲看起來像是「演算法不好」，很容易讓同學往錯的方向調參數。

二、推得太快。掃描匹配需要相鄰兩幀有足夠重疊；推車速度超過感測器更新率所能負擔的範圍時，匹配會失敗而產生跳躍。先以慢速走完一次，再逐步加速找出自己系統的上限。

三、只走一次就下結論。同一個場域至少走三次，看結果穩不穩定。三次結果差異很大，代表系統處在邊界上，這件事本身就值得寫進報告。

31.2　專題二：智慧機器人教室 3D 點雲掃描

這個專題把維度提升到三維，產出的成果視覺效果極佳，適合作為系所展示與招生素材。

專題目標

以 RGB-D 相機或 3D 光達掃描一間實驗室或一段走廊，產出完整的彩色三維點雲模型，並評估其精度。

三個階段

- 階段一——設備準備與校正：相機校正、深度對齊驗證、點雲尺度確認（第 29.2、30.3 節）。產出：校正參數檔與驗證紀錄。
- 階段二——掃描建模：以 RTAB-Map 掃描目標空間。注意規劃路線確保覆蓋完整、重疊充足（第 17.5 節）。產出：三維模型檔與掃描過程錄影。
- 階段三——後處理與輸出：降採樣、離群點剔除、轉為八叉樹或網格格式，產出可分享的模型（第 17.2、17.5 節）。

必做的量化評估

- 牆面厚度：第 17 章案例 17-1 的核心指標。量測一段牆面點雲在垂直方向的分布厚度，理想值應接近感測器精度（數公分）。
- 尺寸誤差：量測模型中的關鍵尺寸（門寬、房間長度）與實測比較。
- 覆蓋完整性：檢查是否有明顯的空洞，並說明成因（遮蔽？玻璃？掃描路線不足？）。
- 資料規模：記錄原始點數、降採樣後點數、以及各階段的處理時間。

進階挑戰

- 多次掃描合併：分次掃描不同區域再合併，體驗第 23 章 Atlas 多階段作業或第 27 章地圖融合的概念。
- 跨感測器比較：同一空間分別以 RGB-D 相機與 3D 光達掃描，比較模型的精度、密度與完整性（第 30.7 節的三維版本）。
- 應用延伸：把模型匯入模擬環境，建立教室的「數位分身」，供後續學生在虛擬環境中測試演算法（第 17.7 節）。這條路也正是第 33 章 Sim-to-Real 的入口——你掃出來的這間教室，之後可以變成訓練場。

成果的長期價值

這個專題的產出有實際用途：系所空間管理、無障礙路線規劃、招生展示、以及作為模擬環境的基礎。建議同學把成果整理成可分享的格式並附上說明文件——這比一份只有自己看得懂的檔案有價值得多。

31.3　專題三：掃地機器人路徑分析

這個專題不需要昂貴硬體，卻能深入覆蓋路徑規劃的核心議題，是高中與大學部入門的理想選擇。

專題目標

分析並改善覆蓋路徑規劃演算法，以覆蓋率與重複率量化評估，並（若可能）與真實掃地機器人的行為比較。

三個階段

- 階段一——紙上與模擬分析：依第 16.7 節的紙上活動，再以 Python 實作弓字形覆蓋規劃，計算覆蓋率與重複率。產出：程式碼與量化數據。
- 階段二——策略比較：實作並比較至少三種策略——隨機碰撞、弓字形、螺旋式，在不同房間形狀下比較效率。產出：比較圖表與分析。
- 階段三——真實觀察（選用）：觀察並記錄真實掃地機器人的路線與 App 地圖，與你的模擬結果比較，推測它使用的策略。產出：觀察紀錄與對照分析。

一、這個專題該做出什麼樣的圖：一次示範

以下是本書為這個專題實際跑出來的結果，可以作為同學階段二產出的樣板——注意它長什麼樣子：有明確的實驗設定、有對照組、有兩個互補的指標、而且結論可以用一句話講完。

實驗設定：8 公尺 × 6 公尺的房間，切成 0.25 公尺的格子；障礙以 3×3 格的家具方塊隨機擺放，佔地比例分別取 0%、6%、12%、20%；每種設定跑 20 次隨機配置。三種策略：隨機碰撞（直行，撞到才隨機轉向，另加每步 5% 的自發轉向以模擬打滑）、弓字形（逐列上下掃掠，遇障礙以最短路徑繞到下一個未掃格）、螺旋式（由外圈往內收）。橫軸一律換算成「已行走距離 ÷ 理論最短覆蓋距離」，這樣三種策略才能公平比較。

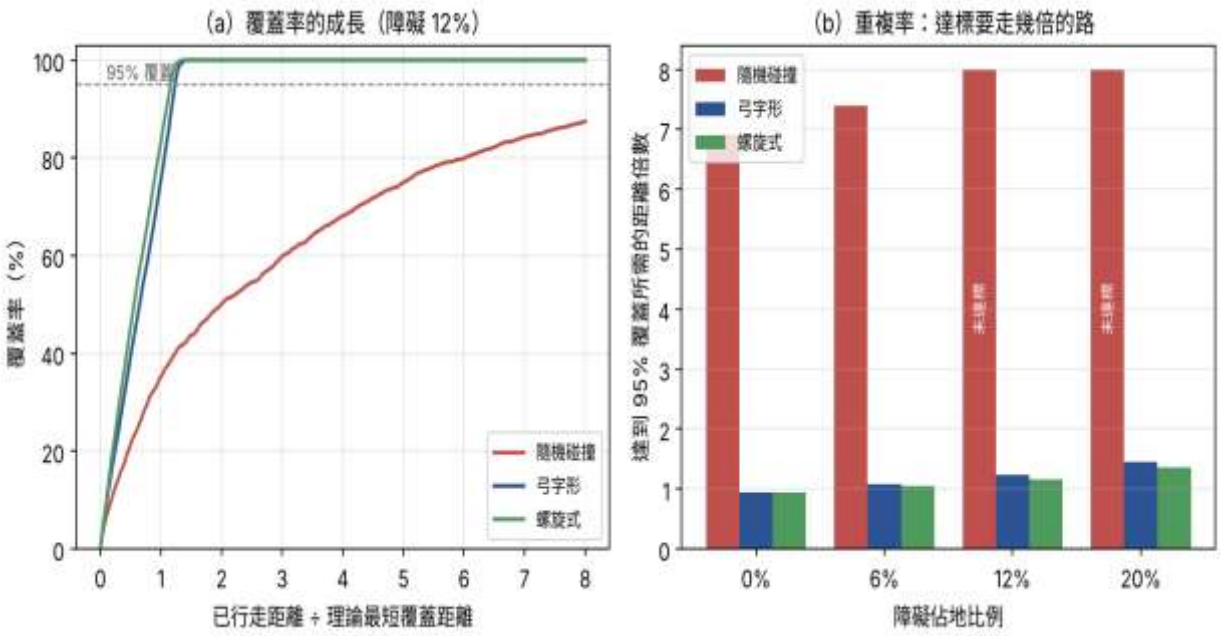


圖 31-1　三種覆蓋策略的效率差距

（a）是障礙 12% 時覆蓋率隨行走距離的成長曲線；（b）是四種障礙密度下，達到 95% 覆蓋所需的距離倍數。隨機碰撞在 12% 與 20% 的設定中，走到 8 倍距離仍未達標，故標為「未達標」。

二、三個可以直接寫進報告的結論

1. 系統性策略的效率是隨機策略的五倍以上。走完一倍的理論最短距離時，螺旋式已覆蓋 83.2%、弓字形 75.6%，隨機碰撞只有 35.3%。要達到 95% 覆蓋，弓字形在無障礙房間只需 0.95 倍距離，隨機碰撞需要 6.9 倍——而在障礙 12% 以上的房間，隨機碰撞走到 8 倍都到不了。

2. 隨機策略的真正問題不是慢，是尾巴收不起來。看（a）的紅線：它在前段其實爬得不算太差，但越接近 100% 越平——最後那幾格永遠靠運氣。這正是早期掃地機器人「掃很久卻總有一塊沒掃到」的數學原因，也是後來各家廠商全部改用建圖加規劃的理由。

3. 障礙會讓系統性策略變貴，但不會讓它失效。弓字形達標距離從 0.95 倍（無障礙）升到 1.45 倍（障礙 20%），螺旋式從 0.95 升到 1.36——多出來的部分就是繞路成本。換句話說，障礙讓重複率上升約五成，但覆蓋率仍然保證達標；隨機策略則是直

接失去保證。

第三點值得多說一句，因為它牽涉到第 16 章的理論：弓字形與螺旋式之所以「保證」達標，是因為它們走的是一條事先算好、涵蓋所有可行格的路徑；繞路只增加成本，不改變涵蓋性。隨機策略沒有這個保證，它的覆蓋率是一個機率量——這就是為什麼它的曲線會有一條永遠貼不到 100% 的漸近線。

> **為什麼螺旋式在這個實驗中略優於弓字形**
>
> 螺旋式每一圈都沿著「已掃區域的邊界」前進，在矩形房間中轉彎次數較少、繞路較短，因此達標距離略低（12% 障礙時 1.16 倍 vs 1.23 倍）。
>
> 但這個優勢與房間形狀高度相關。在狹長走廊或多房間連通的格局中，螺旋式會反覆穿越已掃區域，優勢消失甚至反轉。
>
> 這正是值得同學自己驗證的地方：換一種房間形狀重跑一次，看結論會不會翻盤。一個會隨設定翻盤的結論，只要你說清楚條件，仍然是好結論；沒說清楚條件的結論，才是錯的。

三、必做的量化評估

- 覆蓋率：實際走過的格數佔可行格數的比例，目標接近 100%。
- 重複率：總行走距離除以理論最短覆蓋距離（第 16 章公式一）。建議改用「達到 95% 覆蓋所需的距離倍數」，因為最後 5% 往往佔掉隨機策略一半以上的時間，會把平均值拉得失真。
- 轉彎次數：轉彎耗時且耗電，是實務上的重要指標。
- 對障礙的適應性：在障礙物數量與配置不同的房間中測試，觀察策略的穩健度。

四、進階挑戰

- 房間分割：實作第 16.4 節的距離變換法，把地圖自動切分成房間，再依房間逐一覆

蓋，比較與「整體覆蓋」的效率差異。

- 斷點續掃：實作中斷後記錄已完成區域、重啟後只掃剩餘部分的功能，並驗證覆蓋率仍達 100%。
- 與實體整合：把你的覆蓋規劃器接上第 29 章建好的地圖與 Nav2，讓實體小車真的執行覆蓋清掃。
- 加入定位誤差：把本節的實驗重跑一次，但讓機器人的位置帶有 SLAM 誤差（例如每公尺 2 公分的隨機偏移）。這會讓「已掃過」的紀錄本身變得不可靠——這正是真實掃地機器人最難的部分，也是本書前三十章存在的理由。

五、教學上的價值

這個專題的獨特之處是「人人都有直覺、一動手就發現不簡單」。同學可能自認「不就是一排一排掃」，但一遇到不規則房間、障礙物、多房間連通，就會發現問題的複雜度。這種從直覺到嚴謹的落差，是很好的演算法思維訓練。

31.4　專題四：機器狗校園巡檢 SLAM

這是難度較高的整合型專題，涵蓋第 18、24、27 章的技術，適合專題生投入一到兩個學期。

專題目標

以四足或輪型機器人在校園執行自主巡檢，建立三維地圖、辨識巡檢目標、並產出巡檢報告。

五個階段

- 階段一——資料採集：以手推車或機器人載具搭載光達與 IMU，在走廊、樓梯間、實驗室錄製資料集（第 18.7 節）。這一步不需演算法能力，適合起步。
- 階段二——離線建圖比較：以錄好的資料執行 LeGO-LOAM、LIO-SAM、Fast-LIO，比較軌跡精度、地圖品質與運算負擔（第 18 章）。
- 階段三——退化場景研究：專門針對長走廊，比較純光達與光達加 IMU 緊耦合的差

異，量化 IMU 的貢獻。這是最有研究價值的一段。

- 階段四——語意感知：訓練 YOLO 模型辨識巡檢目標（消防設備、門、異常），注意蒐集機器人視角的資料（第 24.4 節的低視角問題）。
- 階段五——自主巡檢與報告：整合定位、導航與辨識，設定巡檢點自動執行，產出含三維位置標註的巡檢報告。

特別提醒階段一的價值：它是整個專題中唯一「不需要任何演算法能力就能做、而且做完就一定有產出」的階段。一份錄製良好、標註清楚的校園資料集，本身就是可以傳給學弟妹的資產，也是整個專題最穩的保險——即使後面階段全部卡住，你手上仍有一份能寫進報告的東西。

必做的量化評估

- 軌跡精度：以閉環一致性或 AprilTag 提供的近似真值（第 24.3 節）評估。請一併閱讀 31.8 節：只用閉環一致性是不夠的。
- 地圖品質：牆面厚度、樓層結構是否正確、跨樓層是否對齊。
- 辨識準確率：YOLO 模型在機器人視角測試集上的精度，並與人類視角資料集的表現對照。
- 任務完成率：設定十個巡檢點，統計自主抵達並成功採集資料的比例。

進階挑戰

- 多機器人協作：兩台以上機器人分頭巡檢再融合地圖（第 27 章），比較與單機的效率差異。
- 跨時間變化偵測：間隔數週重複巡檢同一路線，自動比對找出環境的變化（第 23 章 Atlas 的多階段作業）。
- 語言任務介面：讓使用者以自然語言指定巡檢任務（第 24.5 節），直接銜接第 34 章的語言導航。

安全與倫理提醒

校園巡檢涉及公共空間與人員，執行前務必注意：取得場域使用許可、避開人流尖峰、機器人明顯標示、緊急停止機制、以及影像資料的隱私規範（第 27.7 節）。

影像資料的處理要特別留意：巡檢錄影必然會拍到師生。建議在資料進入訓練集之前就做人臉模糊，並在報告中說明你的處理方式——這一段寫得好，評審一定會注意到。

這些不是形式，而是專業工程師的基本素養。

31.5　專題五：視覺導航無人機

這個專題結合第 23、26 章的視覺慣性導航，難度高但成果吸引力極強。

專題目標

讓無人機在 GPS 拒止的室內環境自主定位與飛行，並完成指定的航線或探索任務。

四個階段

- 階段一——資料集分析：先以 EuRoC 或 TUM-VI 資料集執行 ORB-SLAM3，熟悉系統行為與評測方法（第 26.6 節）。不需硬體即可完成。
- 階段二——懸停穩定：以微型無人機在防護網內測試光流定位懸停，量測漂移範圍（第 26.5 節的安全原則）。
- 階段三——VIO 整合：讓 VIO 輸出的位姿接入飛控，實現以視覺定位為基礎的位置保持與簡單航線飛行。
- 階段四——自主任務：執行指定航線、簡單避障、或探索建圖任務。

必做的量化評估

- 懸停精度：定點懸停 60 秒的水平漂移範圍。

- 軌跡誤差：以 AprilTag 或動作捕捉提供真值，計算 ATE。
- 初始化時間：從啟動到 IMU 初始化完成所需的時間與運動量（第 23.4 節）。
- 失效邊界：逐步提高飛行速度，記錄系統開始不穩定的臨界速度（第 26.4 節）。

進階挑戰

- 影像品質自適應：實作依影像清晰度動態調整視覺與慣性權重的機制（第 26.4 節第五項對策）。
- 地空協作：與地面機器人協同探索，比較各自的覆蓋能力（第 25.7、27.1 節）。
- 自製資料集：錄製自己校園場域的視覺慣性資料集並公開，供後續學生使用。

> **安全是第一優先**
>
> 第 26.5 節的安全原則在此必須嚴格遵守：防護網或防護籠、微型平台優先、隨時可用的緊急停止、分階段測試、人員位置管理。
>
> 任何自主飛行的測試，都應先在繫繩或手動可隨時接管的狀態下驗證。跳過這個步驟是意外的主因。
>
> 專題的成功不值得用任何人的安全去換。把「今天不飛」列為一個隨時可以做的決定，並且事先講好什麼情況下一定不飛（風大、人多、電量不足、有人沒戴護目鏡）。

31.6　專題六：地磁 + IMU + 視覺融合定位

這是難度最高、也最能展現整合能力的專題，涵蓋第 5 至 7 章的感測器與第 11 章的融合理論。

專題目標

設計並實作一套多感測器融合的定位系統，量化各感測器的貢獻，並在多種環境中驗證融合

的價值。

四個階段

- 階段一——單感測器基準：分別以地磁、IMU、視覺（或光達）各自定位，量測各自的誤差特性與失效情境。這一步建立比較的基準。
- 階段二——兩兩融合：實作 IMU 加地磁（第 6.6 節的 AHRS）、IMU 加視覺（第 23 章的 VIO），量化融合帶來的改善。
- 階段三——完整融合：以 EKF 或圖優化整合三者（第 11、12 章），並加入依情境調整權重的機制（第 5.6 節的信任度切換）。
- 階段四——多環境驗證：在明亮教室、昏暗機房、鋼構旁、戶外等環境測試，驗證融合系統是否在各環境都優於單一感測器。

必做的量化評估

- 各配置的軌跡誤差：單一感測器與各種融合組合，在各環境的 ATE 比較表。
- 失效次數：各配置在各環境的跟丟或發散次數。
- 權重變化：記錄融合系統在不同環境中，各感測器權重的動態變化，驗證機制是否如預期運作。
- 運算成本：各配置的運算負擔，評估融合的代價。

這個專題的研究價值

本專題最有價值之處，在於它能產出一張「感測器在什麼環境該相信誰」的實測地圖。這不只是課堂練習——它正是第 4.7、5.6、7.6、30.7 節反覆討論的融合原則，在你自己的場域中的具體答案。

建議把成果整理成完整的研究報告：實驗設計、資料、圖表、分析與結論。這樣的報告已具備投稿學生研討會或作為研究所申請素材的水準。

與第九篇的銜接

完成這個專題的同學，已經具備進入第九篇的所有基礎。第 32 章要討論的語意 SLAM、第 33 章的模擬訓練與 Sim-to-Real、第 34 章的 VLM 語言導航，都是在這套穩固的幾何定位之上，再疊加理解與決策的能力——而幾何層若不穩，上面那幾層就是空中樓閣。

31.7　評分 Rubric：五個向度

本節提供一套完整的評分標準。它既是教師評量的依據，也是同學自我檢核的清單——建議在專題開始時就閱讀，而非最後才看。

向度	權重建議	評什麼
地圖品質	25%	尺寸誤差、牆面清晰度、閉環一致性、完整性
定位誤差	25%	軌跡精度、重複性、失效次數
系統穩定性	20%	長時間運行、從失敗恢復、對環境變化的耐受
報告表達	20%	問題定義、方法說明、數據呈現、分析深度
程式碼品質	10%	可讀性、模組化、文件、可重現性

表 31-3　專題評分的五個向度

一、地圖品質（25%）

- 優秀：尺寸誤差小於 2%、牆線清晰無重影、迴路完美閉合、覆蓋完整無異常空洞，並能說明殘餘誤差的來源。
- 良好：尺寸誤差小於 5%、牆線大致清晰、迴路閉合、僅有少量可解釋的空洞。
- 及格：地圖可辨識環境結構，但有明顯變形或重影，且能指出問題所在。
- 待改進：地圖嚴重扭曲或不完整，且無法說明原因。

二、定位誤差（25%）

- 優秀：有真值比對的量化數據（ATE / RPE），誤差在合理範圍，並分析誤差隨時

間與環境的變化。

- 良好：有量化評估（如閉環落差、尺寸比對），數據合理。
- 及格：有定性評估（目視軌跡合理），但缺乏量化。
- 待改進：未做任何定位評估。

三、系統穩定性（20%）

- 優秀：能長時間穩定運行、跟丟後能自動恢復、並主動測試過失效邊界（如第 30.6 節的五種情境）。
- 良好：能完成完整任務，偶有失效但能人工恢復。
- 及格：能在理想條件下完成任務。
- 待改進：僅能片段運作，無法完成完整流程。

四、報告表達（20%）

- 優秀：清楚定義問題、說明方法選擇的理由、以圖表呈現數據、深入分析結果、並誠實討論限制與失敗的嘗試。
- 良好：結構完整、數據清楚、有基本分析。
- 及格：能說明做了什麼，但缺乏分析與反思。
- 待改進：僅有操作紀錄，無法看出理解程度。

五、程式碼品質（10%）

- 優秀：結構清晰模組化、有註解與說明文件、版本控制良好、他人能依文件重現結果。
- 良好：程式可執行、有基本註解與說明。
- 及格：程式可執行，但難以閱讀或缺乏說明。
- 待改進：程式無法執行或無法取得。

六、特別強調：誠實報告失敗

評量中一個常被忽略卻極重要的原則：誠實報告失敗的嘗試，應該加分而非扣分。

一份寫著「我們嘗試了 A 方法，因為某某原因失敗；分析後改用 B 方法，並驗證了改善」的報告，展現的理解深度遠高於只寫「我們用 B 方法，得到很好的結果」。前者展現了工程判斷，後者可能只是照抄教學。

本書從第一章就反覆討論「常見失敗」，正是這個精神。教師在評量時應明確傳達這個標準，鼓勵同學誠實面對困難。

七、展示的建議

- 三分鐘能講完核心：先說「解決什麼問題、怎麼做、結果如何」，細節放後面。
- 用影片而非現場演示：現場演示風險高（硬體故障、環境不同），事先錄好的影片更可靠，也能剪出重點。
- 數據要有對照：「誤差 5 公分」沒有意義，「誤差 5 公分，而未校正前是 30 公分」才有說服力。
- 準備一個「如果重來」：能說出「若有更多時間，我會如何改進」，展現的是持續思考的能力。

31.8　評估方法本身可靠嗎

31.7 節的前兩個向度合計佔了一半的分數，而它們都建立在一個假設上：同學報上來的數字，真的反映了系統的好壞。本節要檢查這個假設。

這不是吹毛求疵。在實際的專題評審中，最常見的狀況不是同學沒做評估，而是做了一個「看起來很科學、但其實證明不了任何事」的評估。以下兩個實驗各揭露一種這樣的陷阱。

一、陷阱一：閉環落差為零，不代表地圖是對的

閉環落差——繞一圈回到起點，看地圖上起終點差多少——是同學最愛用的指標，因為它不需要真值設備，拿捲尺就能量。問題是：它只對某一類錯誤敏感。

實驗設定：在一條 12 公尺 × 8 公尺的橢圓形迴路上模擬兩種里程計誤差。家族 A 是尺度偏差（輪徑量錯，走的距離一致地偏大或偏小 0.5% 到 6%）；家族 B 是航向隨機游走（每步的角度量測帶有隨機誤差，逐步累積）。每個家族各 150 次試驗，對每一次都算出三個數字：閉環落差、真實 ATE（與真值軌跡逐點比對）、以及地圖尺寸誤差。

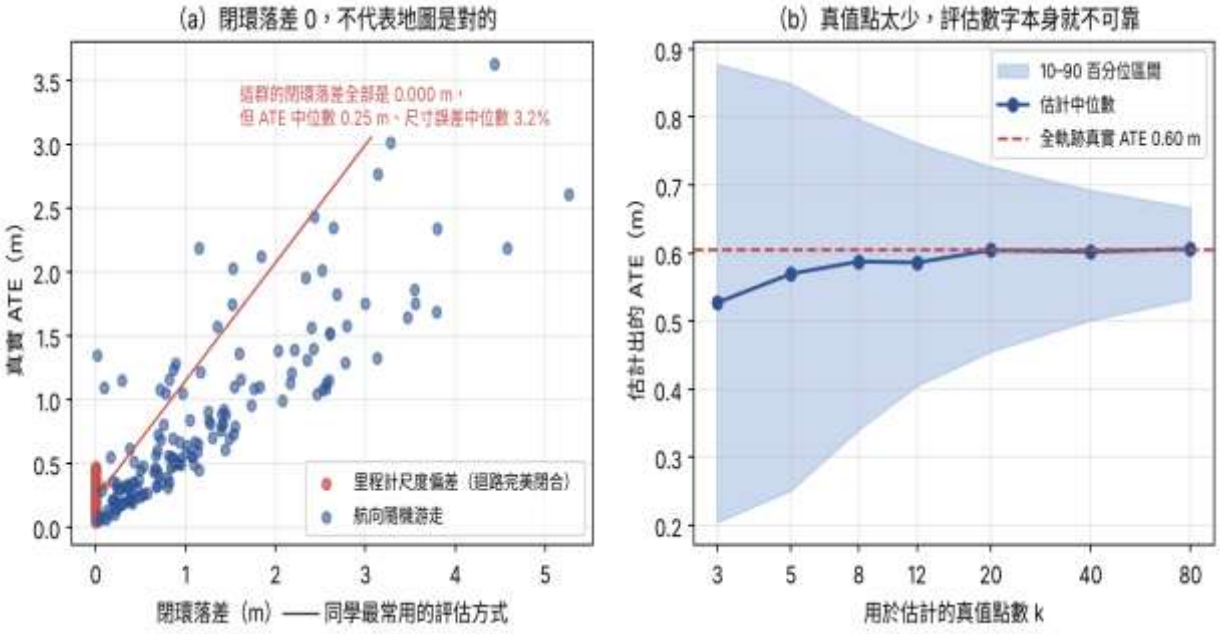


圖 31-2　兩種評估方式各自看得見什麼

（a）每一個點是一次試驗。紅點是尺度偏差家族，藍點是航向游走家族。（b）同一條軌跡，改變用於估計 ATE 的真值點數 k，各重複 800 次抽樣。

（a）的紅點全部擠在橫軸的零點上：尺度偏差家族的閉環落差在數值上是 0.000 公尺（最大 1.5×10^{-14} 公尺，純粹是浮點誤差）。理由很單純——把整張地圖等比例放大或縮小，迴路還是會閉合。但這些地圖的真實 ATE 中位數是 0.25 公尺，尺寸誤差中位數 3.2%、最大到 6.0%。依 31.7 節的標準，一張尺寸誤差 6% 的地圖連「良好」都勾不到，可是它的閉環一致性是滿分。

藍點則呈現另一種樣貌：閉環落差與 ATE 的相關係數 0.84，確實是個有用的指標。把兩個

家族合在一起看，相關係數 0.88 看起來更漂亮，但這個數字是假的——它完全來自藍點那一群，紅點那一整排垂直的點對相關係數幾乎沒有貢獻，卻正是最容易被漏掉的錯誤。

結論：兩個指標要一起看，因為它們互補

閉環落差看得見：航向誤差、掃描匹配的累積漂移、迴路偵測失敗。看不見：尺度偏差。

尺寸誤差（捲尺比對）看得見：尺度偏差。看不見：局部的扭曲與重影——地圖可以長寬都對，中間卻歪掉。

這就是 31.1 節要求「尺寸誤差、閉環一致性、目視檢查、導航成功率」四項一起做的原因。四項各自都有盲區，但盲區不重疊。

若能借到真值設備（動作捕捉、全站儀、或第 24.3 節的 AprilTag 方案），就直接算 ATE / RPE——但請先讀下一小節。

二、陷阱二：真值點太少，ATE 這個數字本身就在晃

假設你借到了設備，可以在場域中佈幾個真值點。問題來了：幾個才夠？

圖 31-2（b）用的是家族 B 的一條代表性軌跡，它的真實 ATE 是 0.60 公尺（用全部 1200 個取樣點算出來）。接著模擬「只量得到 k 個真值點」的情況：從軌跡上隨機抽 k 點算 ATE，每個 k 重複 800 次，畫出估計值的 10 到 90 百分位區間。

結果：k = 3 時，估計值的 10–90 區間是 0.20 到 0.88 公尺——同一套系統，可以報出 0.2 也可以報出 0.9，差了四倍多。k = 8 時區間收到 0.34–0.80，k = 20 時 0.46–0.73，k = 80 時才收到 0.53–0.67。

這代表什麼？如果你用三個真值點量出 ATE = 0.25 公尺，而隔壁組用三個真值點量出 0.55 公尺，你不能說你的系統比較好——這兩個數字都在同一個分布裡，差異完全可能只是抽樣運氣。要讓「我的系統優於對照組」這句話站得住，真值點的數量必須多到區間不重疊為

止。

真值點數 k	ATE 估計的 10–90 區間（m）	相對真值 0.60 m 的偏離
3	0.20 – 0.88	−67% ~ +45%
5	0.25 – 0.85	−58% ~ +41%
8	0.34 – 0.80	−44% ~ +32%
12	0.40 – 0.76	−33% ~ +26%
20	0.46 – 0.73	−25% ~ +20%
40	0.50 – 0.69	−17% ~ +15%
80	0.53 – 0.67	−12% ~ +10%

表 31-4　真值點數與 ATE 估計的不確定度

實務建議：若要做組間或方法間的比較，至少 20 個真值點；若只是要報一個概略的量級，8 到 12 點可以接受，但報告中必須寫明點數，並說明這個數字帶有約 ±30% 的不確定度。把不確定度寫出來，不會讓分數變低——依 31.7 節的標準，那是「優秀」等第的特徵。

三、這兩個實驗要同學帶走的事

1. 評估方法有盲區，而且盲區是方法的性質，不是運氣。選一個指標之前，先問它對哪一類錯誤不敏感。
2. 用兩個以上盲區不重疊的指標，比把一個指標做得很精細更有效。
3. 任何估計值都有不確定度，包括你用來證明系統很好的那個數字。樣本太少時，兩組數字的差異可能完全沒有意義。
4. 在報告中寫出「我的評估方法看不見什麼」，比多報三個小數位更能顯示你真的懂。

給教師的課程規劃建議

本節提供三種課程長度的專題安排建議，供教師參考。

課程類型	時數	建議專題	重點調整
高中選修	18 週 × 1–2 小時	專題一（模擬）或三	以模擬與紙上活動為主，不碰原始碼
大學部	18 週 × 3 小時	專題一 + 二 或 三	完整走過建圖、定位、導航、評估
專題 / 研究	一至兩學期	專題四、五或六	強調研究方法、量化比較與報告撰寫

表 31-5　三種課程的專題安排建議

時程規劃的原則

- 前三分之一講理論與工具：讓同學備妥環境與基礎知識，此時可穿插小型的模擬練習維持動機。
- 中間三分之一做基礎實作：完成第 29、30 章的引導式實作，建立成功經驗。
- 最後三分之一做專題：此時同學已有能力自主探索，教師轉為顧問角色。
- 預留緩衝：實作課程的意外極多（硬體故障、環境問題、相依性錯誤）。建議預留至少兩週的緩衝時間——下一節會說明「至少兩週」這個數字其實還偏樂觀。

分組的建議

- 三到四人一組較理想：太少則工作量過重，太多則容易有人搭便車。
- 明確分工但輪替：例如分為建圖、導航、評估、報告四個角色，但中途輪替，確保每人都接觸到完整流程。
- 把「評估」獨立成一個角色：這是最常被犧牲的工作，因為它不像建圖那樣有立即的成就感。指定專人負責，並要求他在每個里程碑都交出數據。

案例

案例 31-1　一張滿分的地圖，錯了 4%

某組的專題一成果：教室地圖牆線筆直、無重影、繞行一圈起終點完全重合，目視 check 無可挑剔，組員在報告中寫下「閉環誤差為零，地圖品質優秀」。

評審拿捲尺量了教室：實際長 9.2 公尺，地圖上量出 8.83 公尺，誤差 4.0%。原因是輪徑參數用了規格書上的標稱值，而輪胎因為充氣不足實際半徑偏小——這正是圖 31-2 中紅點那一族的真實版本。

這個案例的教訓不是「同學偷懶」，恰恰相反：他們做了評估，只是做的那一項對這種錯誤沒有感覺。補救也很簡單——第 29.3 節的里程計校正程序跑一次，重新建圖後尺寸誤差降到 0.8%。整件事花不到一個下午，但如果沒有人拿捲尺去量，這張地圖會一路帶著 4% 的誤差進到導航階段，然後在 Nav2 反覆錯過目標點時，讓他們去懷疑導航參數。

案例 31-2　沒有對照組的「改善」

某組在專題三中實作了一套改良的覆蓋規劃，報告寫「覆蓋率達 97%，效果良好」。評審問：原本的方法是多少？答不出來。

這是最常見的失分方式。97% 這個數字本身不帶任何資訊——從圖 31-1 可以看到，在 12% 障礙的房間裡，最陽春的弓字形走到 1.23 倍距離就超過 95%，而隨機碰撞走到 8 倍也達不到。97% 到底是了不起還是理所當然，取決於你跟誰比、在什麼條件下比。

正確的寫法是：「在相同的房間配置與相同的行走距離預算下，基準弓字形覆蓋 92.4%、本方法 97.1%，在 20 組隨機配置中本方法有 17 組較優。」這句話裡有對照組、有控制變因、有重複次數，三者缺一，結論就只是一個感想。

案例 31-3　三種典型的專題失敗模式

模式一：目標過大

開學時信心滿滿地設定「做一台能自主巡檢全校的機器狗」，期末交出一份只有模擬的報告。問題不在志向，而在沒有定義「最小可行成果」——當第一個階段就卡住時，沒有任何東西可以撤退。

模式二：環境問題吃掉時間

相依性衝突、驅動程式不相容、硬體到貨延遲、場域借不到。這些事情單一來看都是小事，加起來卻能吃掉半個學期。下一節的模擬會告訴你：這不是運氣差，這是常態。

模式三：沒有量化評估

做了很多、也真的跑起來了，但整份報告只有照片與操作紀錄。依 31.7 節的標準，這樣的專題在地圖品質與定位誤差兩個向度上合計 50% 的分數會嚴重失血，而這五成正是最容易補起來的——一把捲尺加一個下午就能把「及格」拉到「良好」。

這三種模式的共同解方是：在專題開始時就想清楚「最後要交出什麼」。把 31.7 節的評分標準與圖 31-4 的檢核表放在眼前，逆推每個階段該做什麼——這是專案管理的基本功，也是同學進入業界或研究所後最實用的能力之一。

案例 31-4　時程要留多少緩衝：一次模擬

「預留兩週緩衝」是常見的建議，但兩週是怎麼來的？這一節用蒙地卡羅模擬給一個有依據的答案。

模型設定：一個專題切成四個里程碑，規劃工期分別是 3、4、4、3 週，合計 14 週。每個里程碑的實際工期服從對數常態分布，中位數等於規劃值——也就是說，規劃並沒有系統性低估，只是有不確定度。以 σ 代表不確定度的大小：σ = 0.25 是順利的軟體專題，σ = 0.45 是一般情況，σ = 0.65 是牽涉硬體、場域與外部相依的專題。每種設定跑 40000 次。

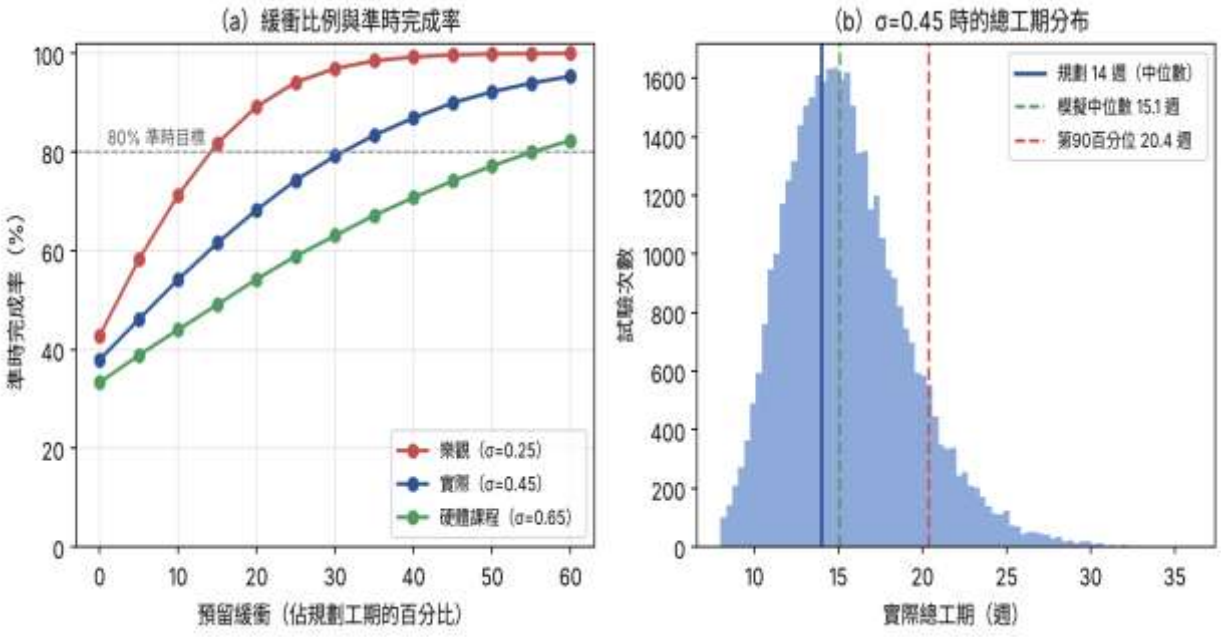


圖 31-3　緩衝比例與準時完成率

（a）橫軸是預留緩衝佔規劃工期的百分比，縱軸是在緩衝內完成的機率。（b）是 σ = 0.45 時總工期的分布。

一、三個反直覺的數字

1. 不留緩衝，準時機率只有三到四成。σ = 0.45 時是 37.9%，σ = 0.65 時只剩 33.3%。注意這裡每個里程碑的中位數都等於規劃值——沒有人低估，但合起來仍然大概率超時。
2. 原因是分布不對稱。看（b）的直方圖：左邊被 0 擋住，右邊拖著一條長尾。一個階段提早完成最多省幾天，但卡住可以卡三週。所以總工期的中位數 15.1 週已經高於規劃的 14 週，而第 90 百分位是 20.4 週——規劃值的 1.46 倍。
3. 要達到 80% 準時，σ = 0.45 需要約 30% 的緩衝（14 週要變 18.2 週），σ = 0.65 需要約 55%。換算回來：一個 14 週的專題，「兩週緩衝」只能撐到 55% 的準時率；要有八成把握，軟體專題要留四週，硬體專題要留將近八週。

二、這對專題規劃的四個意涵

- 緩衝放在最後，不要分散到各階段。分散的緩衝會被帕金森定律吃掉（工作會膨脹

填滿可用時間）；集中的緩衝才看得見剩多少。

- 定義最小可行成果，而且要在第一個里程碑就達成。模擬告訴我們超時是常態，所以必須保證「超時的時候手上已經有東西」。
- 把不確定度最高的階段排在前面。硬體到貨、場域申請、相依性安裝這些 σ 很大的事，越早踩到坑，剩下的時間越能反應。
- 每個里程碑結束就重估。第一個階段若花了 1.5 倍時間，那是你這個專題 σ 偏大的證據，後面三個階段的預期也要跟著上修，而不是指望它們會「補回來」。

這個模型的限制（也請同學在自己的報告中這樣寫）

模型假設四個階段的工期彼此獨立。實際上它們常常相關——硬體不穩定會同時拖慢好幾個階段，這會讓長尾更重，所以上面的數字是偏樂觀的估計。

模型也假設各階段的中位數等於規劃值。若規劃本身偏樂觀（多數人都是），實際情況會比圖 31-3 更差。

對數常態是一個方便的選擇，不是從資料擬合出來的。若你手上有前幾屆專題的實際工期紀錄，換上真實分布重跑一次，那會是一份相當有價值的課程改進報告。

實作活動

實作 31-1　專題規劃與提案

目的：在正式投入專題之前，完成一份完整的規劃提案。這個練習本身就是重要的訓練——多數專題的成敗，在規劃階段就已決定。

第一部分：選題與可行性評估

- 依表 31-2 選出兩到三個候選專題，對每一個評估：所需硬體是否可取得？所需能力是否具備（對照第 28.7 節的五階段）？時間是否充足？

- 對最有興趣的專題，列出「最小可行成果」與「理想完整成果」。前者是必定要達成的底線，後者是行有餘力的目標。
- 風險評估：列出三個最可能失敗的環節，並為每個準備備案。例如「若實體小車無法運作，改以模擬完成階段二」。

第二部分：里程碑規劃

- 把專題切分成三到五個里程碑，每個里程碑明確定義：要完成什麼、產出什麼證據、預計花多少時間。
- 在時程表上標出檢核點。緩衝比例請依案例 31-4：純軟體或模擬的專題留 30%，牽涉硬體、場域或外部相依的專題留 50% 以上，並把緩衝集中放在最後。
- 對每個里程碑，列出「若這一步卡住超過一週，我會怎麼辦」的應對方案。

第三部分：評估方法設計（關鍵）

- 依 31.7 節的五個向度，為你的專題設計具體的評估方法。例如地圖品質要量什麼、定位誤差用什麼當真值。
- 依 31.8 節檢查你選的指標：它對哪一類錯誤不敏感？你有沒有第二個盲區不重疊的指標來補？
- 估算真值點數：你打算量幾個點？依表 31-4，這個點數下你的 ATE 估計有多大的不確定度？這個不確定度小到足以支撐你想下的結論嗎？
- 設計對照組：你的成果要與什麼比較？（不同方法？不同參數？有無某個元件？）沒有對照，數據就沒有意義（案例 31-2）。
- 規劃資料收集：每個里程碑要收集哪些數據？用什麼格式記錄？這能避免最後才發現數據不足（案例 31-3 的模式三）。

第四部分：提案發表

- 把上述內容整理成一份兩頁的提案，並準備五分鐘的口頭報告。

- 向同學與教師報告，重點在說服他們：這個題目有價值、你有能力完成、而且你知道會遇到什麼困難。
- 接受提問並修正提案。他人的質疑往往能點出你的盲點——這比自己埋頭做三週才發現問題有效率得多。

討論問題

1. 你的「最小可行成果」是什麼？若時間只剩一半，你會捨棄哪些部分？
2. 你設計的評估方法中，真值（ground truth）從何而來？若沒有真值，你如何論證成果的品質？
3. 你的指標看不見哪一類錯誤？用什麼補？（對照圖 31-2）
4. 你的對照組是什麼？為什麼這個對照能凸顯你的成果價值？
5. 依案例 31-3 的三種失敗模式，你的專題最可能踩到哪一種？你的預防措施是什麼？
6. 依案例 31-4，你的專題 σ 大概落在哪一檔？你留的緩衝夠嗎？

延伸挑戰

把你的提案寫成正式的研究計畫格式：研究背景與動機、文獻回顧（可直接引用本書各章的延伸閱讀）、研究方法、預期成果、時程規劃、參考文獻。這份文件不只是專題的起點，也是日後撰寫研究所申請資料、國科會大專生研究計畫、或競賽企劃書的基礎。撰寫這份文件的過程，本身就會逼你把想法釐清——這是本活動最大的價值。

實作 31-2　把評估方法做一次壓力測試

目的：用本章 31.8 節的方法，檢驗你自己打算用的評估方式。這個活動只需要一台電腦，一節課可以做完，但做過的同學通常會回頭改掉原本的評估設計。

1. 取一條你自己的軌跡資料（或用模擬產生一條繞一圈的軌跡當作真值）。
2. 人為加入兩種錯誤：（甲）把所有步長乘以 1.03，模擬輪徑量錯；（乙）在每一步的航向加上平均為零的隨機誤差並累積。兩種各產生 50 條軌跡。

3. 對每一條算三個數字：閉環落差、尺寸誤差、以及與真值比對的 ATE。
4. 畫出「閉環落差 vs ATE」與「尺寸誤差 vs ATE」兩張散布圖，並分別計算相關係數。
5. 回答：你原本打算用的那個指標，對哪一種錯誤是瞎的？要加上哪一個指標才補得起來？
6. 進階：從你的軌跡中隨機抽 k 個點估計 ATE（k 取 3、5、10、20、50），每個 k 重複 500 次，畫出估計值的分布。你實際打算佈幾個真值點？那個點數下，你有把握區分「誤差 0.3 公尺」與「誤差 0.5 公尺」兩套系統嗎？

這個活動的產出——兩張散布圖加一段說明——可以直接放進專題報告的「評估方法」一節。依 31.7 節的標準，能寫出「我知道我的指標看不見什麼」的報告，在報告表達這個向度上就已經站到「優秀」的位置。

延伸閱讀

- J. Sturm et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," IROS, 2012.（ATE 與 RPE 評測方法的權威來源，TUM RGB-D 資料集的說明論文）
- evo 軌跡評測工具（GitHub: MichaelGrupp/evo）。（專題評估的必備工具，支援多種資料集格式與圖表輸出）
- Z. Zhang and D. Scaramuzza, "A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry," IROS, 2018.（軌跡評測的完整教程，含尺度對齊等細節；本章 31.8 節的問題意識與這篇一致）
- C. Cadena et al., "Past, Present, and Future of SLAM," IEEE T-RO, 2016.（第 1 章即引用的權威回顧，適合作為專題文獻回顧的起點）
- 本書第 13.7 節（地圖品質評估）、第 17 章案例 17-1（牆面厚度指標）、第 22.7 節（軌跡評測）。（專題評估方法的內部參照）

- S. Macenski et al., "The Marathon 2: A Navigation System," IROS, 2020.（Nav2 論文，含系統性能評估的方法論可供參考）
- E. Galceran and M. Carreras, "A Survey on Coverage Path Planning for Robotics," Robotics and Autonomous Systems, 2013.（覆蓋路徑規劃的回顧，專題三的理論依據）
- 各校大專學生研究計畫與專題競賽的優秀作品集。（觀摩他人如何組織一份完整的專題報告）

兩張可以貼在桌前的圖

圖 31-4　專題從規劃到展示的完整流程

【規劃階段】

① 依表 31-2 選定專題 → ② 評估資源與能力 → ③ 切分三至五個里程碑

④ 依案例 31-4 決定緩衝比例（軟體 30%、硬體 50% 以上，集中在最後）

↓

【執行階段】

⑤ 模擬驗證（先把流程與參數調通）

⑥ 實體實作（預留兩倍除錯時間）

⑦ 每個里程碑產出可驗證的證據（第 29.7 節），並重估剩餘時程

↓

【評估階段】

⑧ 依 31.7 節的五向度自我檢核

⑨ 量化評估：ATE / RPE / 尺寸誤差 / 覆蓋率（依專題而定）

⑩ 依 31.8 節檢查指標盲區與真值點數是否足夠

⑪ 誠實記錄失敗的嘗試與原因

↓

【展示階段】

⑫ 撰寫報告：問題、方法、數據、分析、限制

⑬ 準備三分鐘核心說明 + 展示影片

⑭ 準備「如果重來會怎麼做」的反思

圖 31-5　繳交前的自我檢核清單

□ 地圖品質：有量化的尺寸誤差數據嗎？能說明誤差來源嗎？

□ 定位精度：有 ATE 或閉環落差的數據嗎？兩者都量了嗎？

□ 指標盲區：我的指標看不見哪一類錯誤？有補上第二個指標嗎？

□ 真值點數：點數夠支撐我想下的結論嗎？不確定度寫出來了嗎？

□ 系統穩定：測試過失效邊界嗎？知道系統在什麼條件下會壞嗎？

□ 對照組：有與其他方法或參數設定比較嗎？控制變因說清楚了嗎？

□ 失敗紀錄：誠實記錄了失敗的嘗試與解決過程嗎？

□ 證據完整：地圖檔、軌跡圖、錄影、程式碼都齊全嗎？

□ 可重現：他人依你的文件能重現結果嗎？

□ 反思：能說出「如果重來會怎麼做」嗎？

小結

- 六個專題難度遞增：教室 2D 建圖（★）、三維空間掃描（★★）、掃地機器人路徑分析（★★）、機器狗校園巡檢（★★★★）、視覺導航無人機（★★★★）、多感測器融合定位（★★★★★）；選擇時應評估資源、能力與延續性。
- 專題一涵蓋 SLAM 完整循環且成果直觀，評量重點不在「有沒有建出地圖」，而在「能不能說清楚地圖為什麼好或不好」；專題二的產出對系所空間管理與模擬環境有長期價值，也可以直接成為第 33 章模擬訓練的場景。
- 專題三的實驗結果：走完一倍的理論最短距離時，螺旋式覆蓋 83.2%、弓字形 75.6%、隨機碰撞只有 35.3%；要達到 95% 覆蓋，弓字形在無障礙房間需 0.95 倍距離、隨機碰撞需 6.9 倍，而在障礙 12% 以上的房間隨機碰撞走到 8 倍仍未達標。系統性策略的優勢不只是快，而是它有涵蓋性的保證。
- 評分採五個向度：地圖品質（25%）、定位誤差（25%）、系統穩定性（20%）、報告表達（20%）、程式碼品質（10%）；建議在專題開始時就閱讀標準並逆推規劃。

- 評估方法本身有盲區：尺度偏差造成的地圖錯誤，閉環落差完全看不見——實驗中這一族的閉環落差是 0.000 公尺，但 ATE 中位數 0.25 公尺、尺寸誤差中位數 3.2%、最大 6.0%。閉環落差與尺寸誤差必須一起用，因為兩者的盲區不重疊。
- 評估數字本身帶有不確定度：同一條真實 ATE 為 0.60 公尺的軌跡，用 3 個真值點估計的 10–90 區間是 0.20–0.88 公尺，用 20 點才收到 0.46–0.73。要做組間比較，真值點至少 20 個，並在報告中寫明點數與不確定度。
- 時程的不確定度是不對稱的：四個里程碑、規劃 14 週的專題，在一般不確定度下不留緩衝只有 37.9% 準時，總工期中位數 15.1 週、第 90 百分位 20.4 週。要有八成把握，軟體專題約需 30% 緩衝、硬體專題約需 55%，且緩衝應集中放在最後而非分散。
- 專題失敗的三種典型模式為目標過大、環境問題吃掉時間、以及沒有量化評估；共同解方是在開始時就想清楚「最後要交出什麼」，並依評分標準逆推每階段的工作。
- 誠實報告失敗的嘗試應該加分而非扣分——展現發現問題、分析原因、驗證改善的完整歷程，遠比只呈現漂亮結果更能說明理解深度；寫出「我的評估方法看不見什麼」同樣是優秀等第的特徵。
- 展示的建議：三分鐘能講完核心、用預錄影片而非現場演示、數據要有對照組、並準備一個「如果重來會怎麼做」的反思。

第八篇結語

第八篇「ROS / Python / C++ 實作」至此全部完成。從第 28 章的環境建置、第 29 章的光達建圖導航、第 30 章的視覺 SLAM 與跨感測器比較，到本章的專題整合與評量，同學已經走完了從理論到實作的完整路徑。

也請記得本章 31.8 節的那兩張散布圖。前面三十章教的是怎麼讓機器人知道自己在哪裡；這一章補上的是怎麼知道「它真的知道」——而後者比前者更容易被跳過，也更能分辨一份作品的成色。

最後一篇，我們把視野投向未來。傳統 SLAM 解決的是「我在哪裡、地圖長什麼樣」；而 AI 與 VLM 要解決的是「我看到了什麼、這個地方代表什麼、下一步該怎麼做」。第九篇共有三章：第 32 章「語意 SLAM：讓地圖帶有意義」讓幾何地圖升級為語意地圖；第 33 章「模擬訓練與 Sim-to-Real」讓機器人在虛擬環境中學會技能並部署到真實機身；第 34 章「VLM、語言導航與具身智慧」則讓機器人聽懂「請去教務處」這樣的指令，並為全書作結。

你在本章建立的那套評估習慣，到了第九篇只會更重要。語意與語言這一層的失敗，往往不會當場報錯——它只是安靜地把機器人帶到錯的房間。